

Introduction to machine learning for polygenic risk prediction

Post-GWAS course | Aarhus University | 26th Aug. 2026

Simon Wengert
NNF Center for Basic Metabolic Research,
University of Copenhagen

simon.wengert@sund.ku.dk



GWAS under the hood &
the power of the line

GWAS under the hood

A linear regression equation is shown with labels for each component. The equation is $\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x + \varepsilon$. The labels are in red text and connected to the equation by black lines. The label 'response variable' points to $\hat{y}$. The label 'explanatory variable' points to x . The label 'error term' points to ε . The label 'first parameter (intercept)' points to $\hat{\beta}_0$. The label 'second parameter (slope)' points to $\hat{\beta}_1$.

response variable

explanatory variable

error term

first parameter
(intercept)

second parameter
(slope)

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x + \varepsilon$$

GWAS under the hood

response variable explanatory variable

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x + \varepsilon$$

error term

first parameter
(intercept)

second parameter
(slope)

The diagram shows the linear regression equation $\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x + \varepsilon$. Red text labels are connected to the equation by black lines: 'response variable' points to $\hat{y}$, 'explanatory variable' points to x , 'error term' points to ε , 'first parameter (intercept)' points to $\hat{\beta}_0$, and 'second parameter (slope)' points to $\hat{\beta}_1$.

GWAS: a remarkably successful approach

GWAS have helped in mapping the genetic architecture of thousands of complex traits

response variable

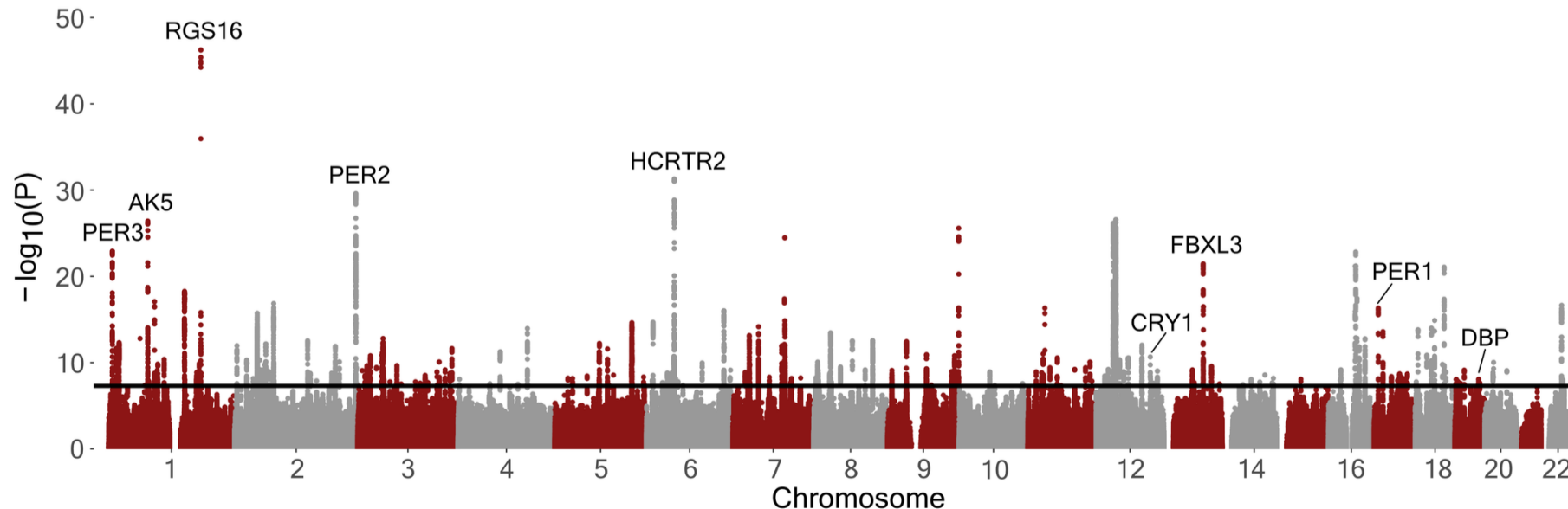
explanatory variable

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x + \varepsilon$$

error term

first parameter
(intercept)

second parameter
(slope)



(Pritchard 2026, An Owner's Guide to the Human Genome)

GWAS: a remarkably successful approach

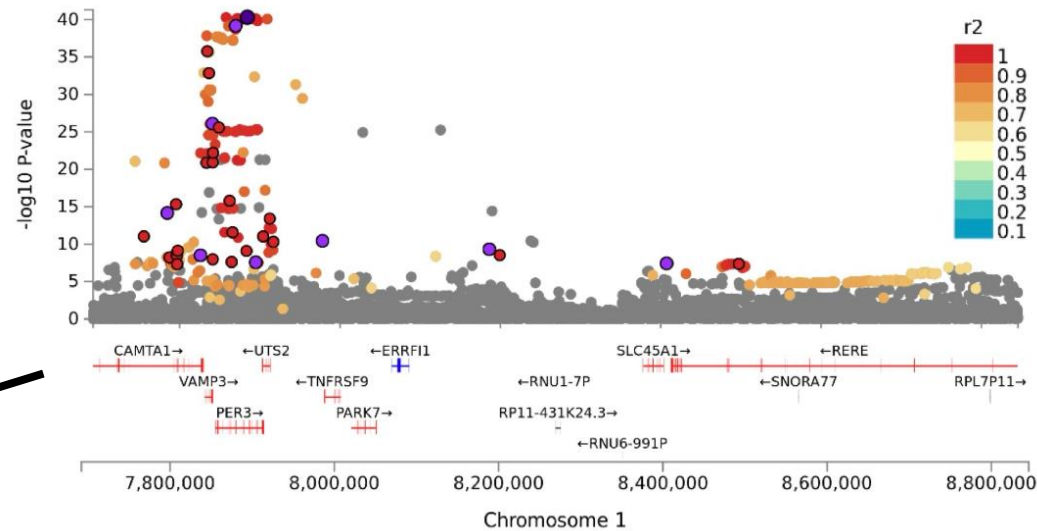
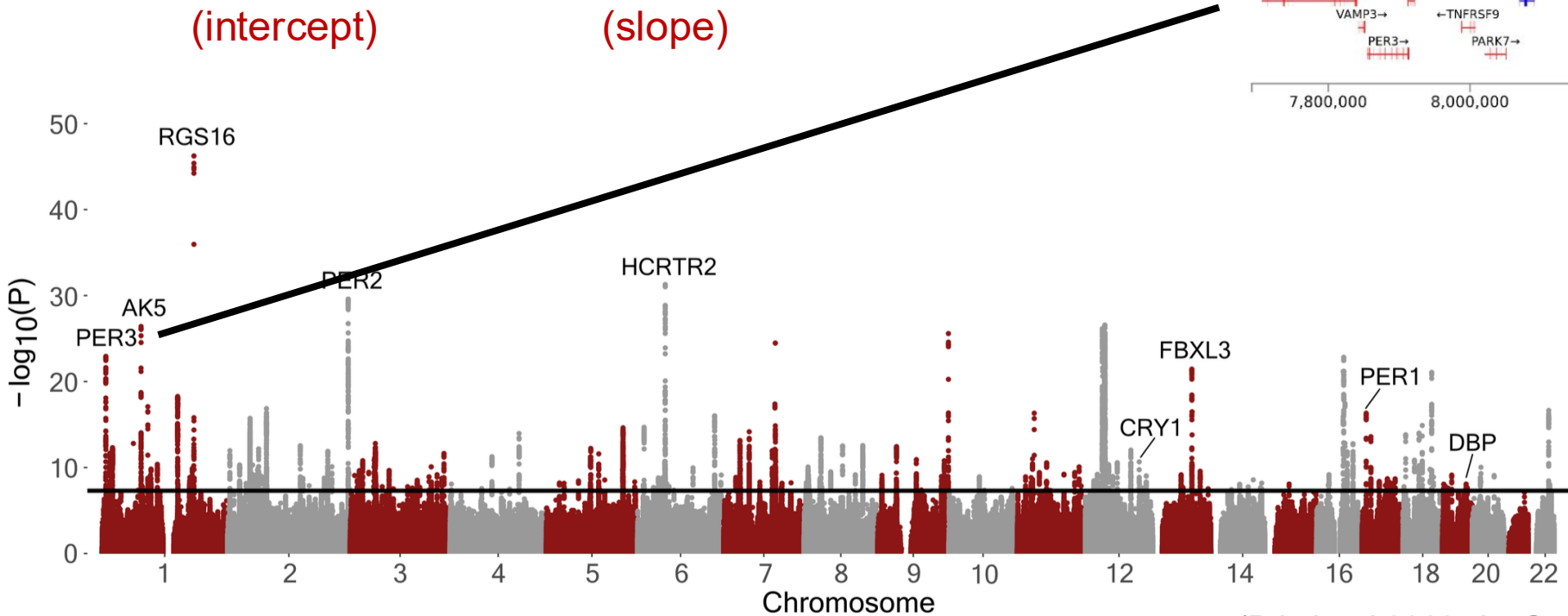
GWAS have helped in mapping the genetic architecture of thousands of complex traits

response variable explanatory variable

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x + \varepsilon$$

error term

first parameter (intercept) second parameter (slope)



(Pritchard 2026, An Owner's Guide to the Human Genome)

GWAS: a remarkably successful approach

GWAS have helped in mapping the genetic architecture of thousands of complex traits

2008



2012



2016



2019



GWAS: **so many signals?!?!**

GWAS have helped in mapping the genetic architecture of thousands of complex traits

2008



2012



2016



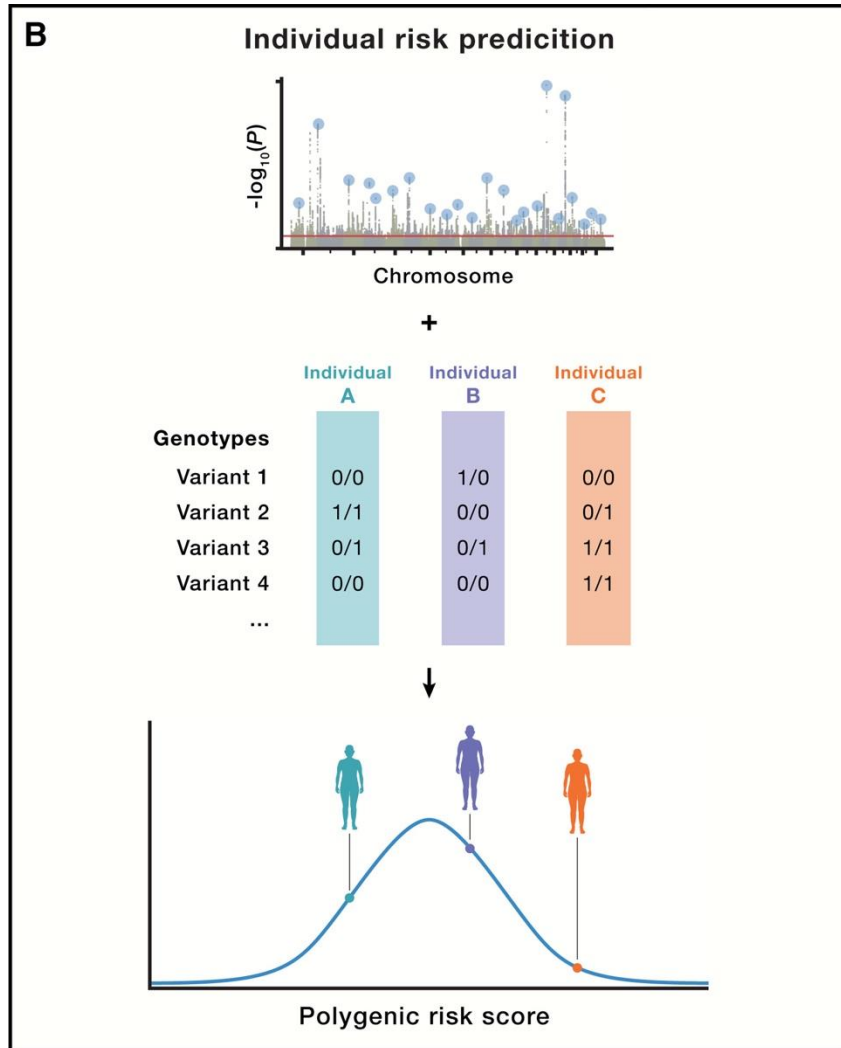
2019



(Pritchard 2026, An Owner's Guide to the Human Genome)

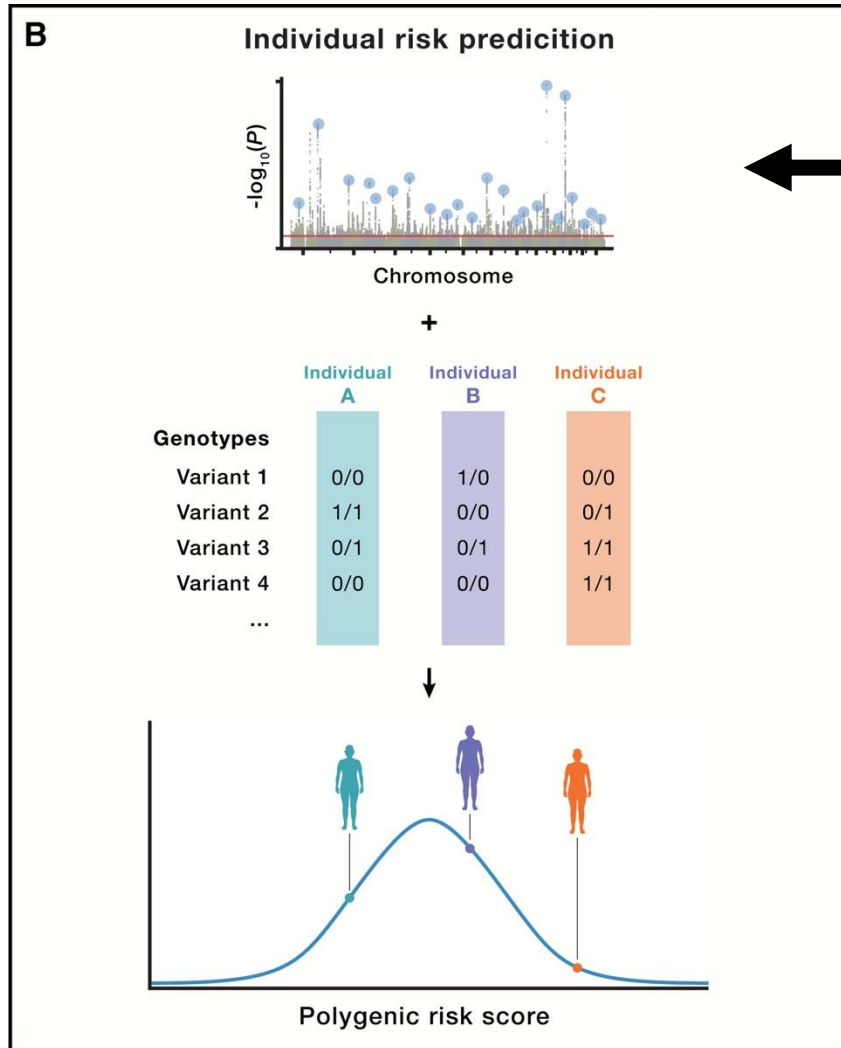
Polygenic scores: a way to aggregate GWAS signals

PGSs are widely used to predict one individual's genetic risk of developing a disease



Polygenic scores: a way to aggregate GWAS signals

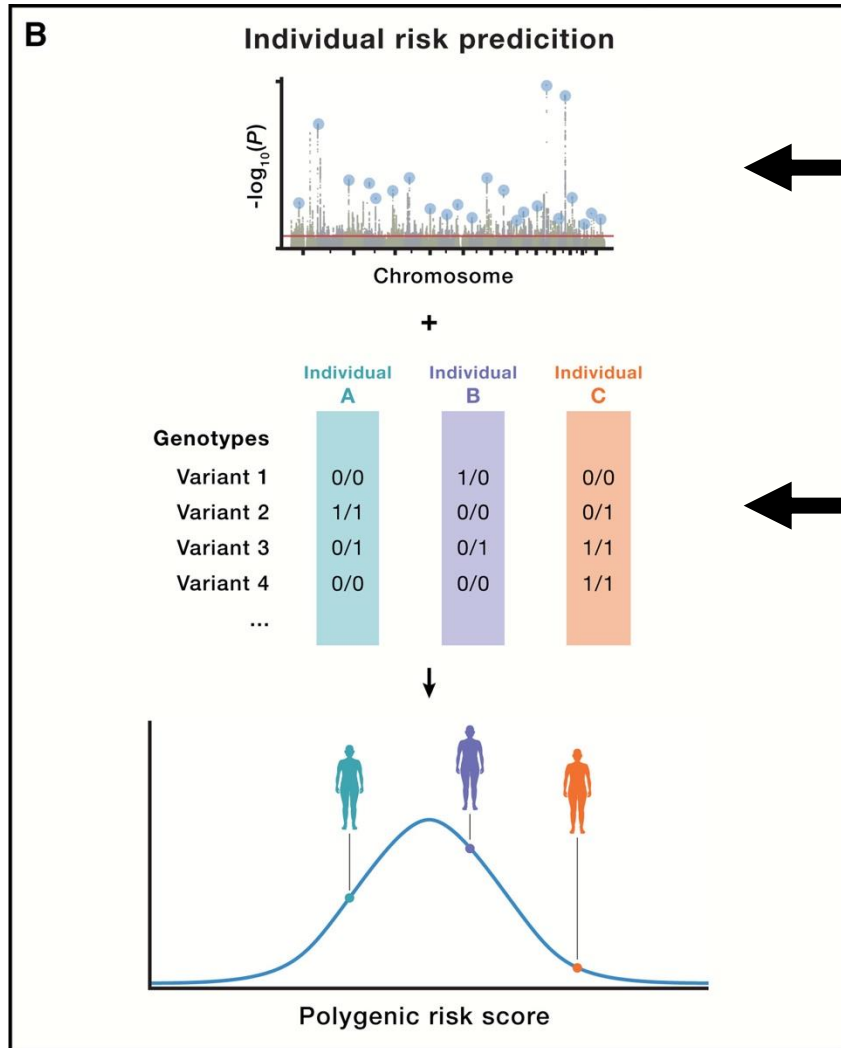
PGSs are widely used to predict one individual's genetic risk of developing a disease



how can we assess the contribution of these GWAS signals...

Polygenic scores: a way to aggregate GWAS signals

PGSs are widely used to predict one individual's genetic risk of developing a disease

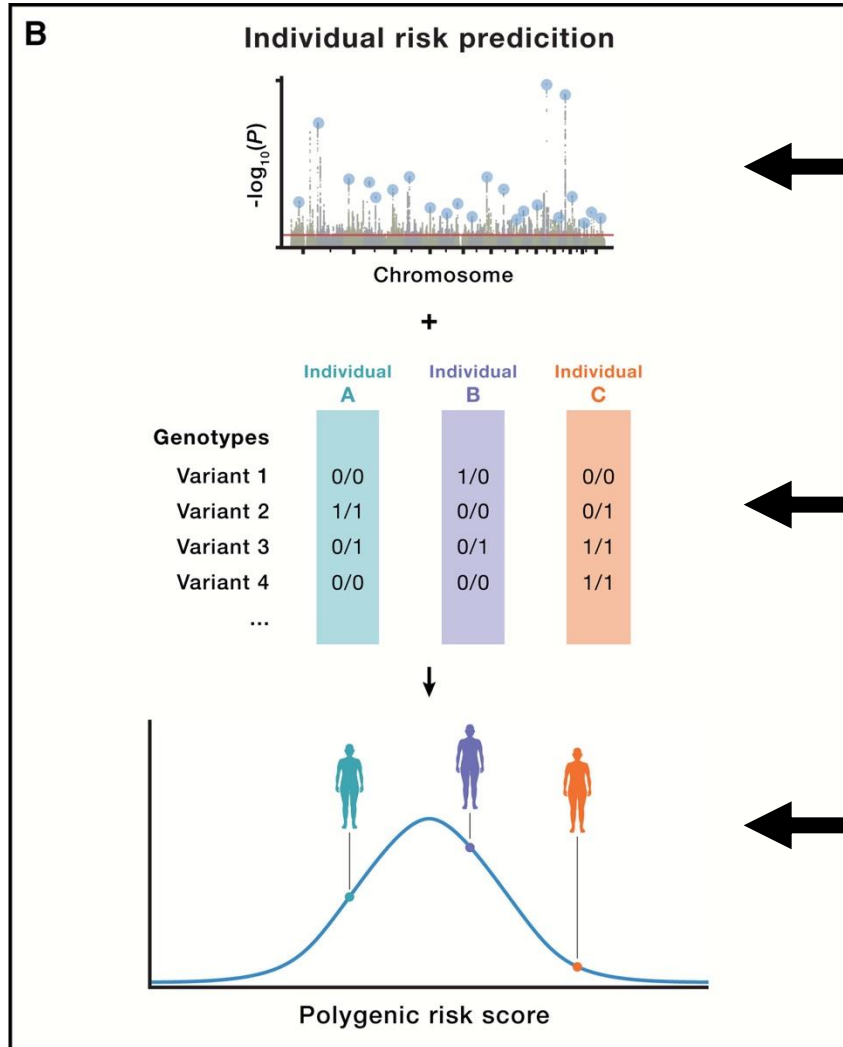


how can we assess the contribution of these GWAS signals...

arising from these variants...

Polygenic scores: a way to aggregate GWAS signals

PGSs are widely used to predict one individual's genetic risk of developing a disease



how can we assess the contribution of these GWAS signals...

arising from these variants...

to these individuals' genetic risk of developing a disease?

Polygenic scores: a way to aggregate GWAS signals

PGSs are widely used to predict one individual's genetic risk of developing a disease

$$PGS_j = \sum_i \beta_i \times G_j^i$$

Polygenic scores: a way to aggregate GWAS signals

PGSs are widely used to predict one individual's genetic risk of developing a disease

effect size from
the GWAS

$$PGS_j = \sum_i \beta_i \times G_j^i$$

Polygenic scores: a way to aggregate GWAS signals

PGSs are widely used to predict one individual's genetic risk of developing a disease

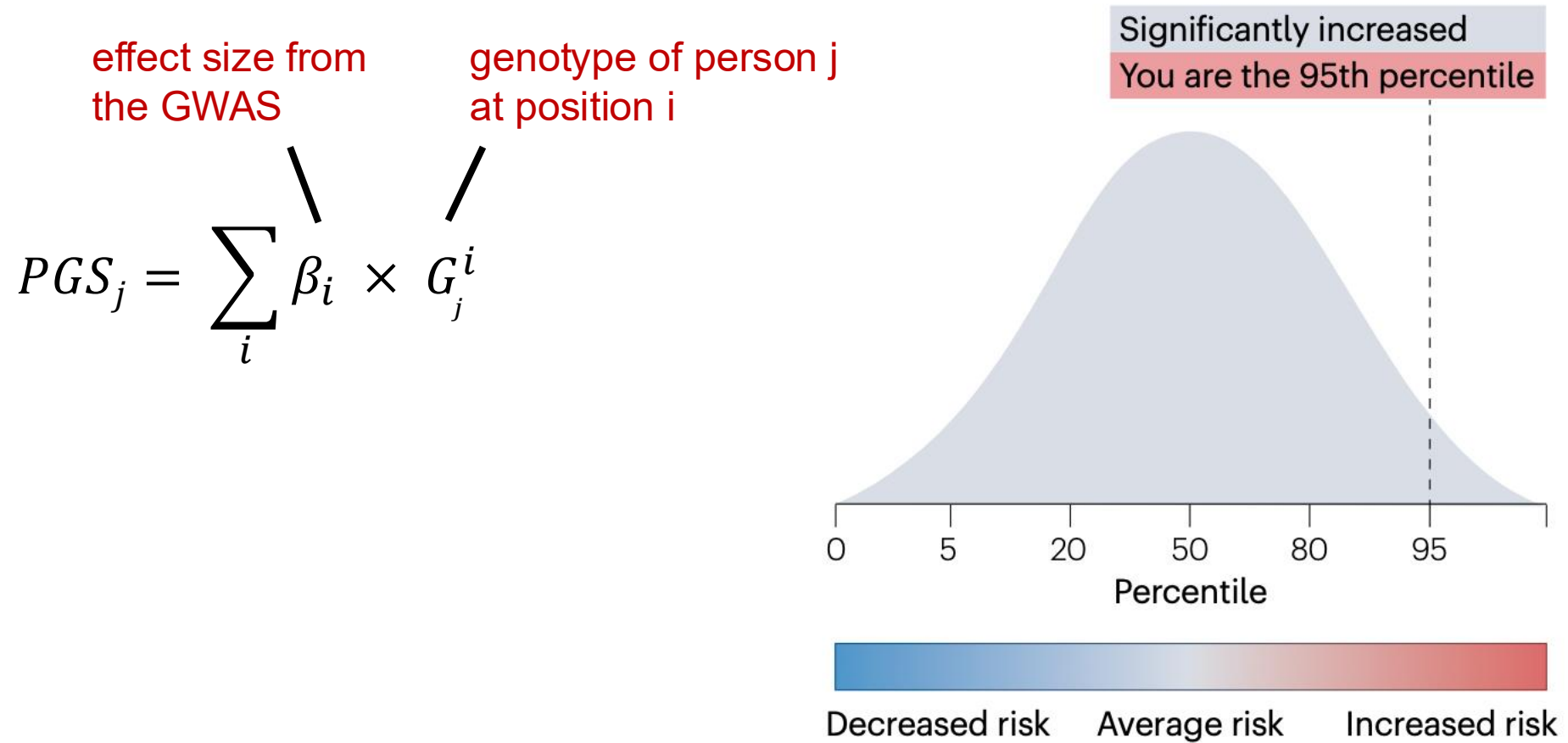
effect size from
the GWAS

genotype of person j
at position i

$$PGS_j = \sum_i \beta_i \times G_j^i$$

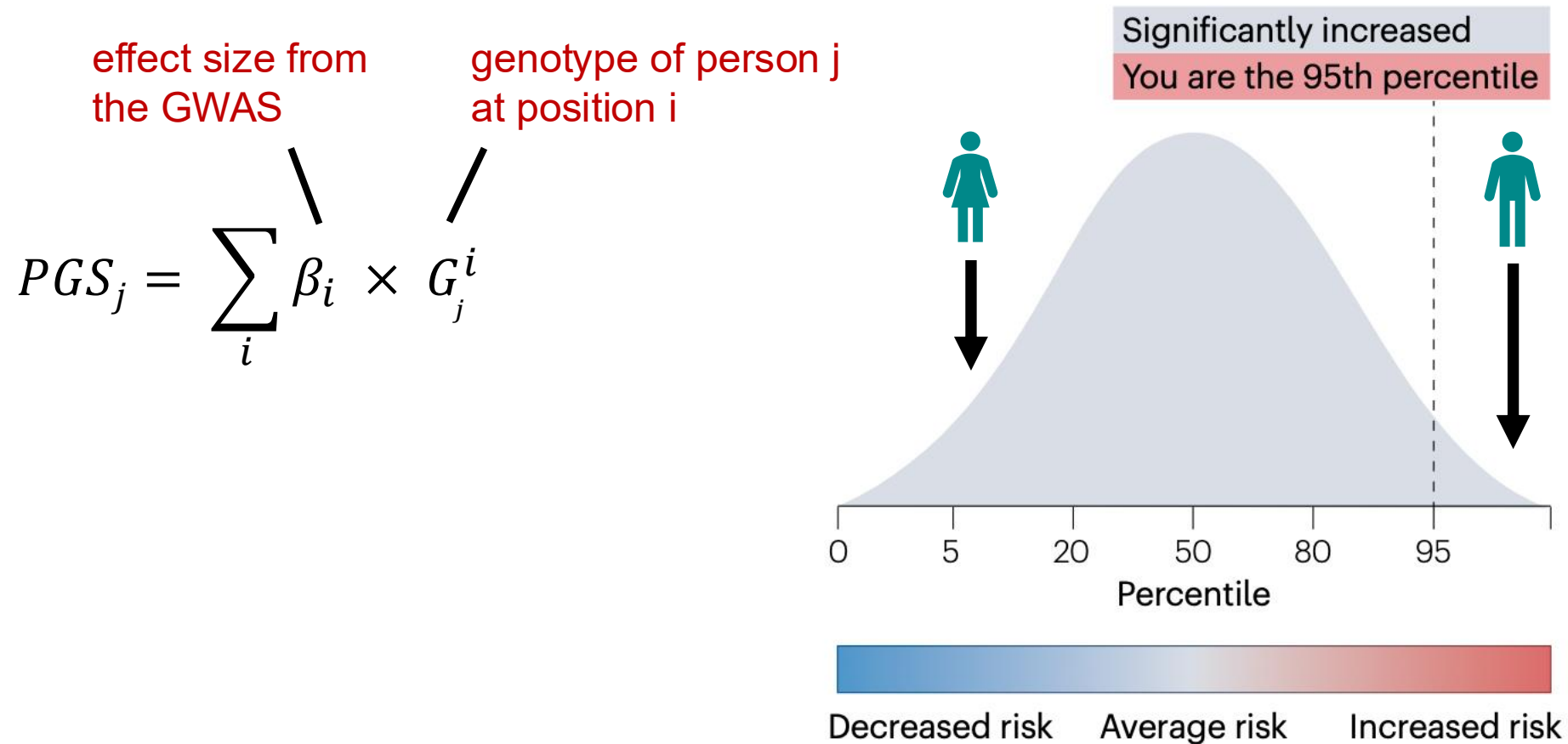
Polygenic scores: a way to aggregate GWAS signals

PGSs are widely used to predict one individual's genetic risk of developing a disease



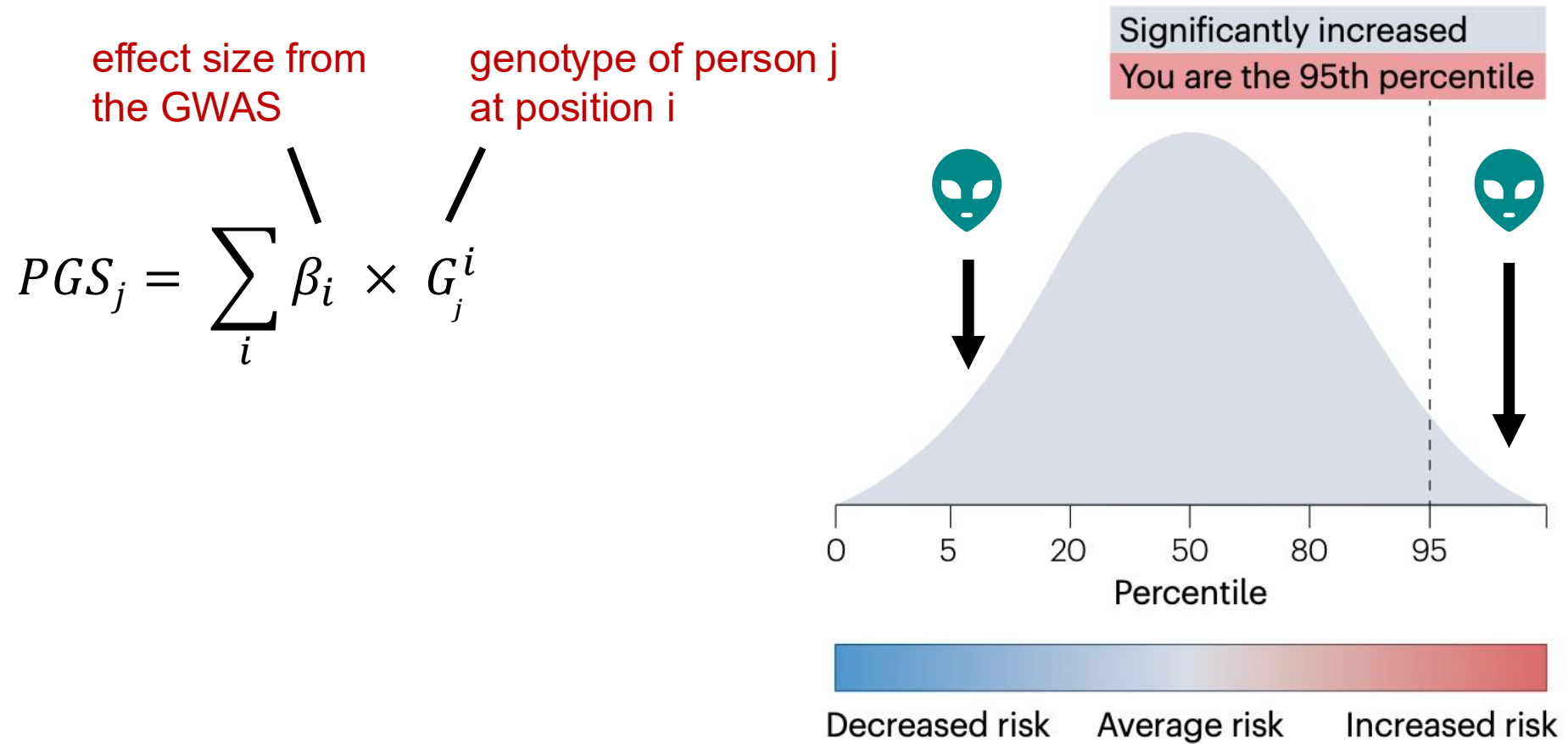
Polygenic scores: a way to aggregate GWAS signals

PGSs are widely used to predict one individual's genetic risk of developing a disease



Polygenic scores: a way to aggregate GWAS signals

PGSs are widely used to predict one individual's genetic risk of developing a disease



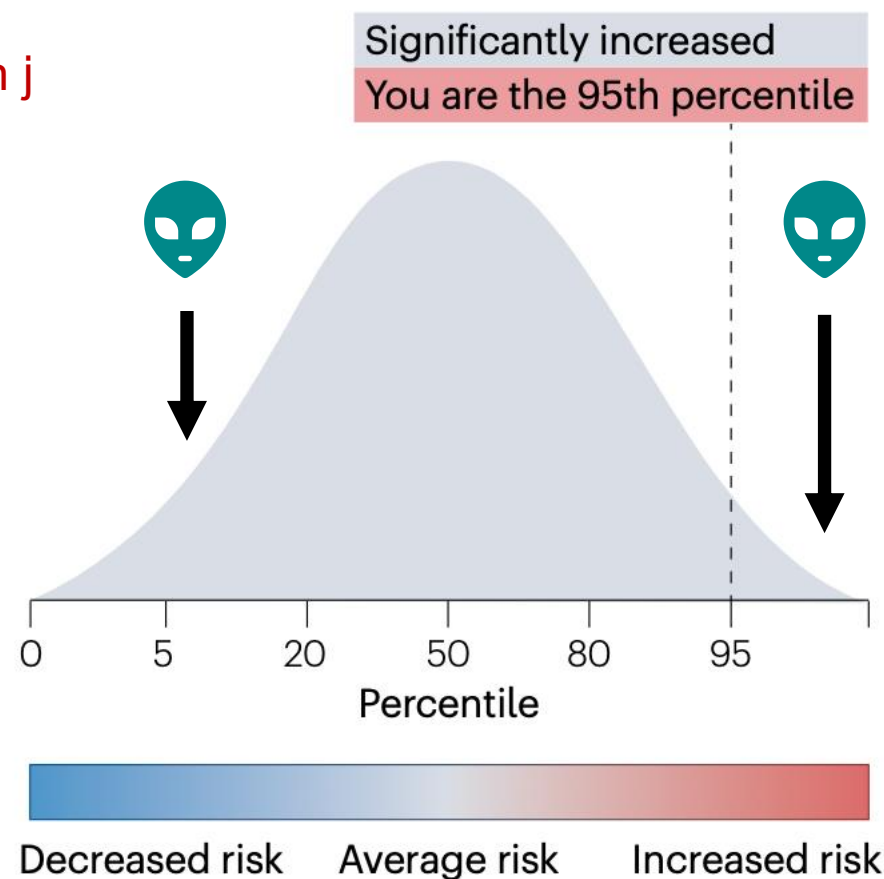
Polygenic scores: a way to aggregate GWAS signals

PGSs are widely used to predict one individual's genetic risk of developing a disease

effect size from the GWAS genotype of person j at position i

$$PGS_j = \sum_i \beta_i \times G_j^i$$

→ a PGS is simply a single number that summarises a person's estimated genetic risk of developing a disease.



**stop & think #1: what is the main
idea behind a PGS? Why are they
useful?**

GWAS & PGS have been very successful, but:

- population level only
- only capture additive relationships
- only one phenotype at the time + no interactions between them
- only one data type at the time (uni-modal) + no interactions between them
- non-linear relationships/complex interactions possible in theory, but hard to do in practice
- hard to transfer across different populations

GWAS & PGS have been very successful, but:

- population level only
- only capture additive relationships
- only one phenotype at the time + no interactions between them
- only one data type at the time (uni-modal) + no interactions between them
- non-linear relationships/complex interactions possible in theory, but hard to do in practice
- hard to transfer across different populations

→ majority of these are methods limitations!

different ways of learning from data

two cultures: statistics vs. machine learning

different ways of learning from data

conventional GWAS/PGS methods

two cultures: statistics vs. machine learning

different ways of learning from data

conventional GWAS/PGS methods

Statistics

data-modeling culture

Goal	infer relationships on full data
Model	stochastic model of the data; explicit assumptions, lower model complexity
Methods	Regression, GLMs, hypothesis testing
Priority	interpretability and significance

two cultures: statistics vs. machine learning

different ways of learning from data

conventional GWAS/PGS methods

Statistics

data-modeling culture

Goal	infer relationships on full data
Model	stochastic model of the data; explicit assumptions, lower model complexity
Methods	Regression, GLMs, hypothesis testing
Priority	interpretability and significance

Machine Learning

algorithmic-modeling culture

Goal	prediction accuracy on new data
Model	no assumptions about mechanism, flexible algorithms, higher model complexity
Methods	regression trees, random forest, boosting, NNs & deep learning
Priority	out-of-sample prediction accuracy

Framing after Breiman (2001), "Statistical Modeling: The Two Cultures"

**The DIFFERENCE between
STATISTICS and MACHINE
LEARNING lies in the MINDSET not
in the TOOLS.**

learning from data:
the statistics way

learning from data: the statistics way

Fitting a line to data

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x + \varepsilon$$

response variable explanatory variable

error term

first parameter
(intercept)

second parameter
(slope)

The diagram shows the linear regression equation $\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x + \varepsilon$. Red text labels are placed around the equation with black lines pointing to specific parts: 'response variable' points to $\hat{y}$, 'explanatory variable' points to x , 'error term' points to ε , 'first parameter (intercept)' points to $\hat{\beta}_0$, and 'second parameter (slope)' points to $\hat{\beta}_1$.

learning from data: the statistics way

Fitting a line to data

response variable

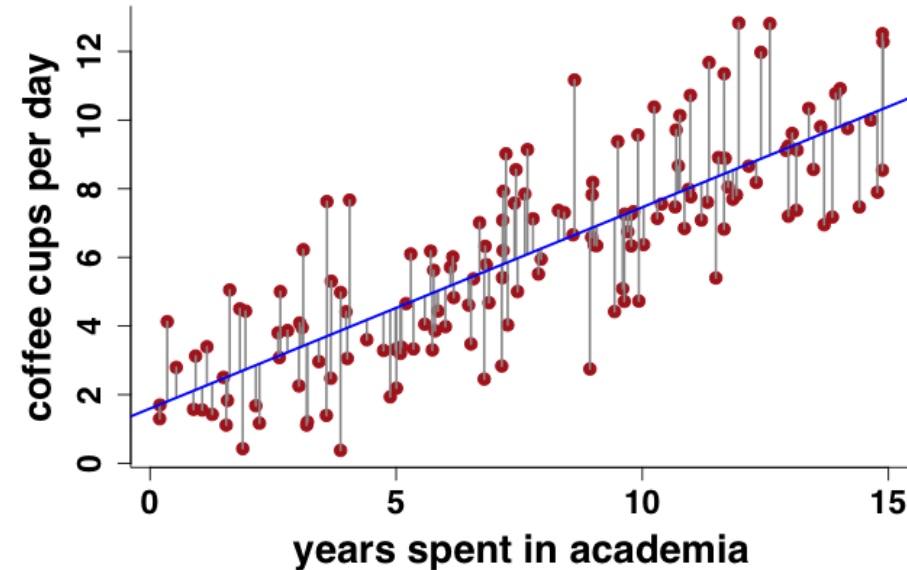
explanatory variable

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x + \varepsilon$$

first parameter (intercept) second parameter (slope) error term

In statistical inference we optimise the model parameters on the full sample such that they describe the data best.

Parameter optimization with OLS:



$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$$

learning from data: the statistics way

Fitting a line to data

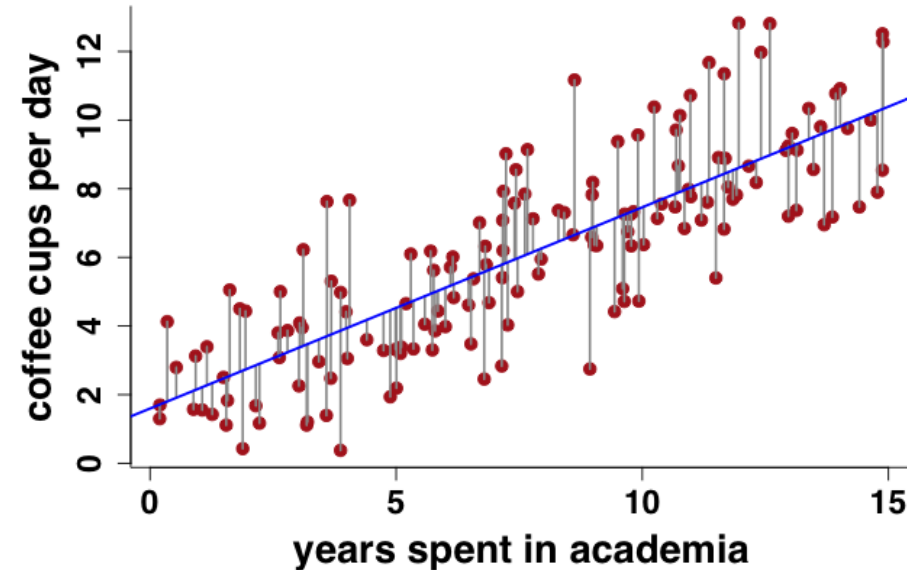
response variable

explanatory variable

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x + \varepsilon$$

first parameter (intercept) second parameter (slope) error term

Parameter optimization with OLS:



$$(\hat{\beta}_0, \hat{\beta}_1) = \arg \min_{\beta_0, \beta_1} \sum_i (y_i - \beta_0 - \beta_1 x_i)^2$$

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$$

learning from data: the statistics way

Fitting a line to data

response variable

explanatory variable

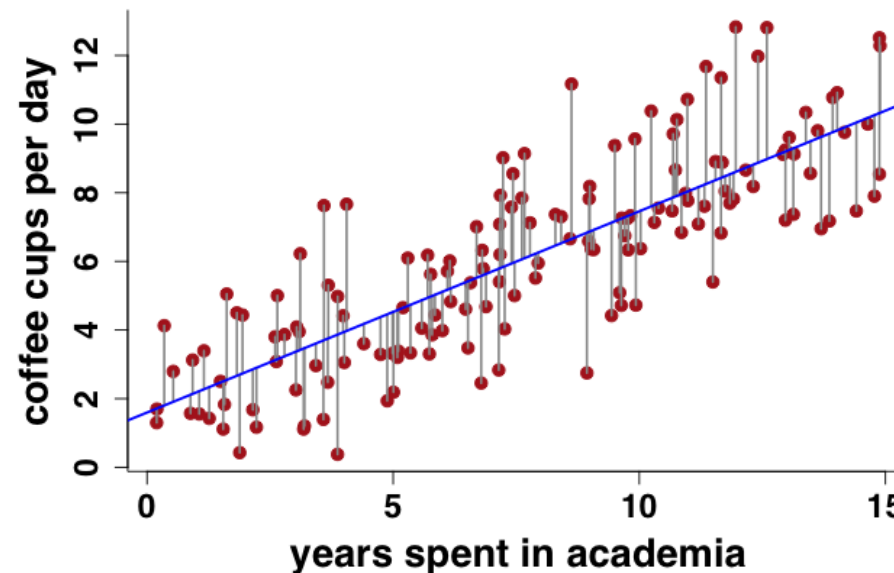
$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x + \varepsilon$$

first parameter (intercept) second parameter (slope) error term

RSS = SSE := sum of squared errors

$$(\hat{\beta}_0, \hat{\beta}_1) = \arg \min_{\beta_0, \beta_1} \sum_i (y_i - \beta_0 - \beta_1 x_i)^2$$

Parameter optimization with OLS:



$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

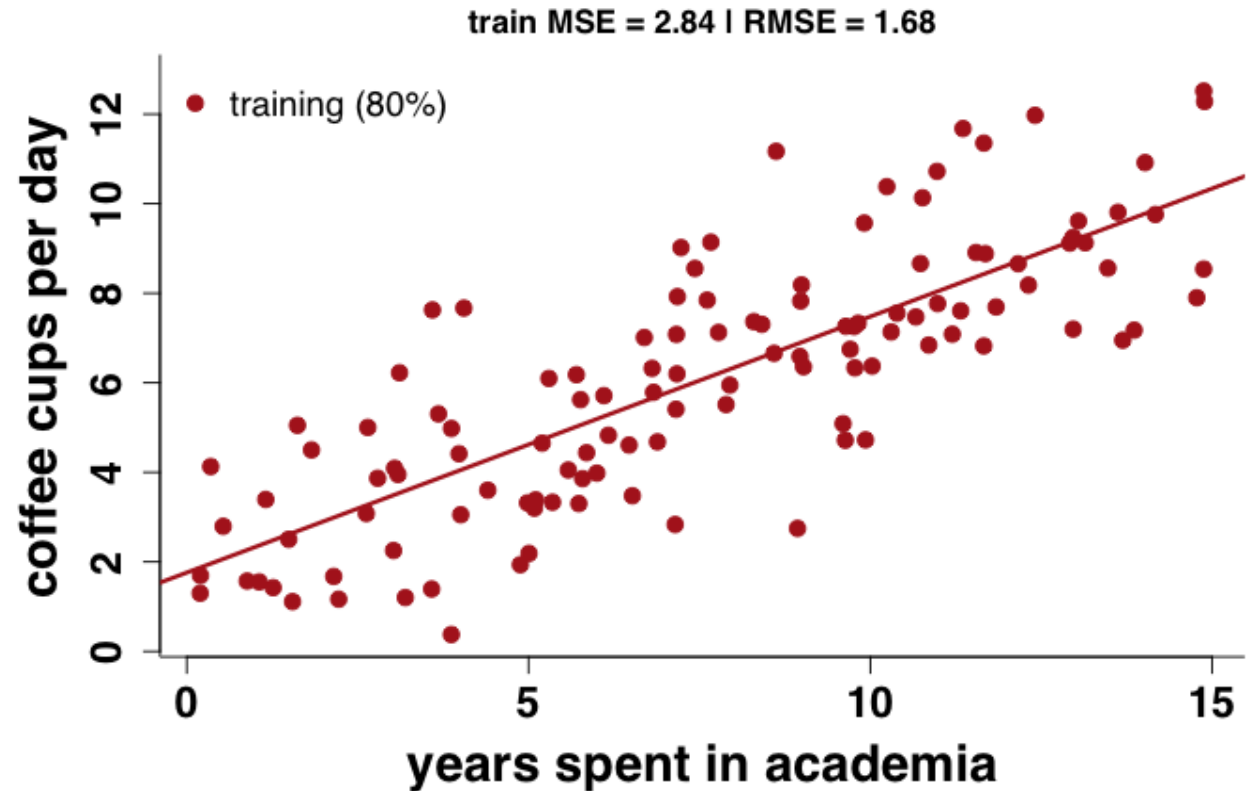
$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$$

learning from data:
the machine learning way

learning from data: the machine learning way

Can we learn from the data more autonomously and predict unseen measurements?

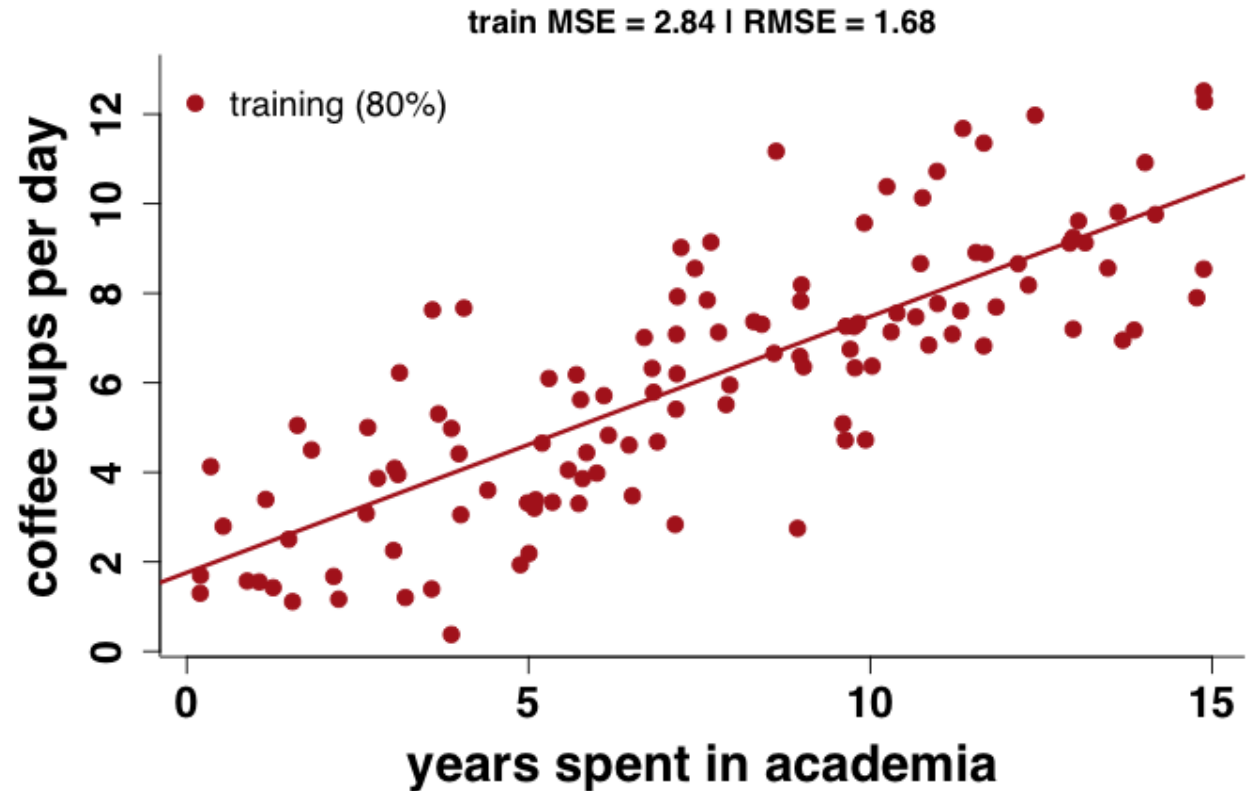
- 1) split the data: training & test set
- 2) fit machine learning model to the **training set**



learning from data: the machine learning way

Can we learn from the data more autonomously and predict unseen measurements?

- 1) split the data: training & test set
- 2) fit machine learning model to the **training set**



learning from data: the machine learning way

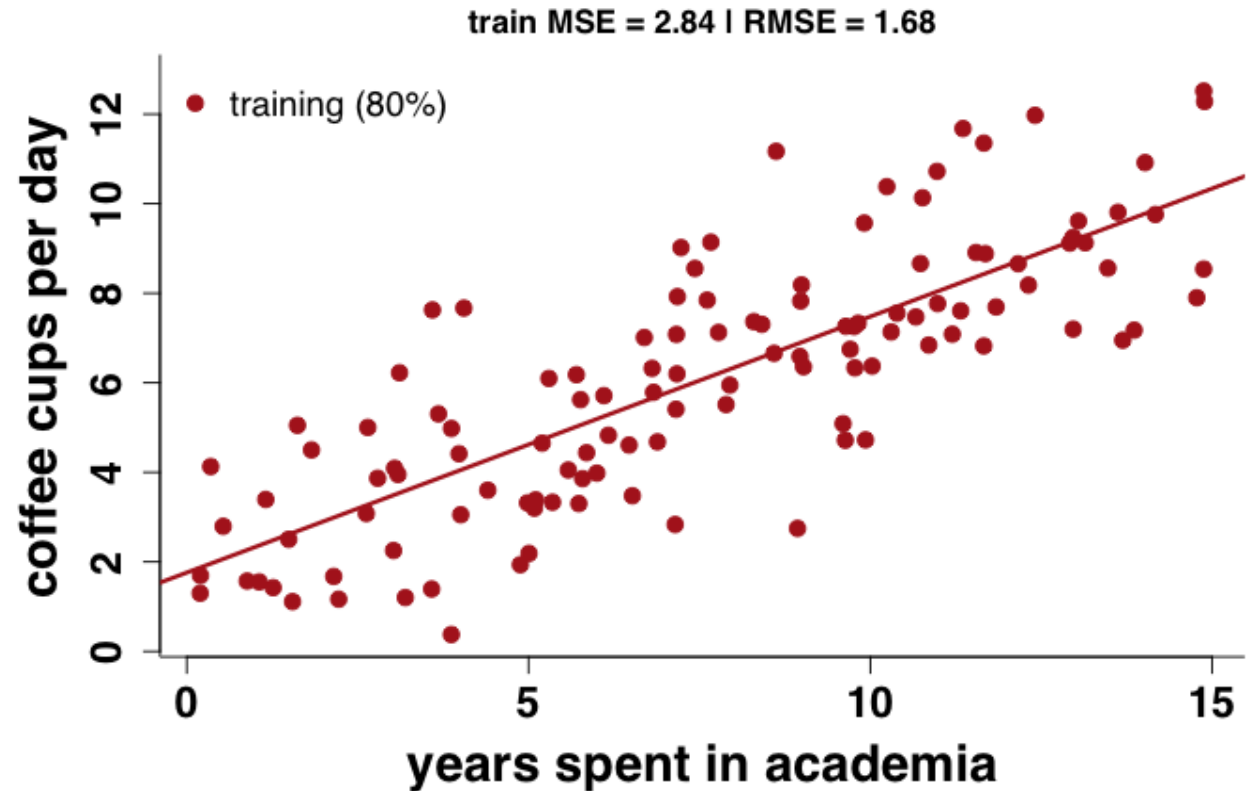
Can we learn from the data more autonomously and predict unseen measurements?

- 1) split the data: training & test set
- 2) fit machine learning model to the **training set**

coffee prediction # years measured

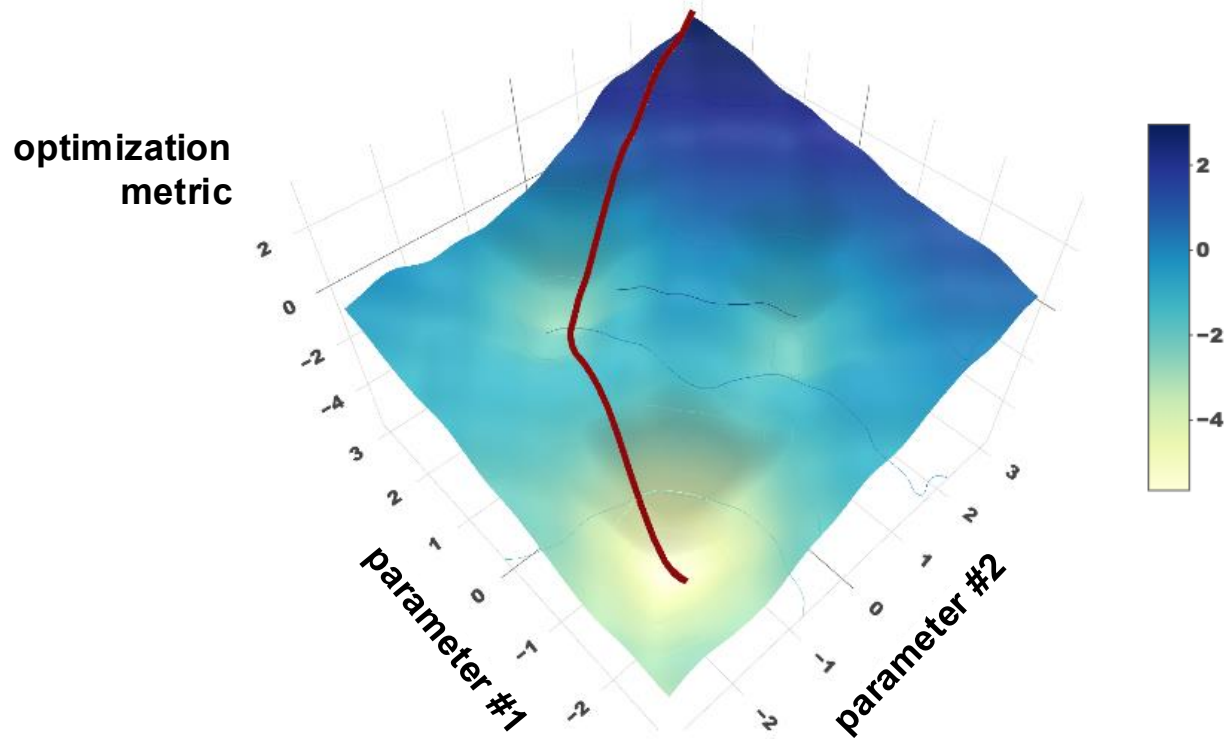
$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x + \varepsilon$$

trained parameter #1 (intercept) trained parameter #2 (slope)



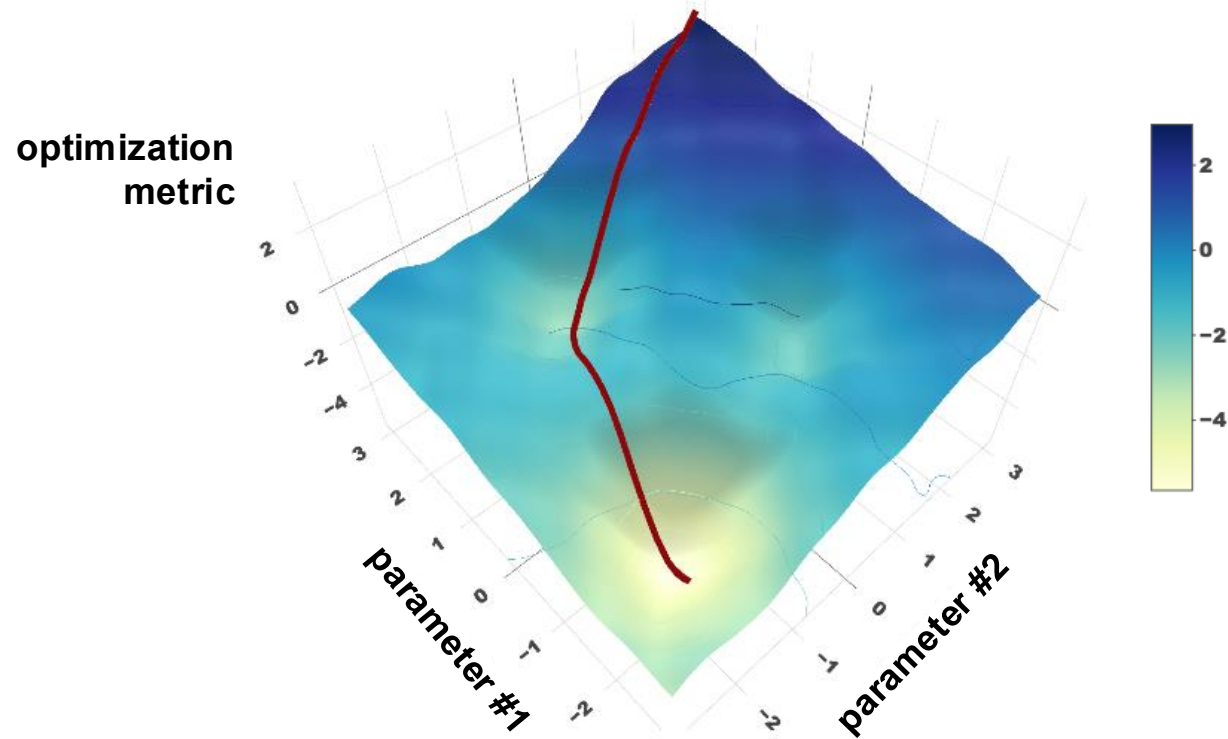
**stop & think #2: how could we fit the
model to the data in a machine
learning way?**

Optimization is everywhere in science



- genetics, ecology & evolution (fitness landscape)
- epigenetics (waddington landscape)
- protein folding (funnel)
- physics/chemistry (energy landscapes)
- Bayesian optimization

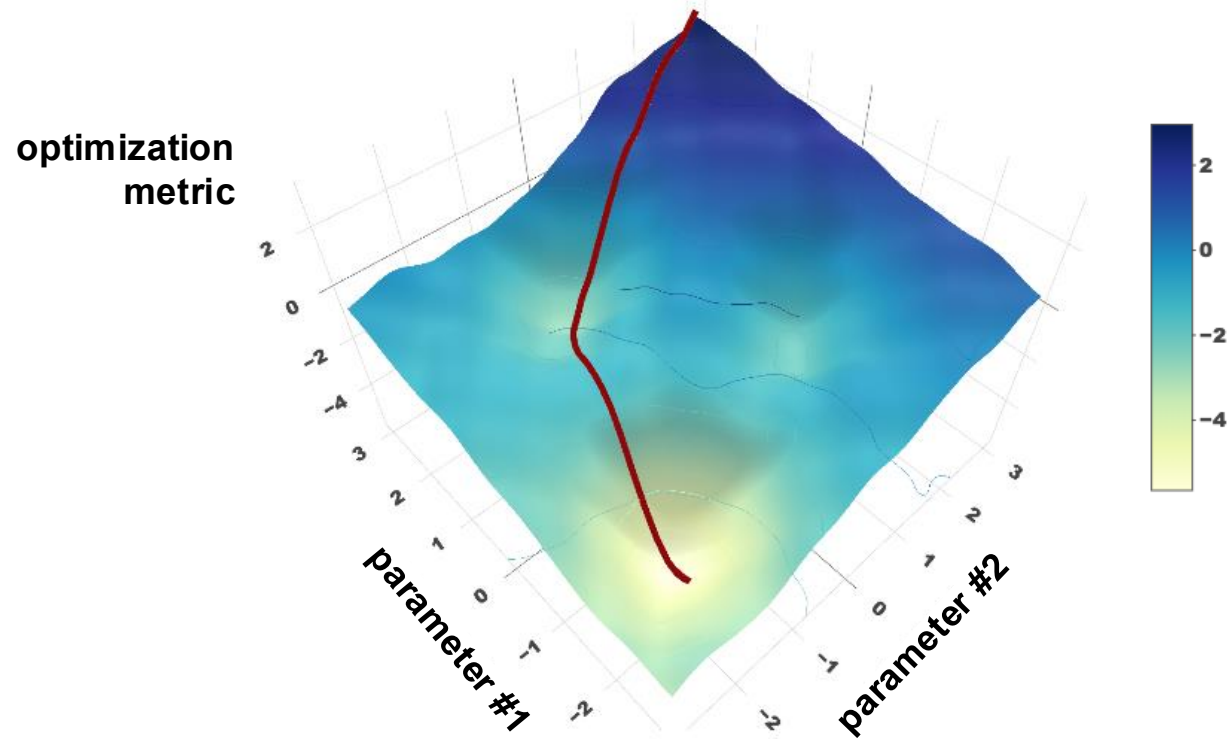
Optimization is everywhere in science



goal: find parameter values that produce best possible mapping I/O

we need:

Optimization is everywhere in science

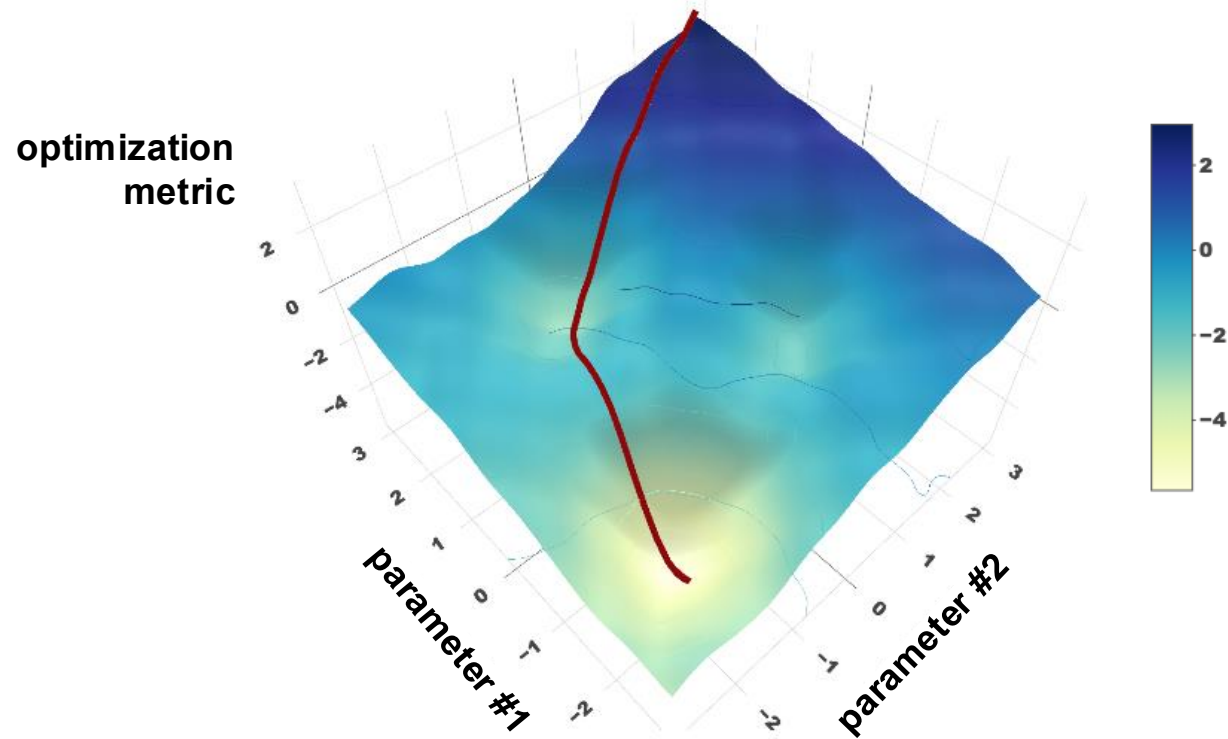


goal: find parameter values that produce best possible mapping I/O

we need:

- the model

Optimization is everywhere in science

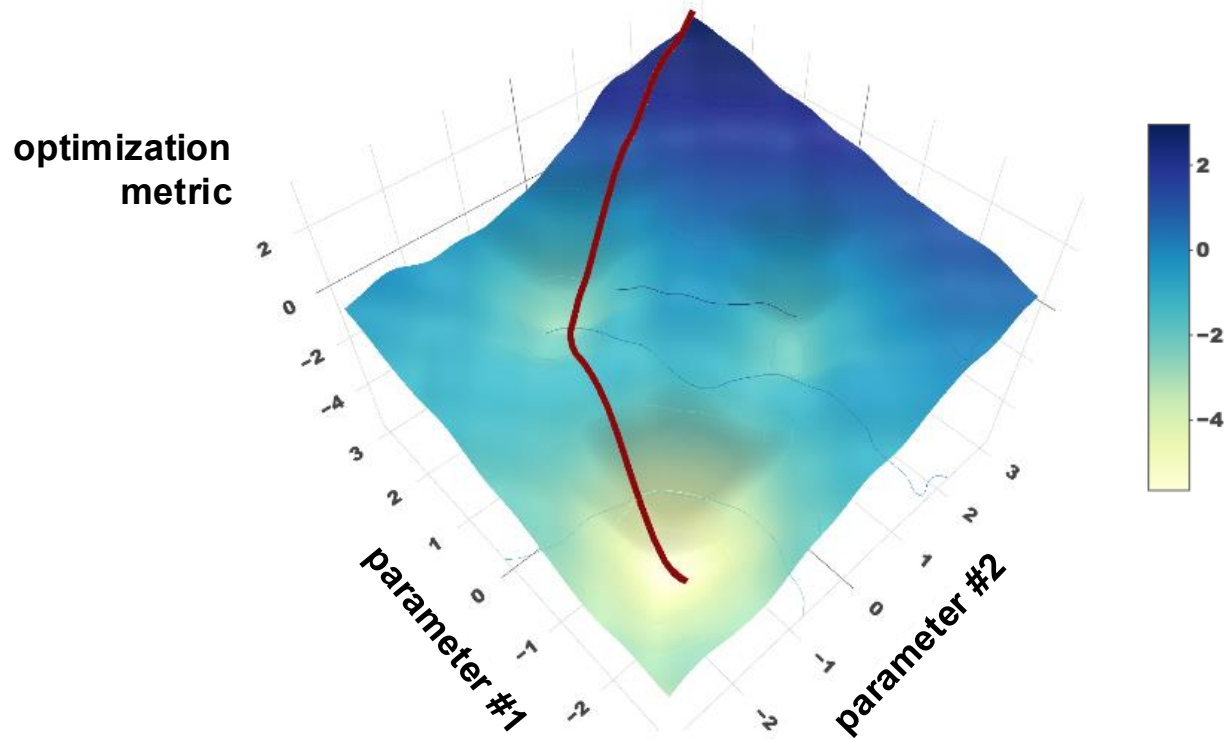


goal: find parameter values that produce best possible mapping I/O

we need:

- the model
- training set

Optimization is everywhere in science



goal: find parameter values that produce best possible mapping I/O

we need:

- the model
- training set
- a way to assess prediction quality:
mapping → **loss function**

More general & a slight change in notation

Bridging the gap: statistics & machine learning evolved different notations for the same things...

the model

$$y = f[x, \phi]$$

the loss function

$$\mathbf{L}[\phi]$$

More general & a slight change in notation

Bridging the gap: statistics & machine learning evolved different notations for the same things...

the model

$$y = f[x, \phi]$$

output input parameters
 = **weights**

the loss function

$$L[\phi]$$

More general & a slight change in notation

Bridging the gap: statistics & machine learning evolved different notations for the same things...

the model

$$y = \phi_0 + \phi_1 x$$

$$\phi = \begin{bmatrix} \phi_0 \\ \phi_1 \end{bmatrix} \begin{array}{l} \text{--- 'intercept'} \\ \text{--- 'slope'} \end{array}$$

the loss function

$$\mathbf{L}[\phi]$$

More general & a slight change in notation

Bridging the gap: statistics & machine learning evolved different notations for the same things...

the model

$$y = \phi_0 + \phi_1 x$$

$$\phi = \begin{bmatrix} \phi_0 \\ \phi_1 \end{bmatrix} \begin{array}{l} \text{--- 'intercept'} \\ \text{--- 'slope'} \end{array}$$

the loss function

$$L[\phi] = \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2$$

More general & a slight change in notation

Bridging the gap: statistics & machine learning evolved different notations for the same things...

the model

$$y = \phi_0 + \phi_1 x$$

$$\phi = \begin{bmatrix} \phi_0 \\ \phi_1 \end{bmatrix} \begin{array}{l} \text{--- 'intercept'} \\ \text{--- 'slope'} \end{array}$$

the loss function

$$L[\phi] = \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2$$

this is the **exact same function** (RSS)
we used to learn $\hat{\beta}_0$ and $\hat{\beta}_1$ in our
previous example

More general & a slight change in notation

Bridging the gap: statistics & machine learning evolved different notations for the same things...

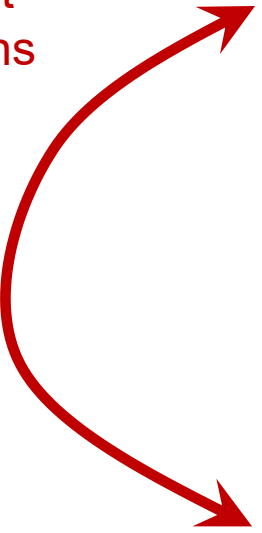
the model

$$y = \phi_0 + \phi_1 x$$

$$\phi = \begin{bmatrix} \phi_0 \\ \phi_1 \end{bmatrix} \begin{array}{l} \text{--- 'intercept'} \\ \text{--- 'slope'} \end{array}$$

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x + \varepsilon$$

same things
different
notations

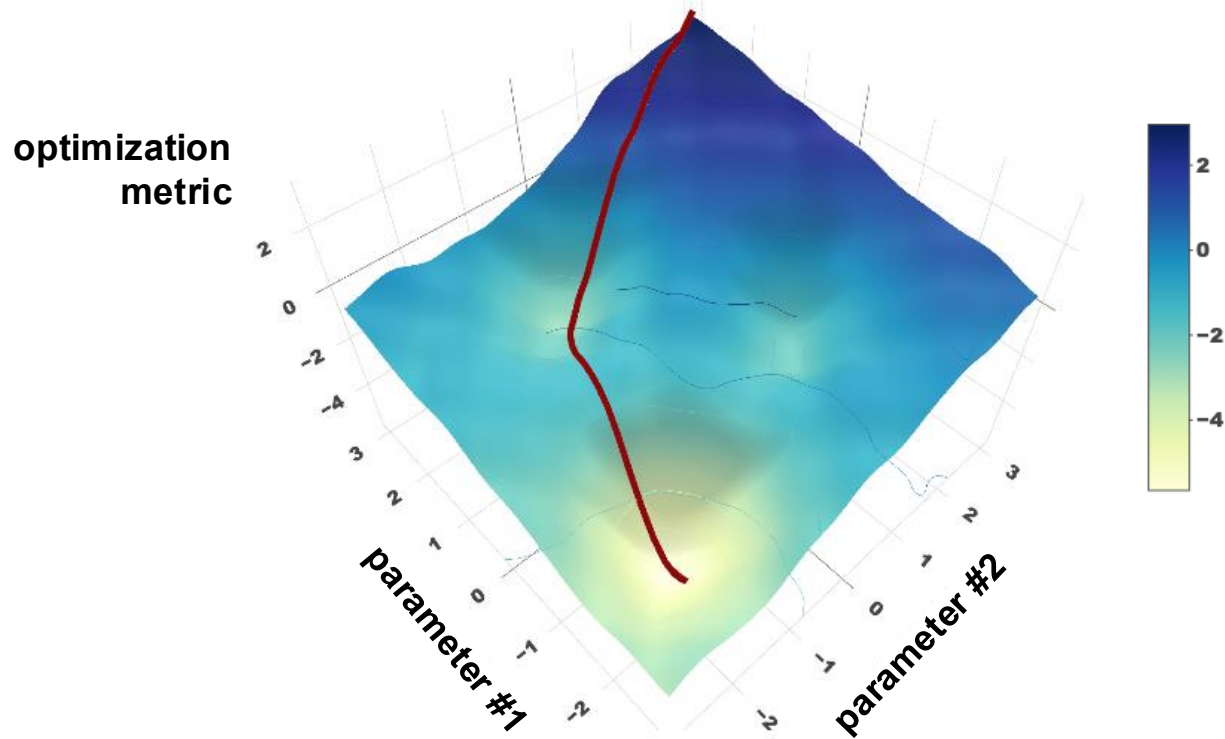


the loss function

$$L[\phi] = \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2$$

this is the **exact same function** (RSS)
we used to learn $\hat{\beta}_0$ and $\hat{\beta}_1$ in our
previous example

Optimization is everywhere in science

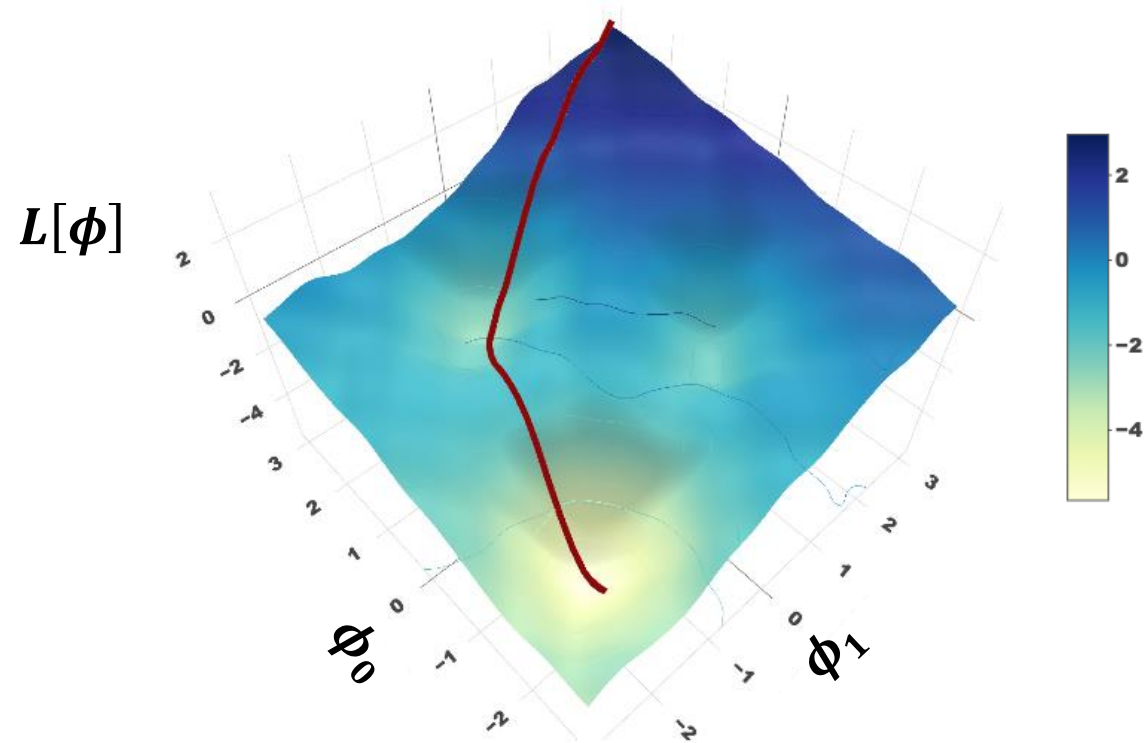


goal: find parameter values that produce best possible mapping I/O

we need:

- the model
- training set
- a way to assess prediction quality:
mapping → **loss function**

Optimization is everywhere in science



goal: find parameter values that produce best possible mapping I/O

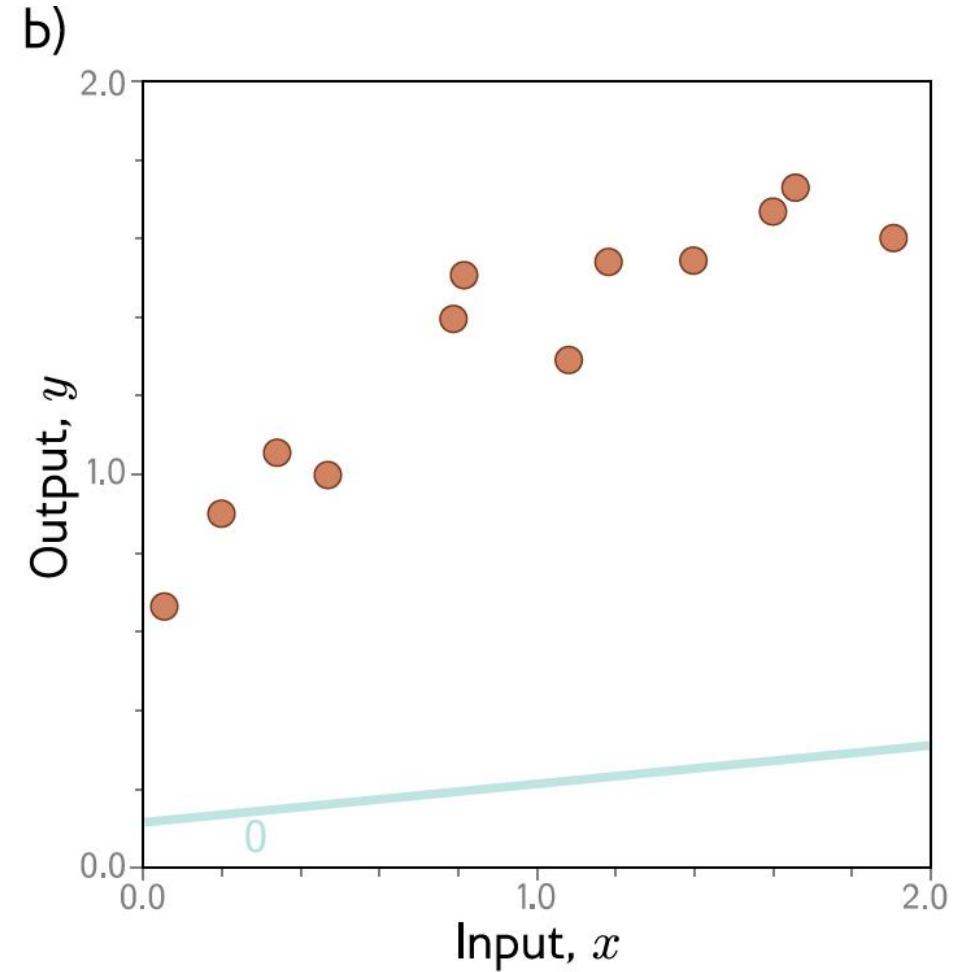
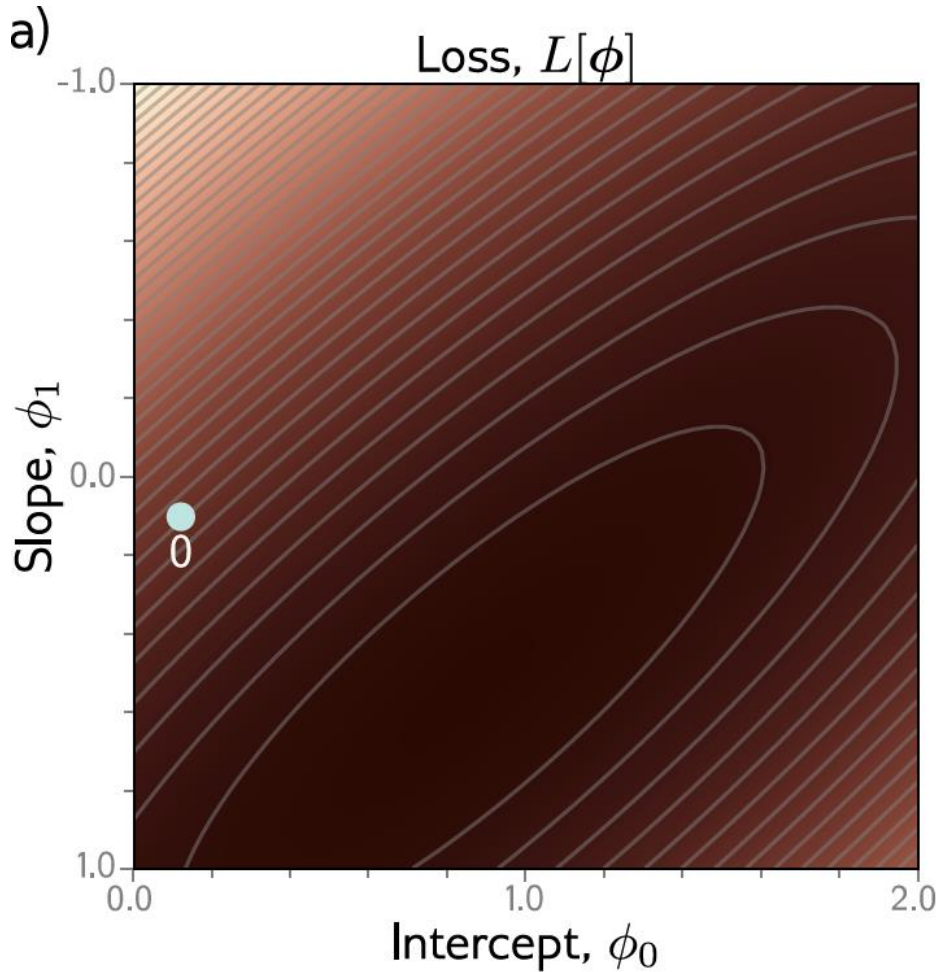
we need:

- the model
- training set
- a way to assess prediction quality:
mapping $\rightarrow$ **loss function**

how to train a model?

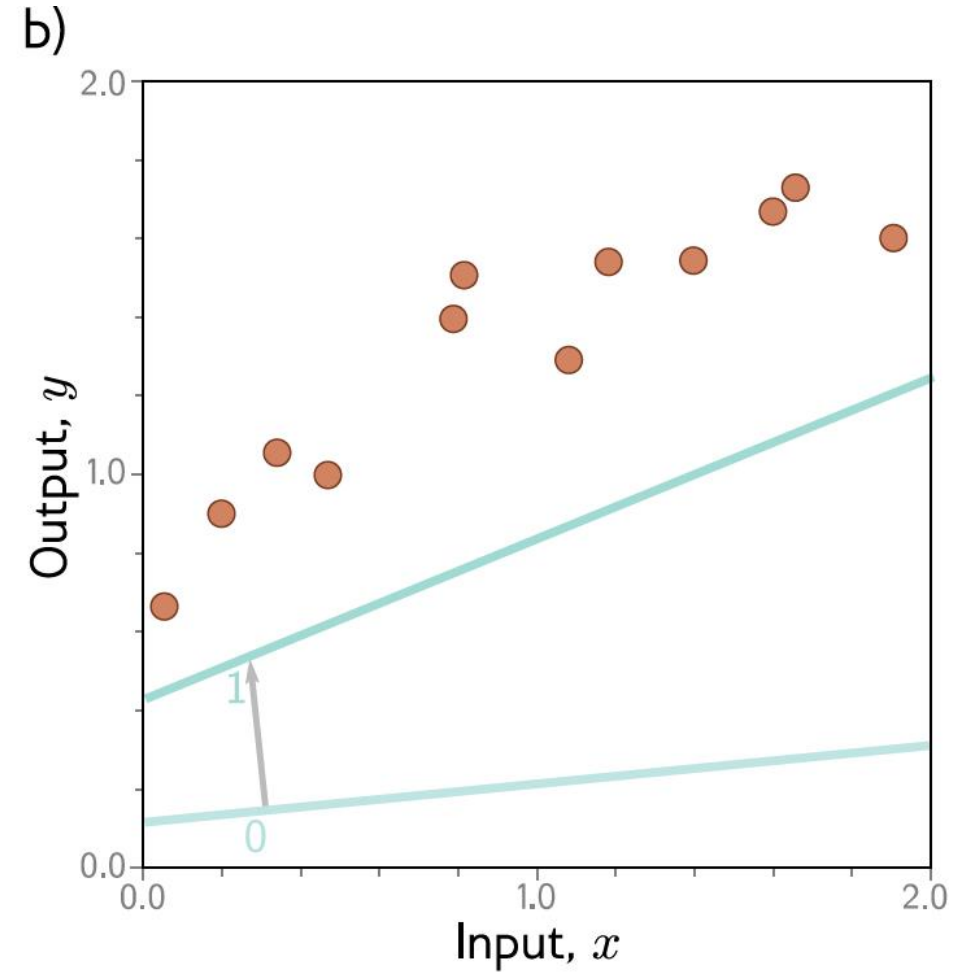
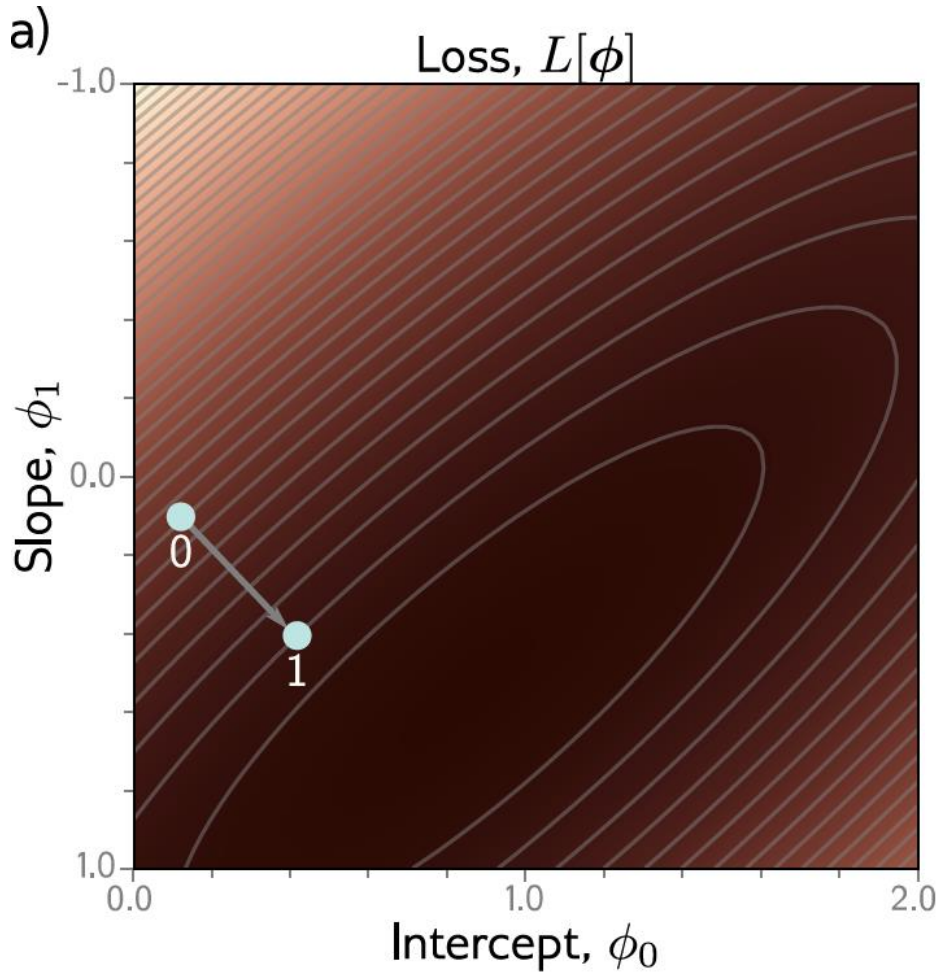
learning from data: the machine learning way

How to train a model?



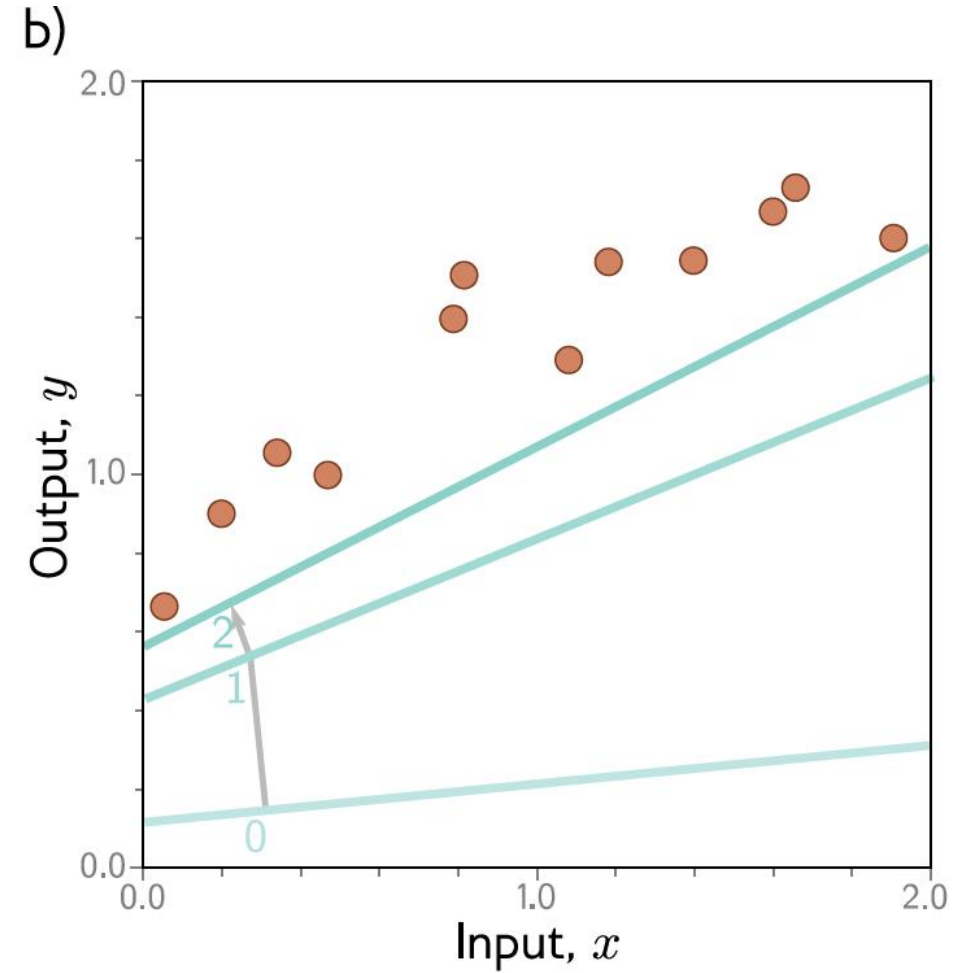
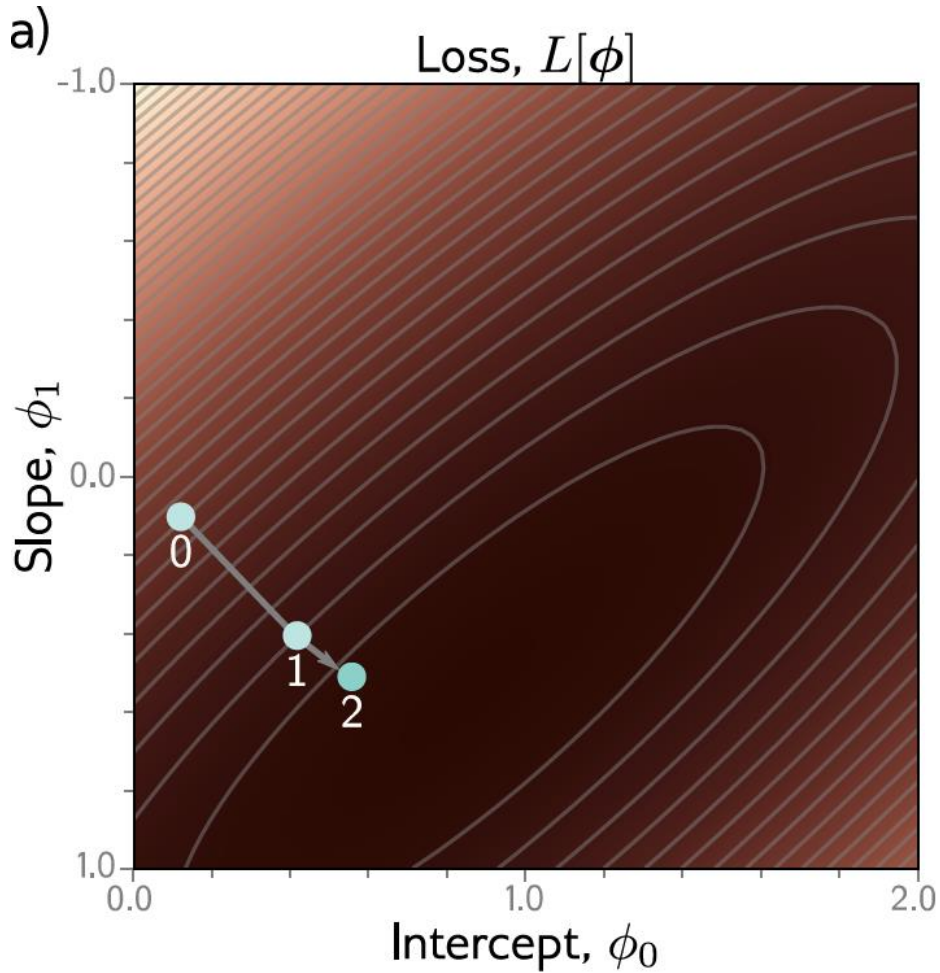
learning from data: the machine learning way

How to train a model?



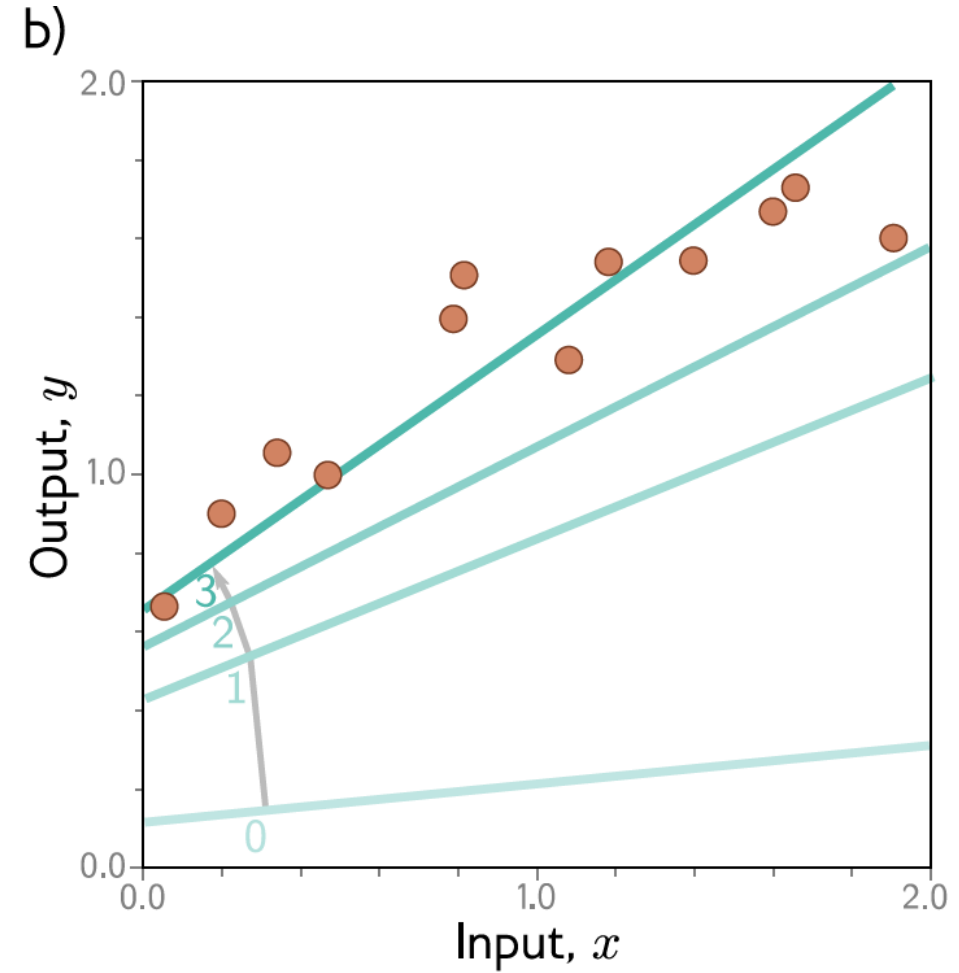
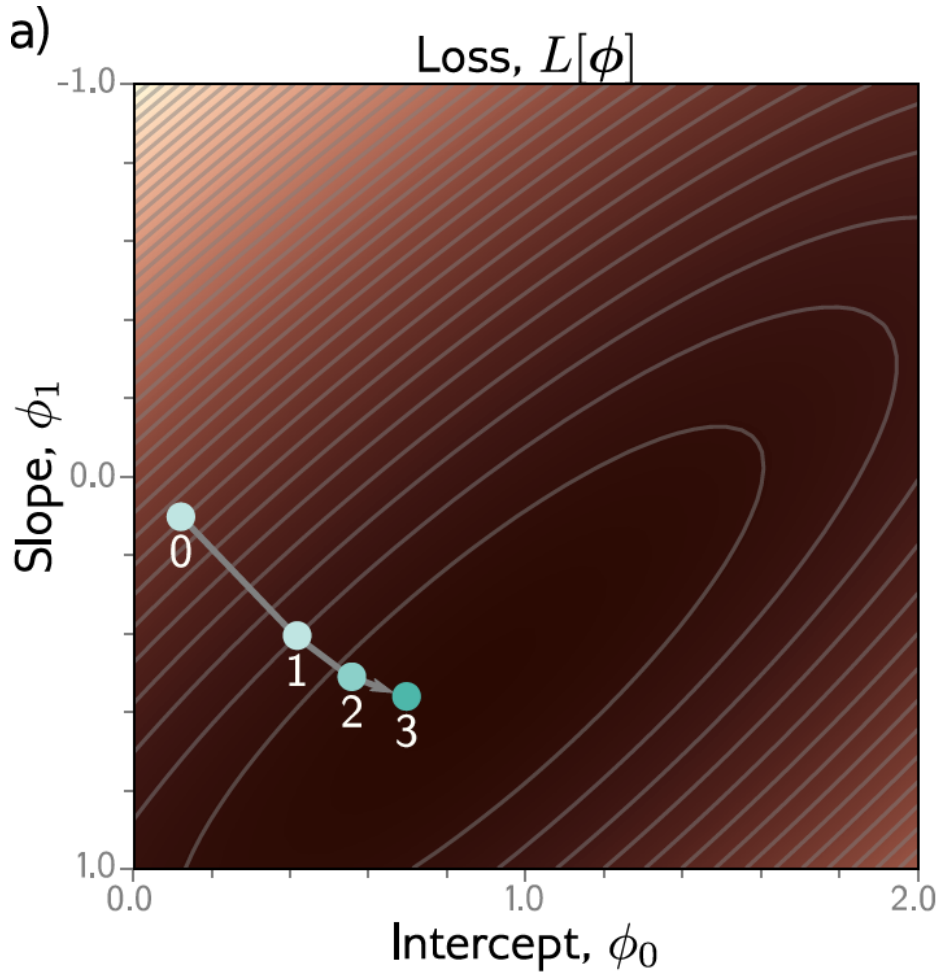
learning from data: the machine learning way

How to train a model?



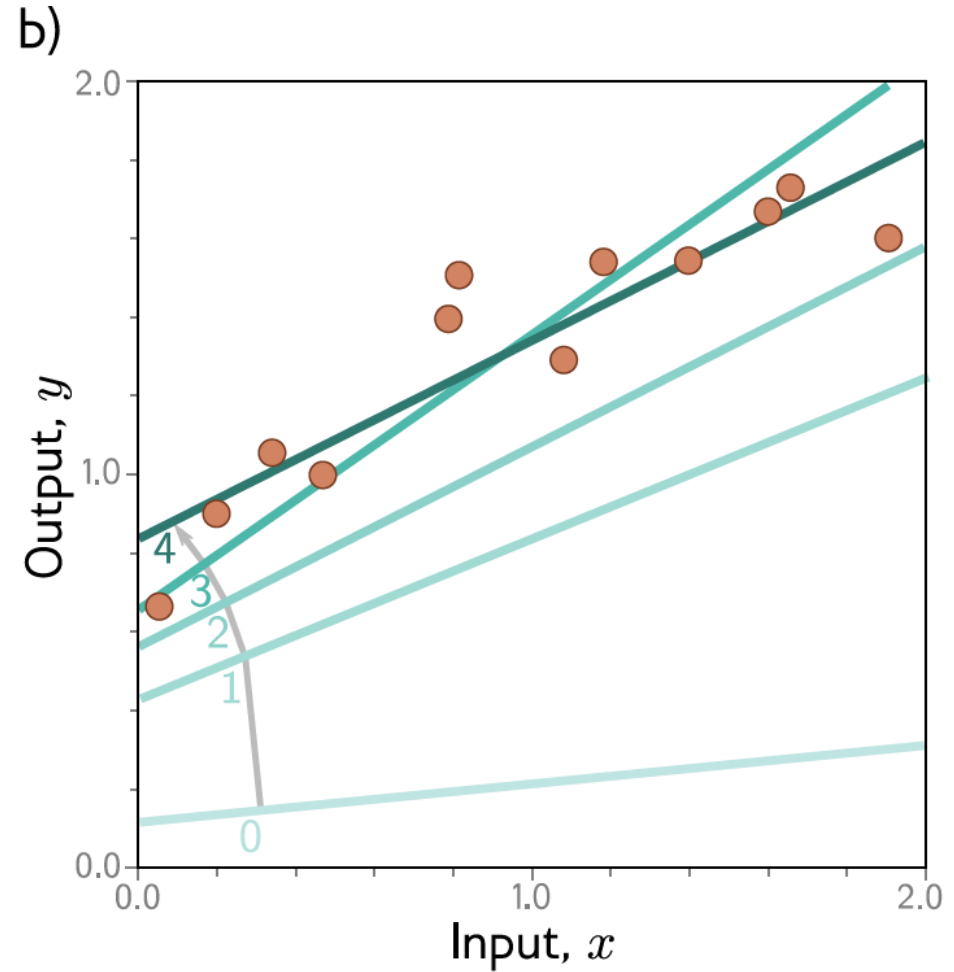
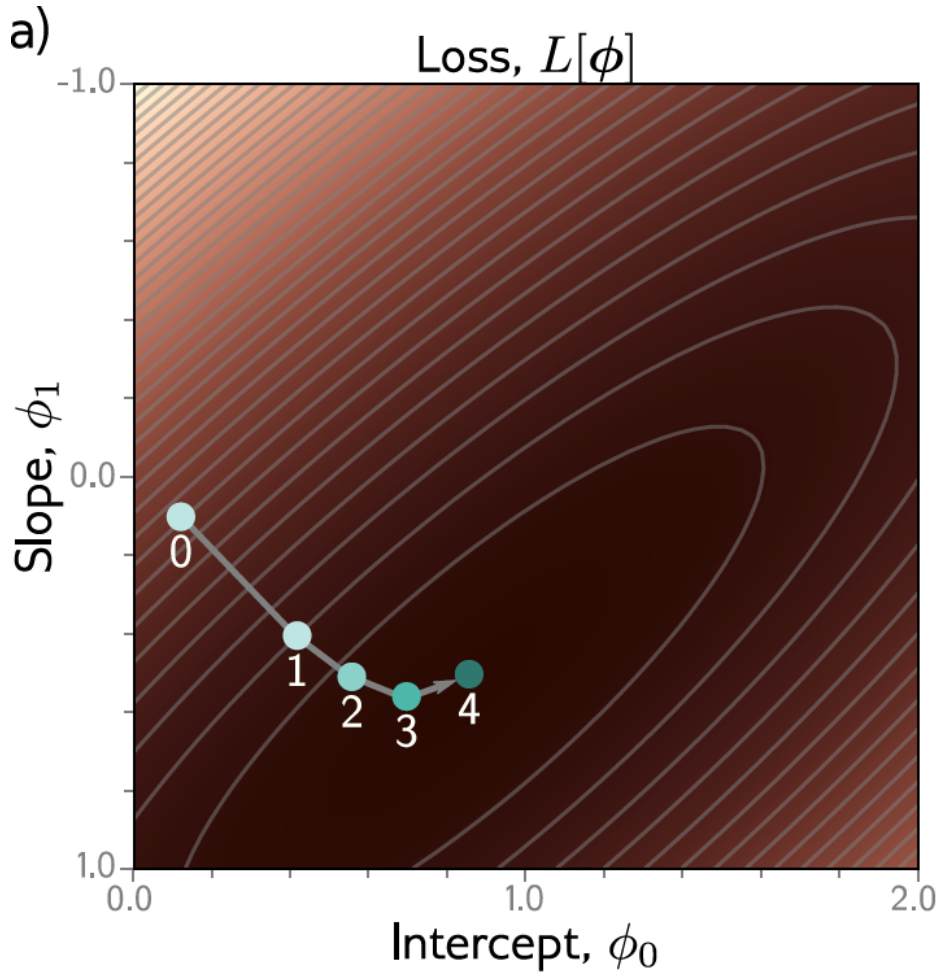
learning from data: the machine learning way

How to train a model?

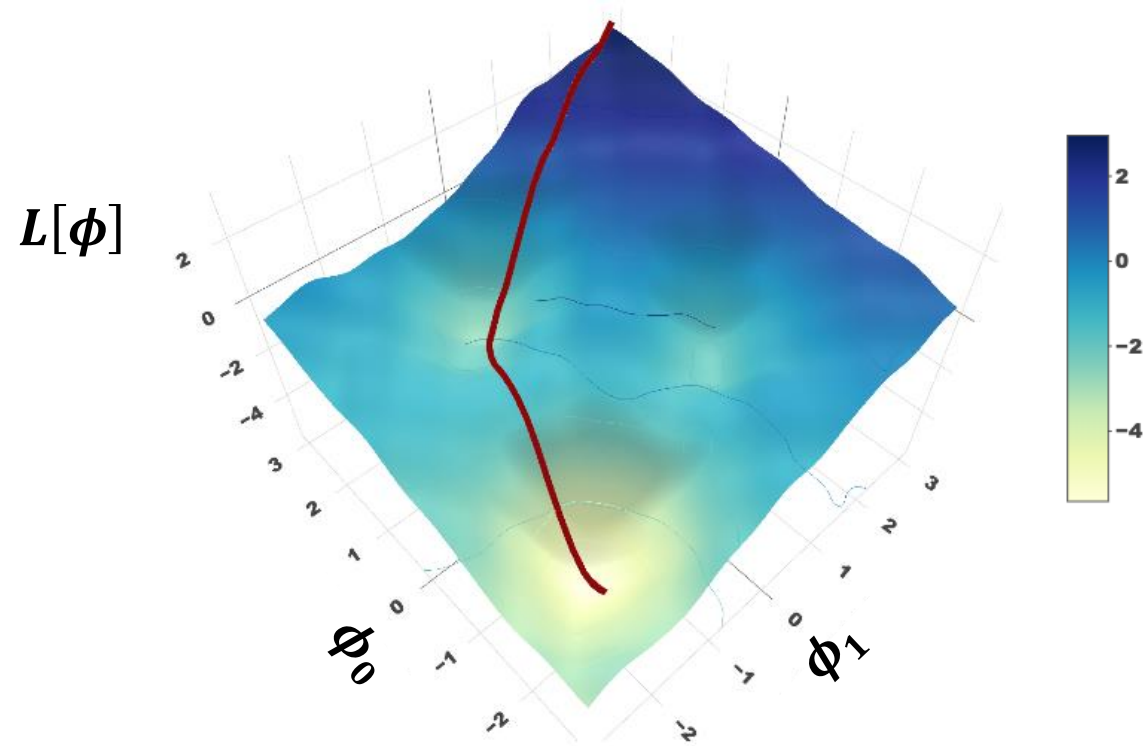


learning from data: the machine learning way

How to train a model?



Optimizing machine learning models in practice



- after each step → **update the weights**
- steps are also called model iteration or **epoch**
- why don't we just solve the equation?
 - often no-closed form solutions
 - exhaustive parameter search infeasible

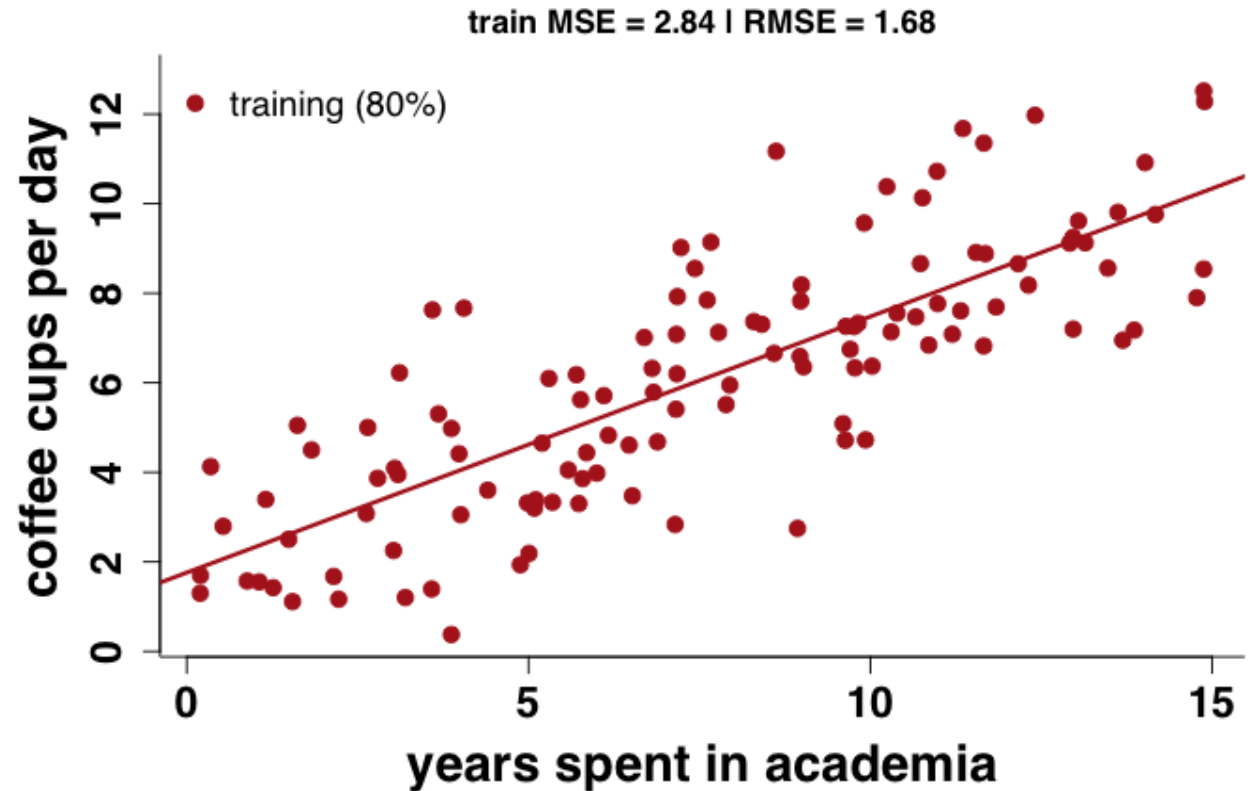
how good is the model?

machine learning: testing the model

Assessing model generalisability to unseen data.

how well does a pre-trained model describe unseen data?

1) in the **test set** predict outputs from inputs



machine learning: testing the model

Assessing model generalisability to unseen data.

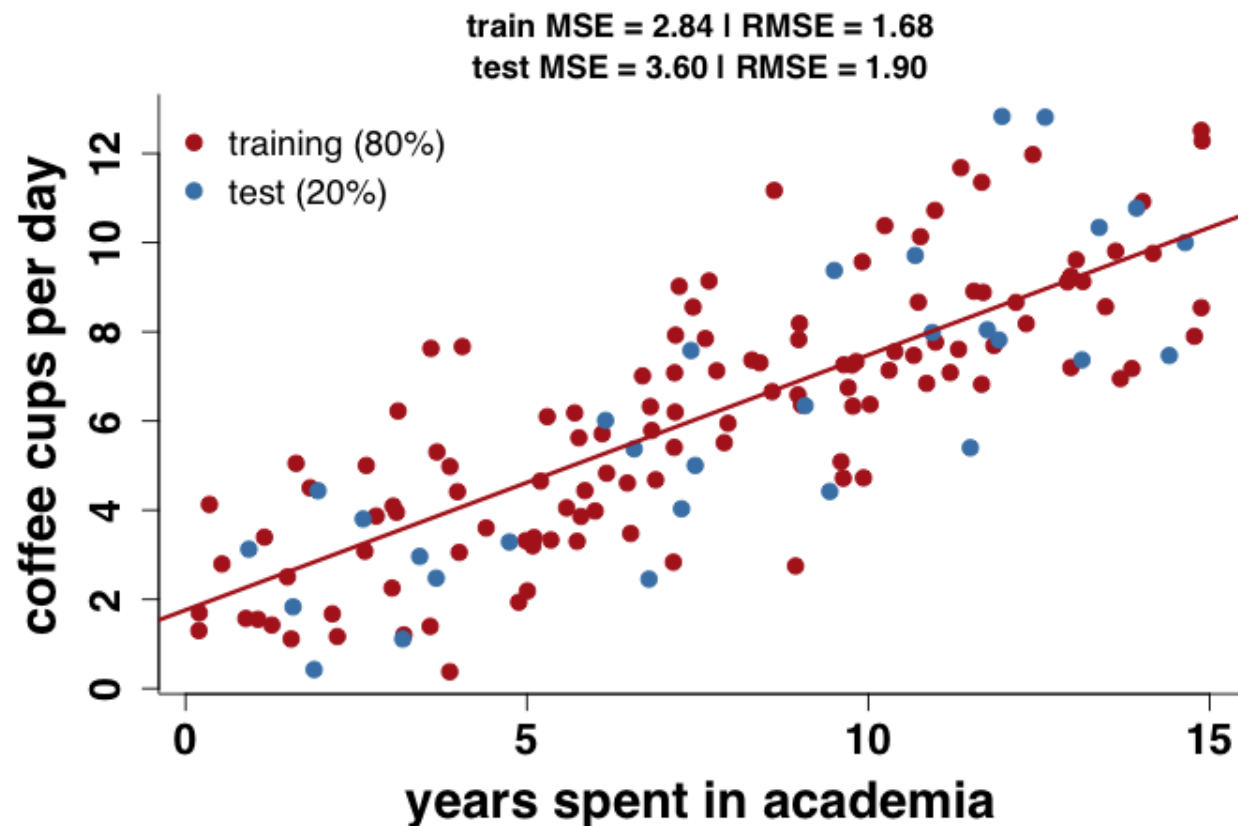
how well does a pre-trained model describe unseen data?

1) in the **test set** predict outputs from inputs

coffee prediction # years measured

$$y = \phi_0 + \phi_1 x$$

trained parameter #1 (intercept) trained parameter #2 (slope)



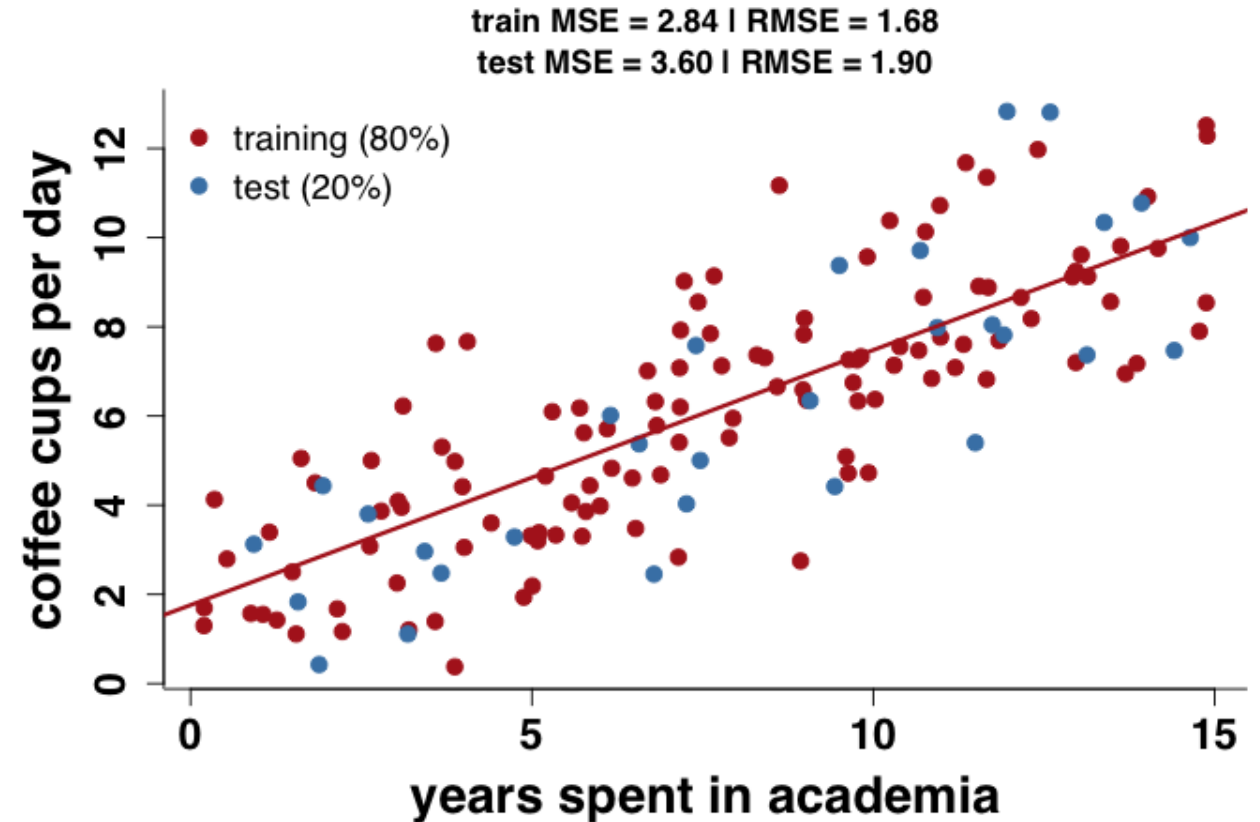
machine learning: testing the model

Assessing model generalisability to unseen data.

how well does a pre-trained model describe unseen data?

1) in the **test set** predict outputs from inputs

2) measure **prediction accuracy**

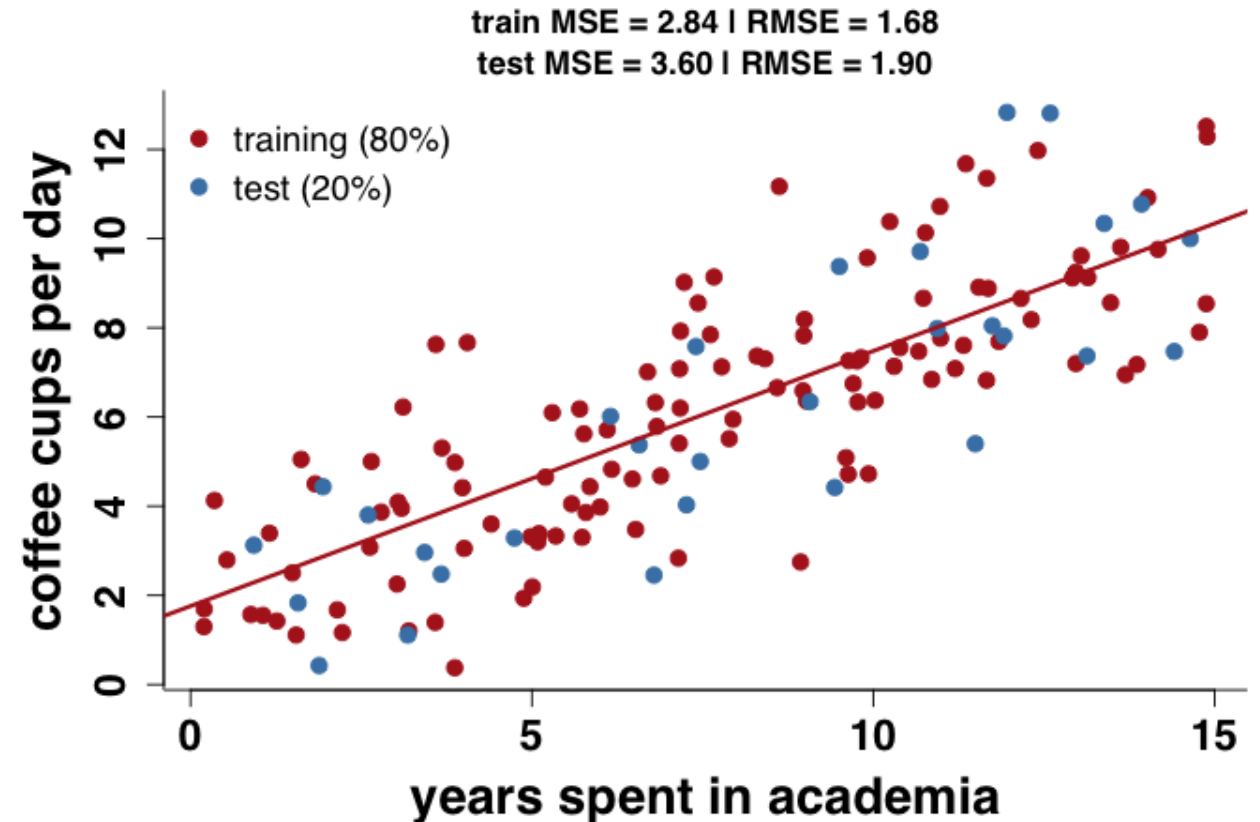


machine learning: testing the model

Assessing model generalisability to unseen data.

how well does a pre-trained model describe unseen data?

- 1) in the **test set** predict outputs from inputs
- 2) measure **prediction accuracy**
- 3) training loss close to testing loss
→ **good model!**

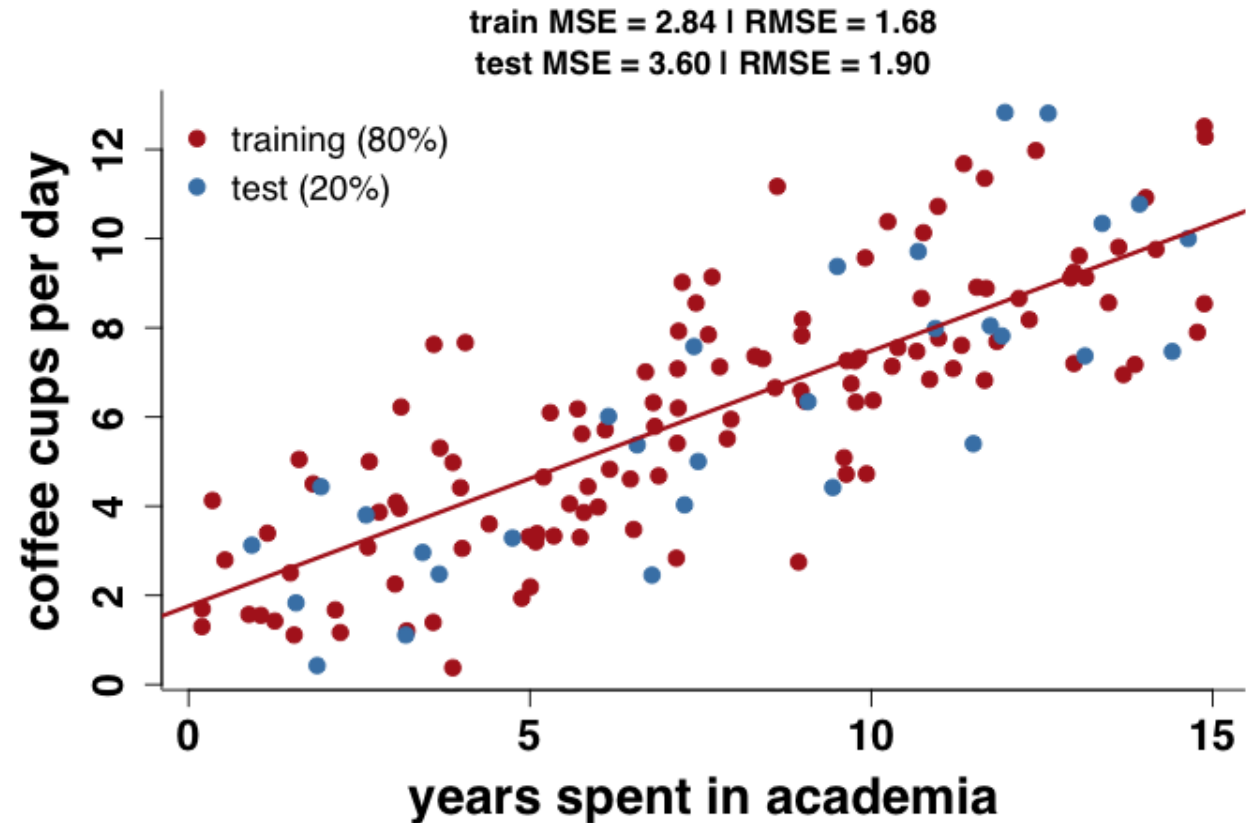


machine learning: testing the model

Assessing model generalisability to unseen data.

how well does a pre-trained model describe unseen data?

- 1) in the **test set** predict outputs from inputs
- 2) measure **prediction accuracy**
- 3) training loss close to testing loss
→ **good model!**
- 4) if not: **wrong model** or **overfitting**



Only thinking about coffee by now?
Then it's time for a...

05:00 min break

**stop & think #3: what is the main
conceptual difference between the
statistics and the machine learning
way when learning from data?**

how to build a neural network

Limitations of a single line

& the case for neural networks and deep learning

The key advance of neural networks (NN),
especially deep neural networks (DNN),
is scale.

Limitations of a single line

& the case for neural networks and deep learning

The key advance of neural networks (NN),
especially deep neural networks (DNN),
is scale.

...works incredibly well!

learning from data: the deep learning way

building a neural network from a bunch of lines

the model $y = f[x, \phi]$

learning from data: the deep learning way

building a neural network from a bunch of lines

the model $y = f[x, \phi]$

the line $y = \phi_0 + \phi_1 x$

learning from data: the deep learning way

building a neural network from a bunch of lines

the model $y = f[x, \phi]$

the line $y = \phi_0 + \phi_1 x$

the NN $y = \phi_0 + \phi_1 a[\theta_{10} + \theta_{11} x] + \phi_2 a[\theta_{20} + \theta_{21} x] + \phi_3 a[\theta_{30} + \theta_{31} x]$

$$y = \phi_0 + \phi_1 \mathbf{h}_1 + \phi_2 \mathbf{h}_2 + \phi_3 \mathbf{h}_3$$

learning from data: the deep learning way

building a neural network from a bunch of lines

the model

$$y = f[x, \phi]$$

the line

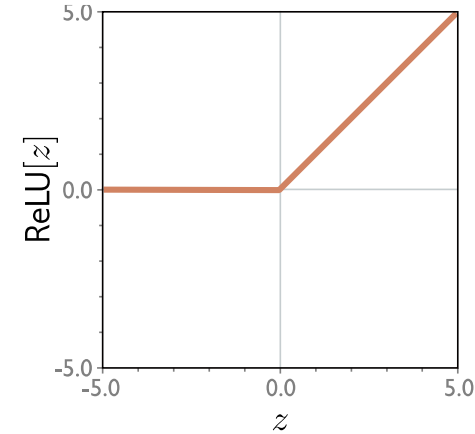
$$y = \phi_0 + \phi_1 x$$

the NN

$$y = \phi_0 + \phi_1 a[\theta_{10} + \theta_{11}x] + \phi_2 a[\theta_{20} + \theta_{21}x] + \phi_3 a[\theta_{30} + \theta_{31}x]$$

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$

activation
function



allows for non-
linearity &
positive output

learning from data: the deep learning way

building a neural network from a bunch of lines

the model

$$y = f[x, \phi]$$

the line

$$y = \phi_0 + \phi_1 x$$

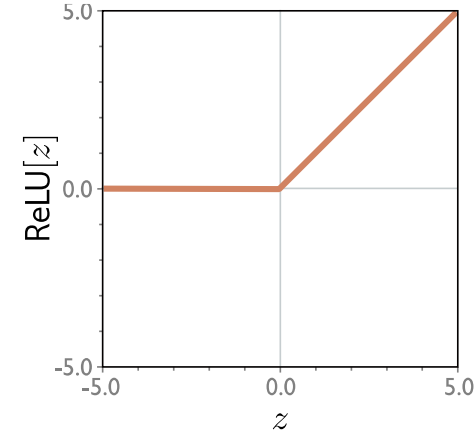
the NN

$$y = \phi_0 + \phi_1 a[\theta_{10} + \theta_{11}x] + \phi_2 a[\theta_{20} + \theta_{21}x] + \phi_3 a[\theta_{30} + \theta_{31}x]$$

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$

one hidden unit
or neuron

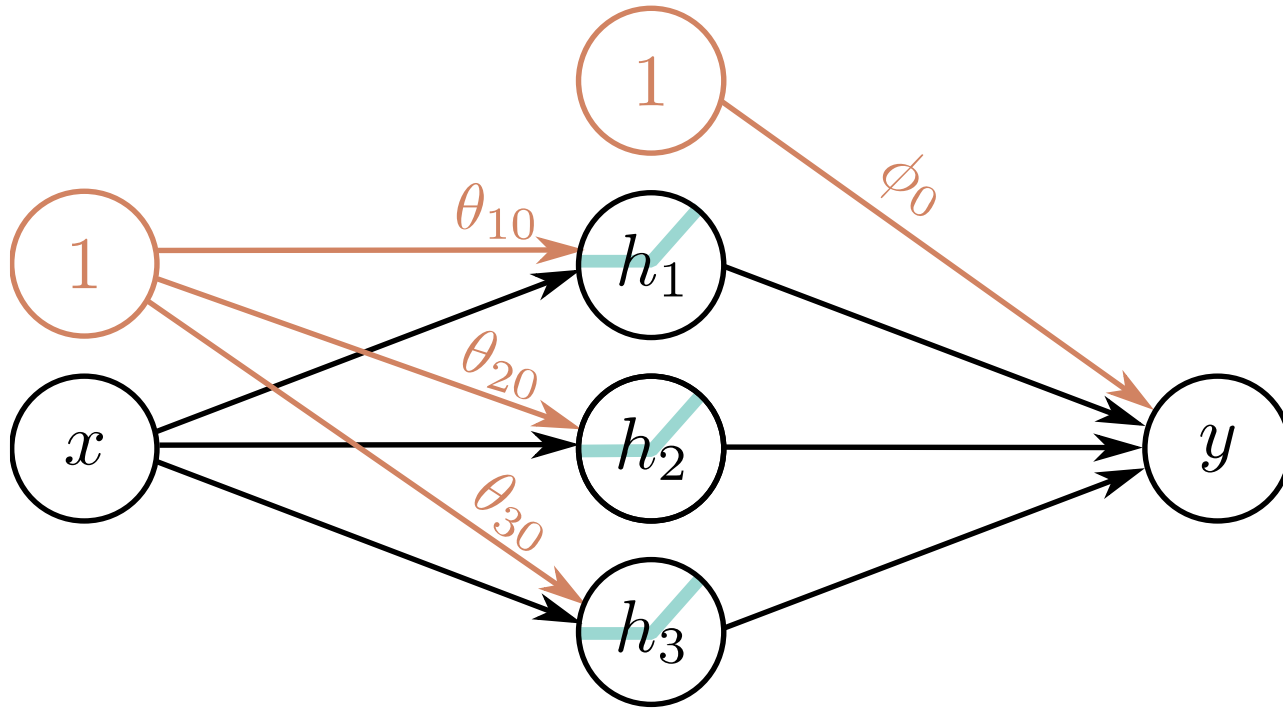
activation
function



allows for non-
linearity &
positive output

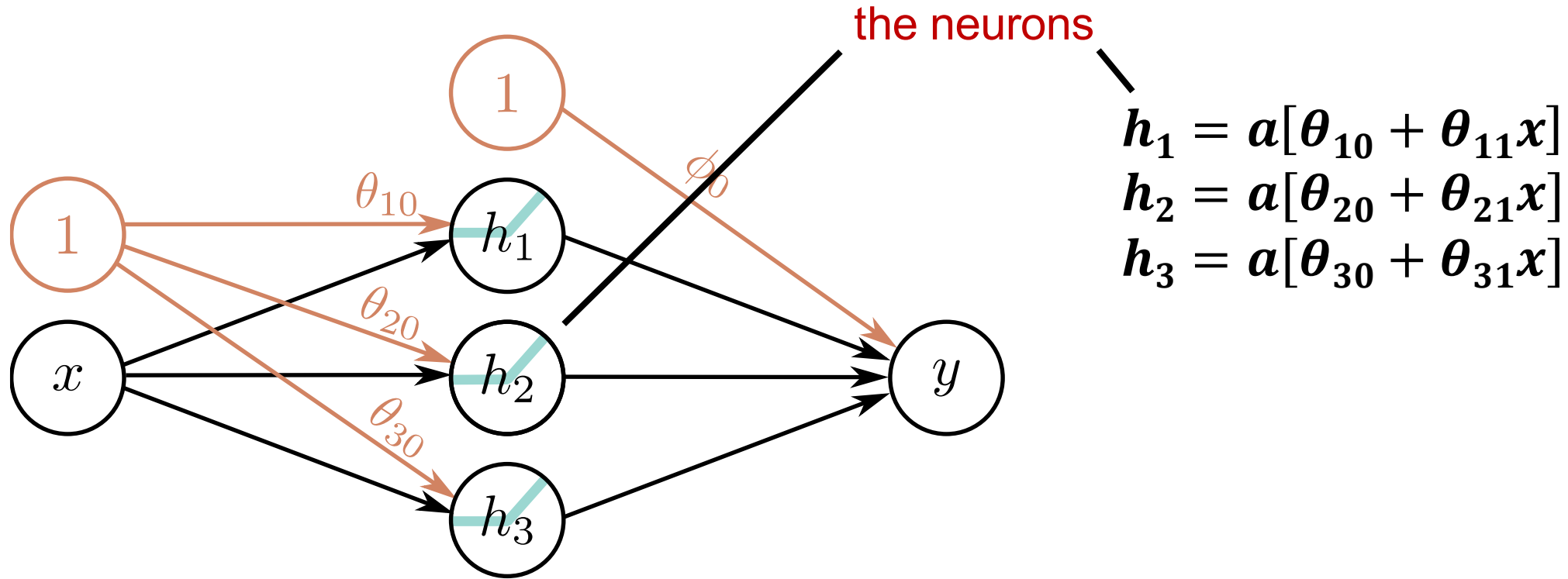
learning from data: the deep learning way

building a neural network from a bunch of lines



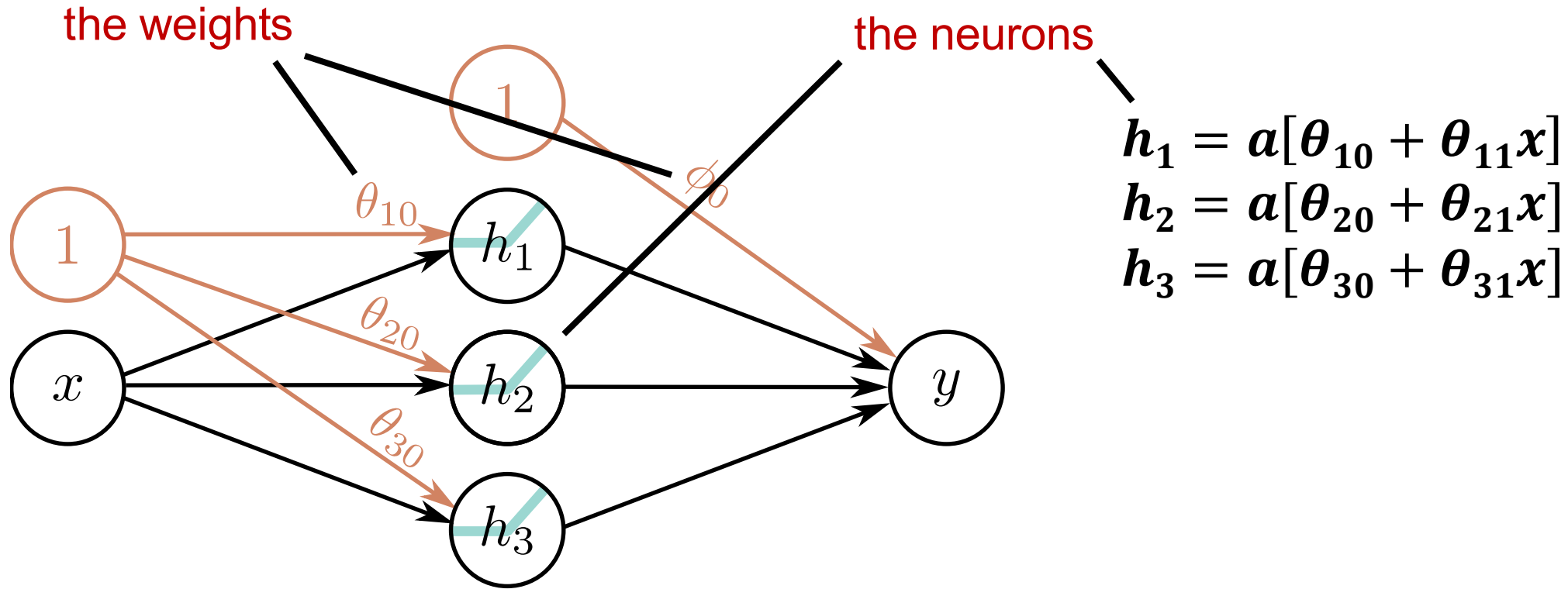
learning from data: the deep learning way

building a neural network from a bunch of lines



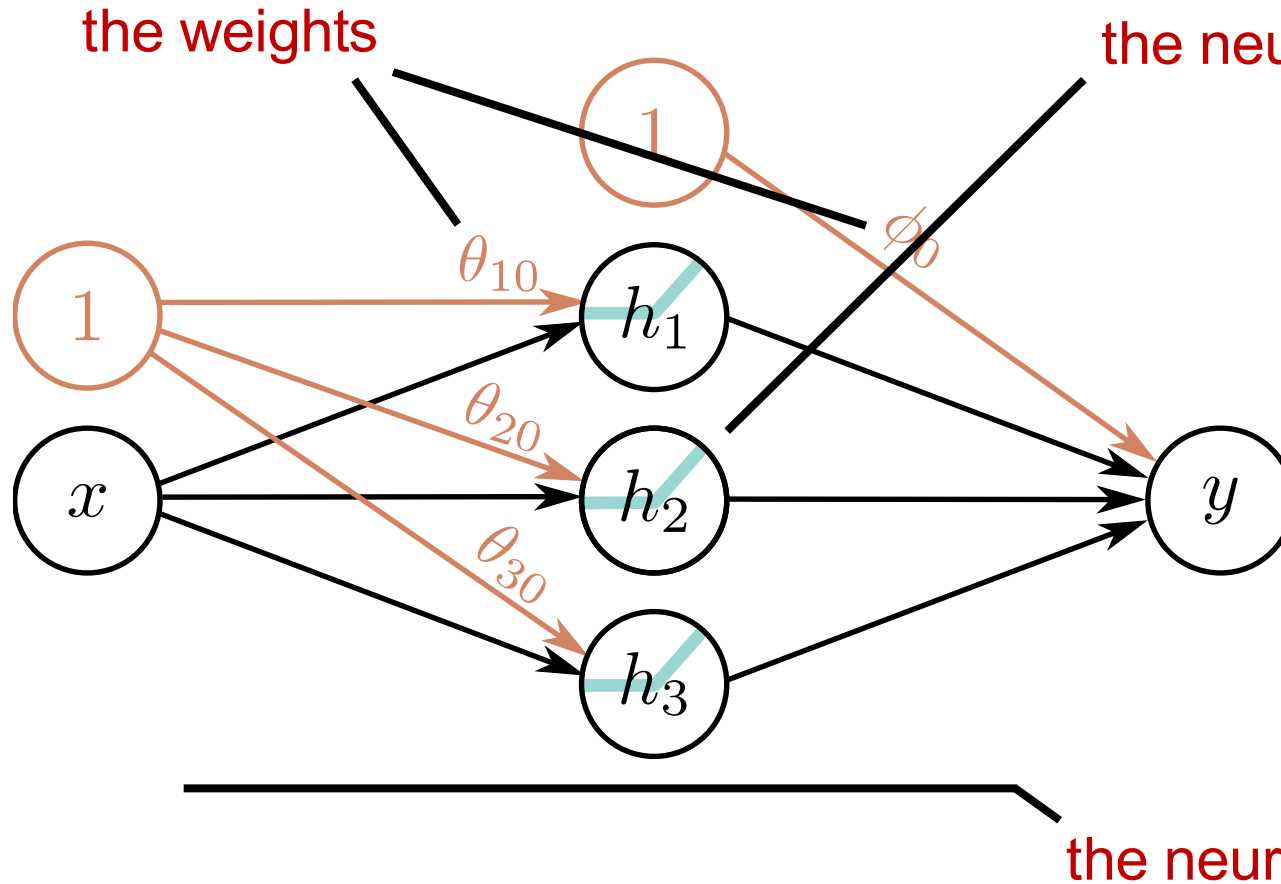
learning from data: the deep learning way

building a neural network from a bunch of lines



learning from data: the deep learning way

building a neural network from a bunch of lines



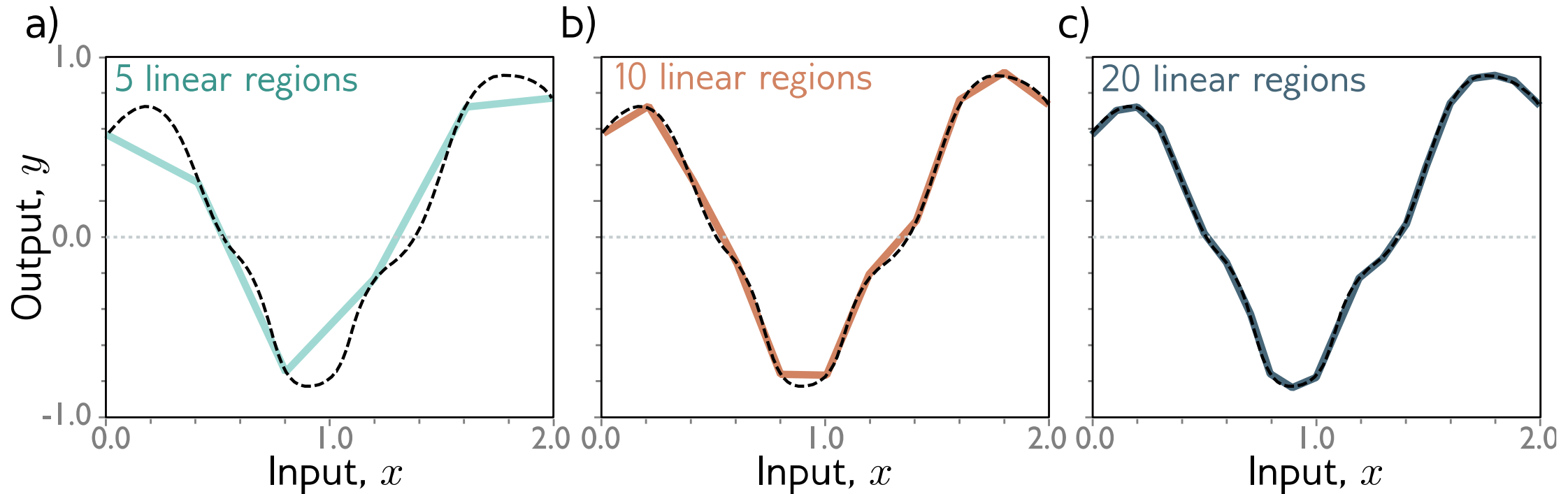
$$h_1 = a[\theta_{10} + \theta_{11}x]$$

$$h_2 = a[\theta_{20} + \theta_{21}x]$$

$$h_3 = a[\theta_{30} + \theta_{31}x]$$

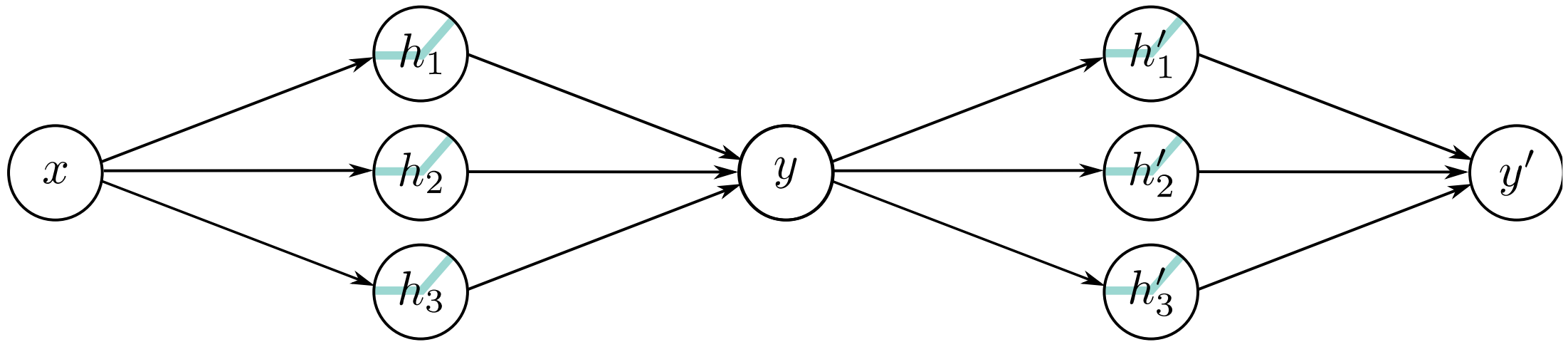
$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$

deep learning: universal approximation theorem

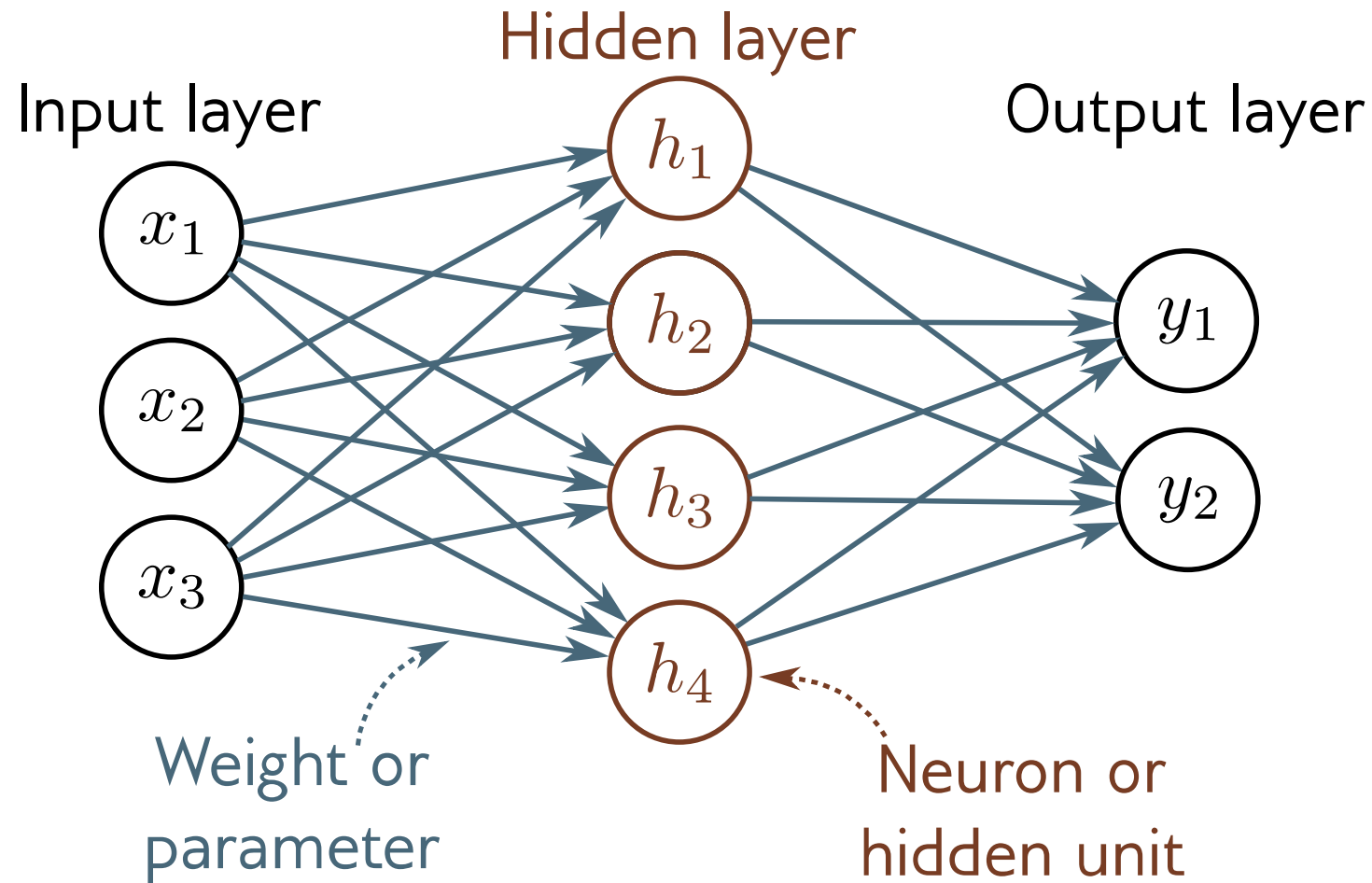


→ with a sufficiently large number of neurons we can describe any continuous function

deep learning: deep neural networks (DNN)



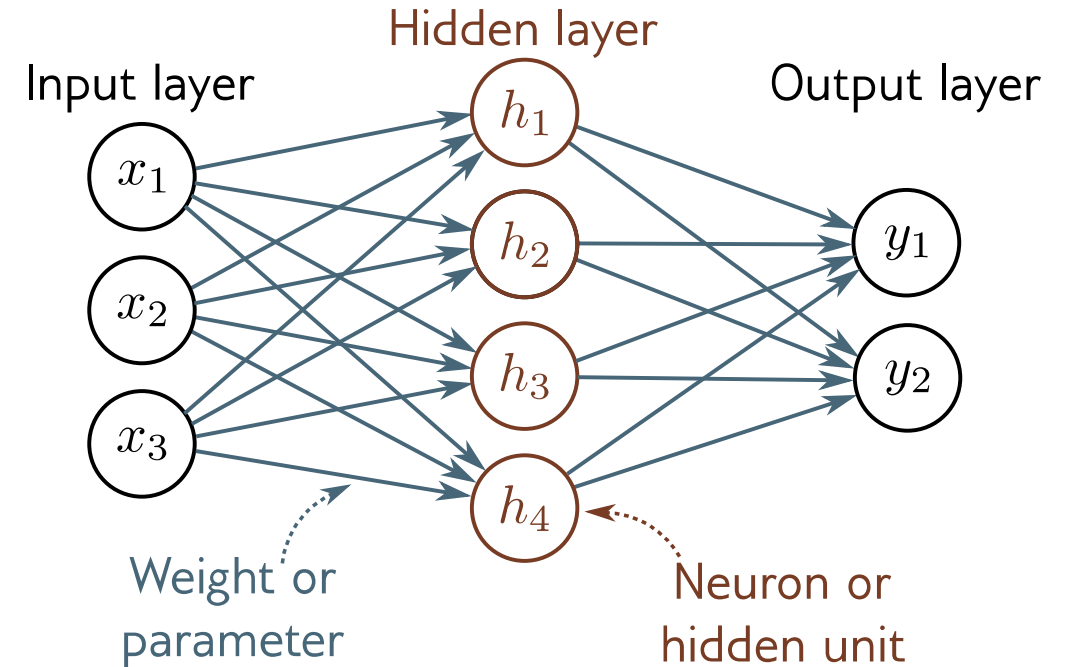
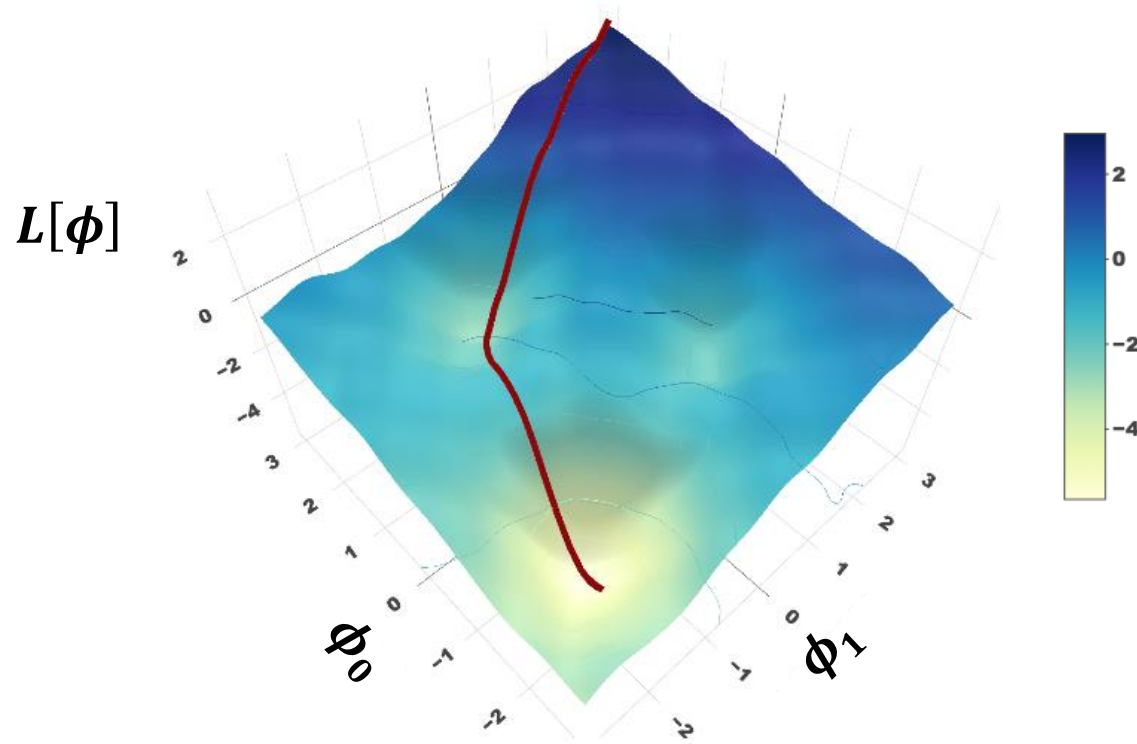
deep learning: multiple I/O



**stop & think #4: what differentiates a
deep learning model from simpler
machine learning models?**

deep learning: training the model

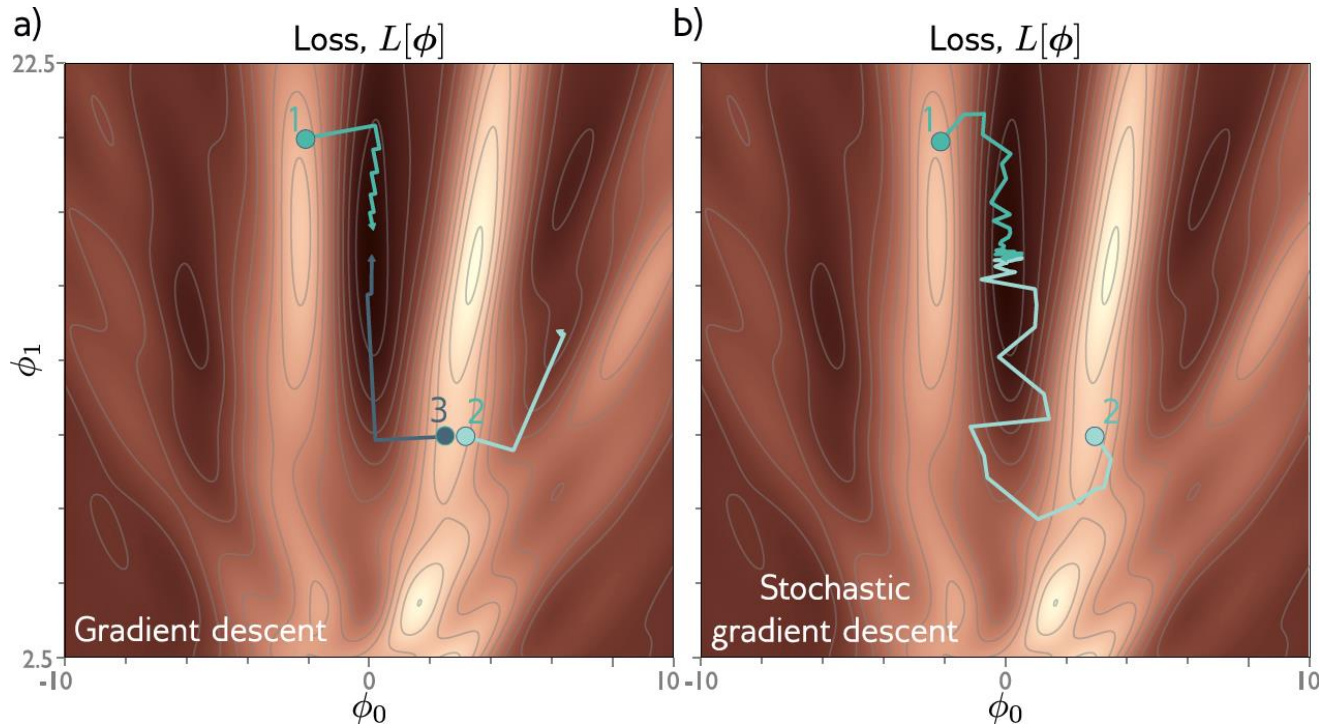
minimizing the loss function for heavily overparameterized deep neural networks



how do we find
optimal values
for these?

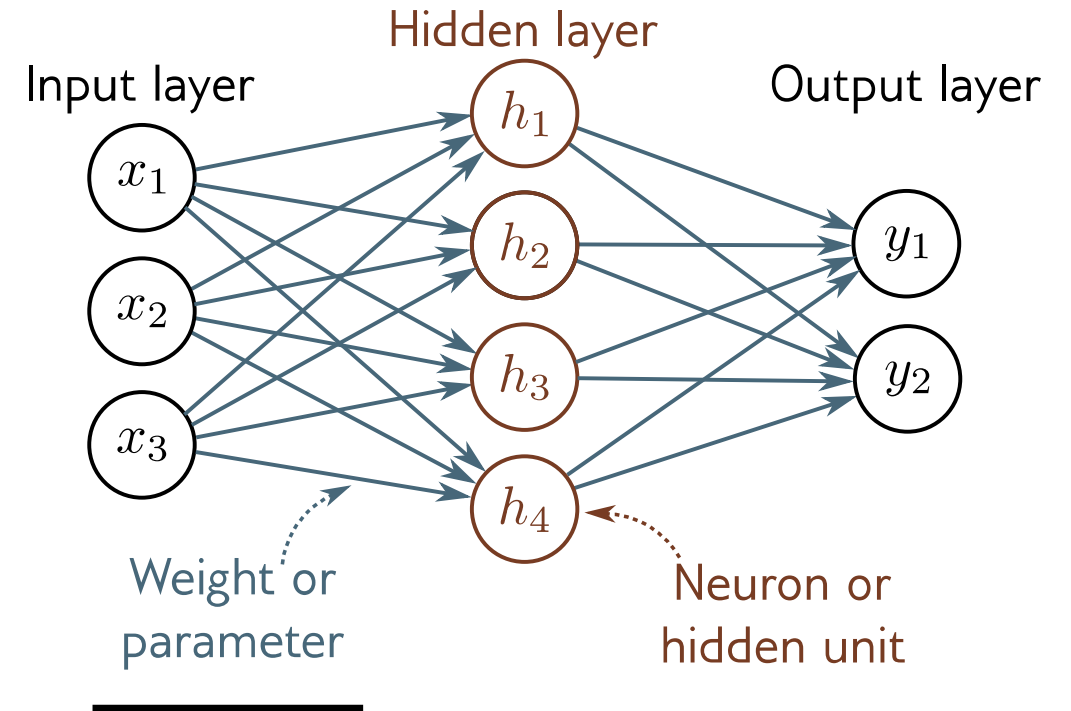
deep learning: training the model

minimizing the loss function for heavily overparameterized deep neural networks



minimum in gradient descent depends entirely on the starting point (esp. big parameter spaces).

→ stochastic gradient descent

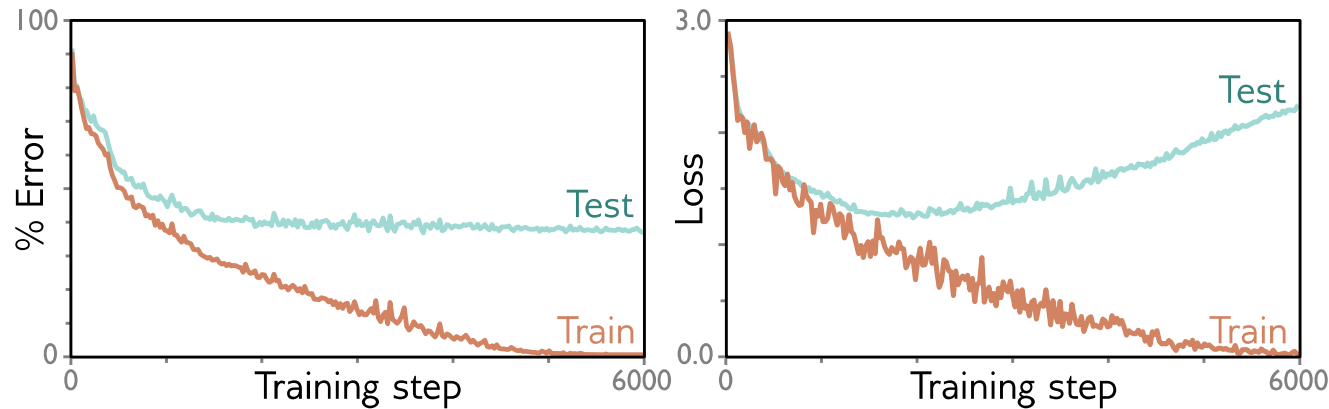


how do we find optimal values for these?

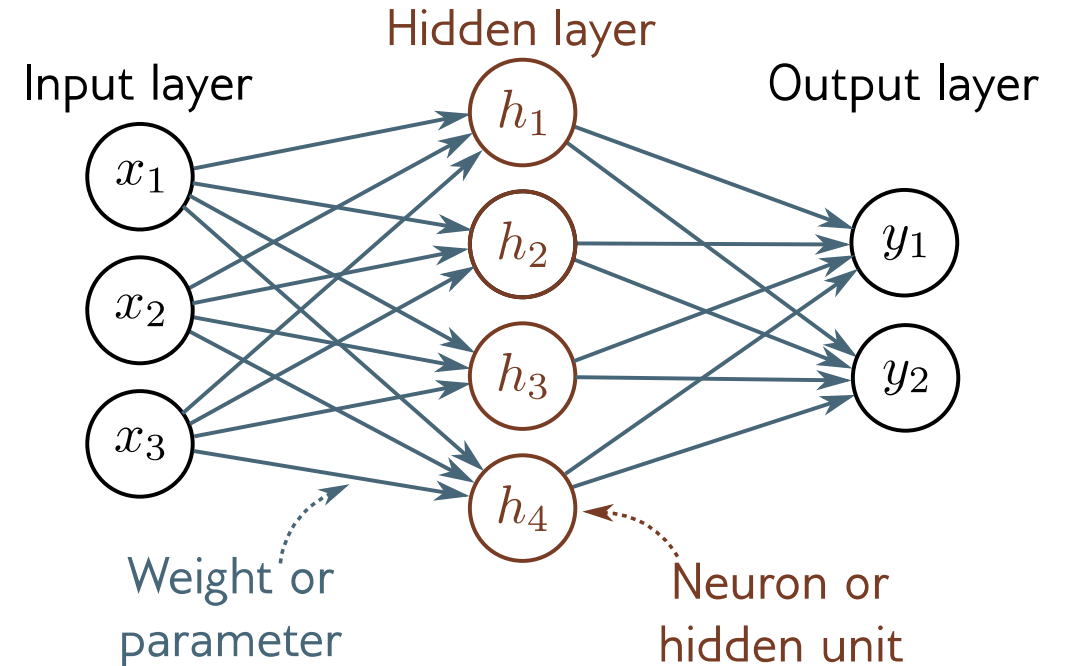
deep learning: model performance

using the same simple logic of test/train split as in our previous examples

training →



- typical #epochs < 1000
- if loss „goes up“ in later epochs, sign of overfitting:
 - model memorises ‚noise‘ in training set
→ early stopping/regularization



deep learning: how to do it

doing deep learning in practice is no big deal

the 'holy trinity':



→ the language: accessible & performant

deep learning: how to do it

doing deep learning in practice is no big deal

the 'holy trinity':



→ the language: accessible & performant



→ the library: vast ecosystem

deep learning: how to do it

doing deep learning in practice is no big deal

the 'holy trinity':



→ the language: accessible & performant



→ the library: vast ecosystem



→ the notebook: easy to use & share

deep learning: how to do it

doing deep learning in practice is no big deal

the 'holy trinity':



→ the language: accessible & performant



→ the library: vast ecosystem



→ the notebook: easy to use & share



deep learning: how to do it

doing deep learning in practice is no big deal

the 'holy trinity':



→ the language: accessible & performant



→ the library: vast ecosystem



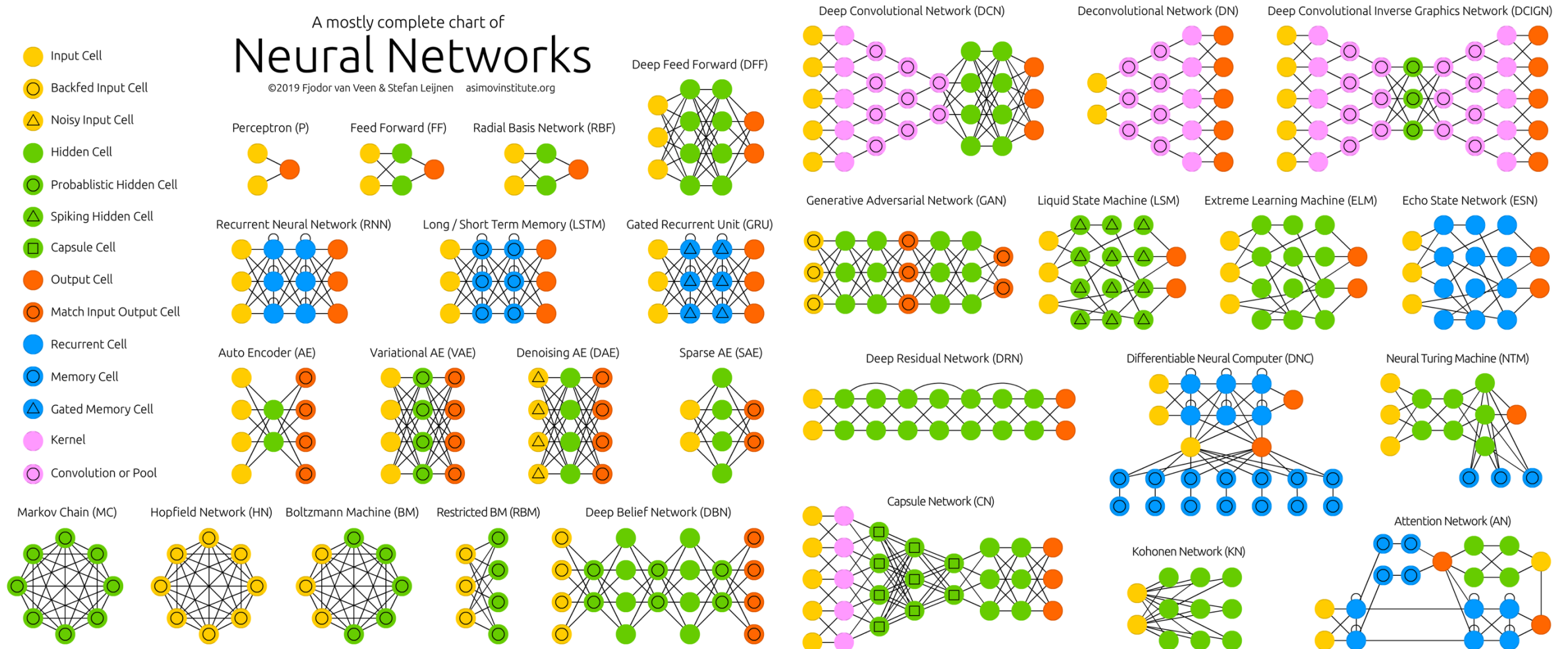
→ the notebook: easy to use & share



sinner or saint?

deep learning: model architectures

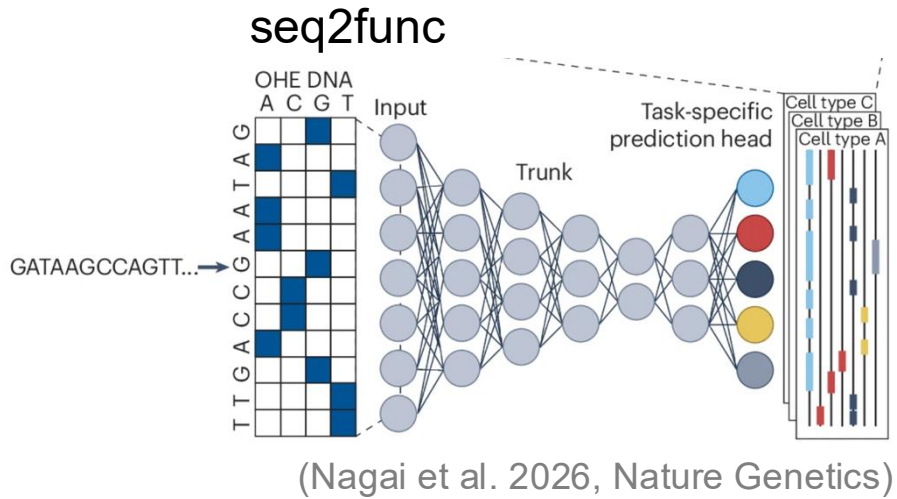
DNN architectures are made of simple, reoccurring building blocks



(Fjodor Van Veen, "The Neural Network Zoo", asimovinstitute.org)

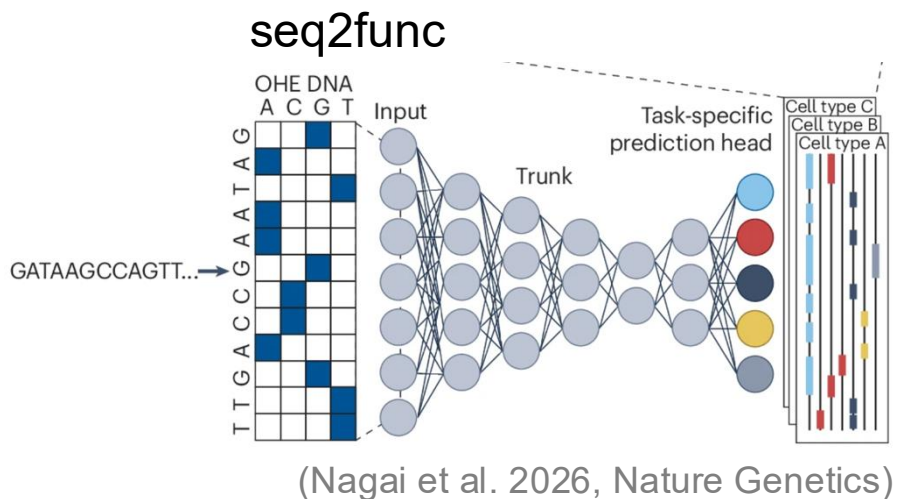
deep learning: model architectures

DNN architectures are made of simple, reoccurring building blocks

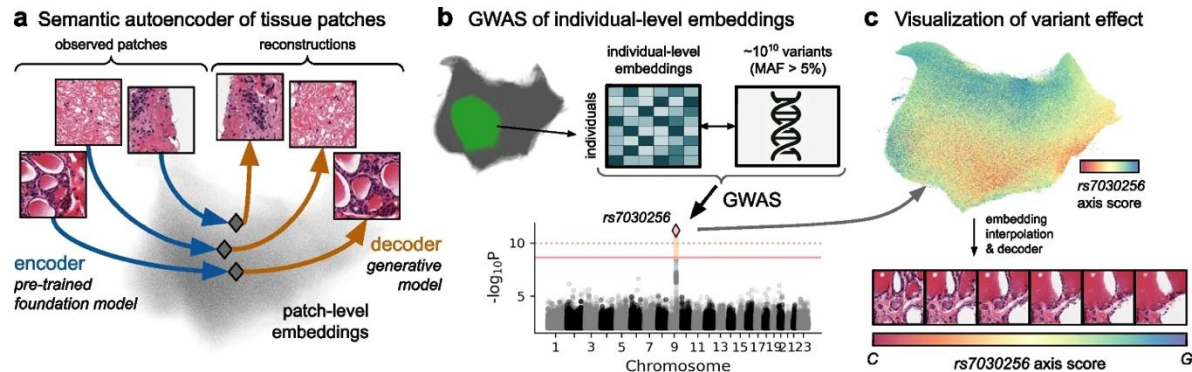


deep learning: model architectures

DNN architectures are made of simple, reoccurring building blocks



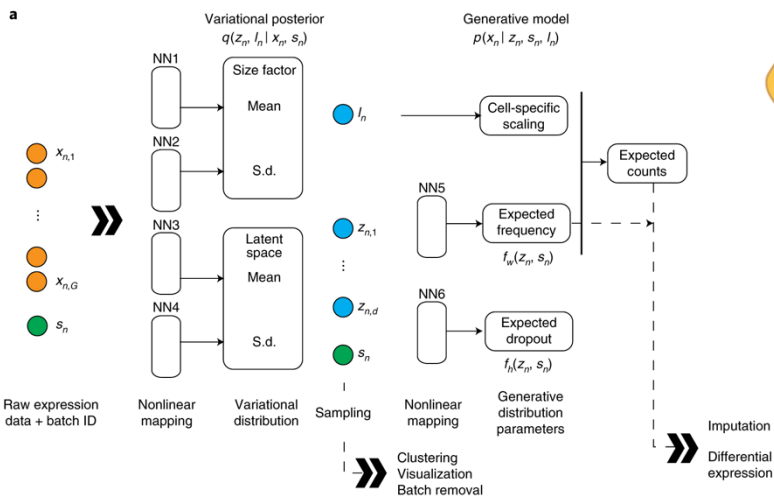
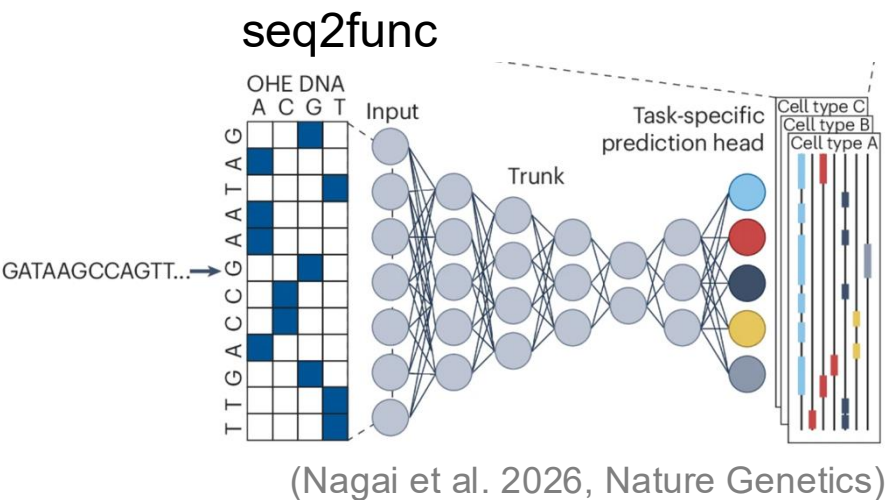
GWAS



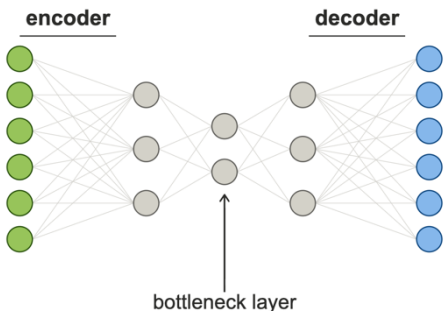
(Chaudhary et al. 2026 , Genome Biology)

deep learning: model architectures

DNN architectures are made of simple, reoccurring building blocks

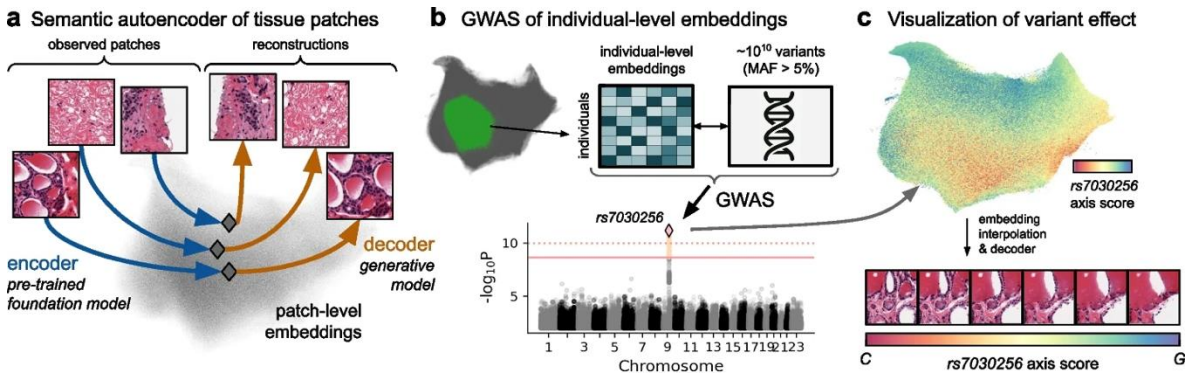


single cell data integration



(Lopez et a. 2018, Nature Methods)

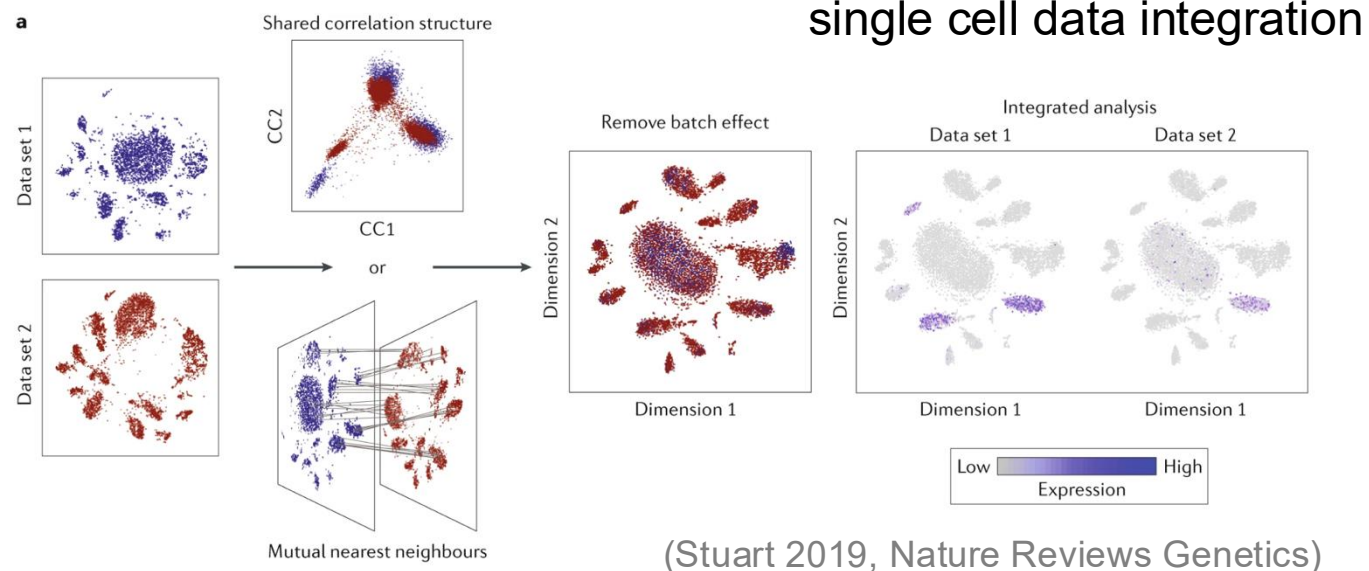
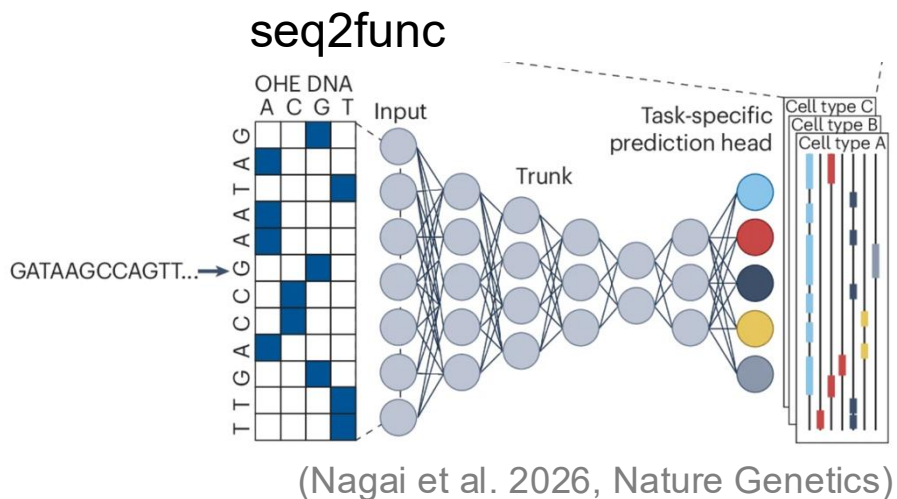
GWAS



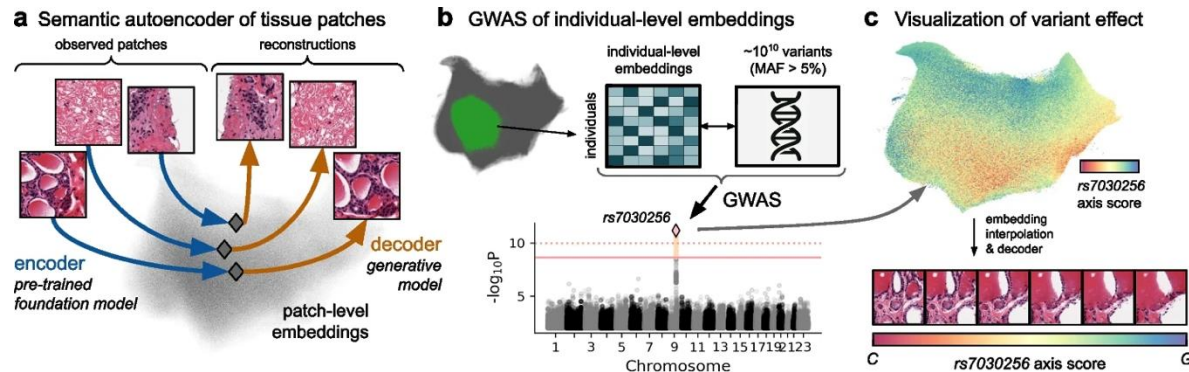
(Chaudhary et al. 2026 , Genome Biology)

deep learning: model architectures

DNN architectures are made of simple, reoccurring building blocks

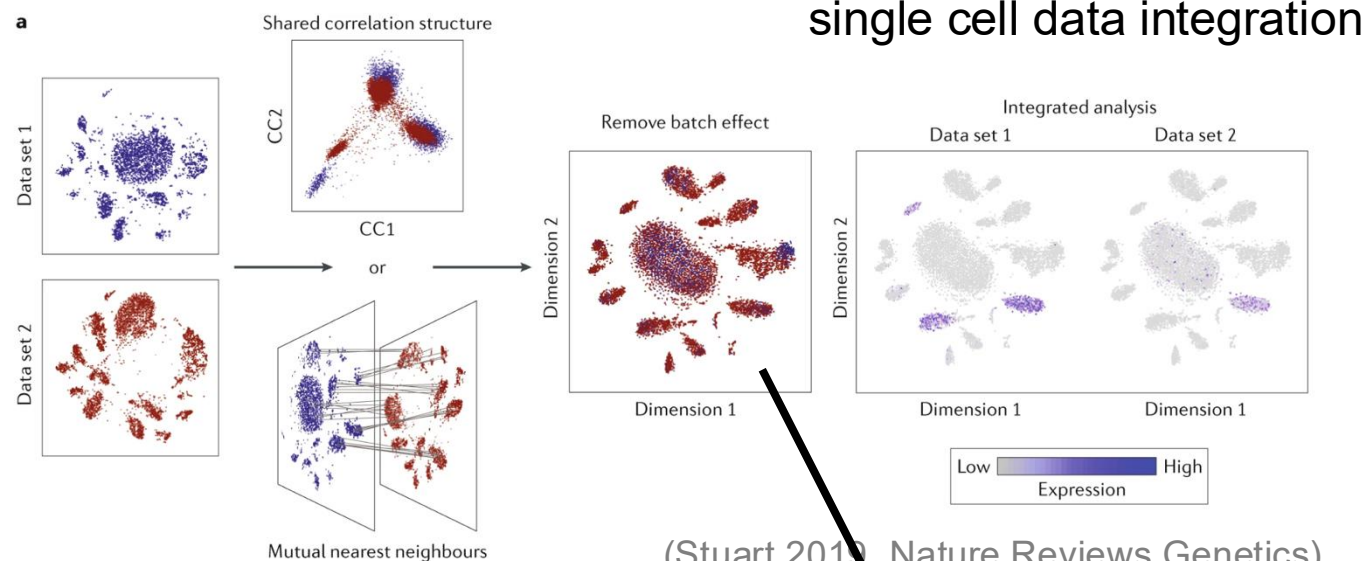
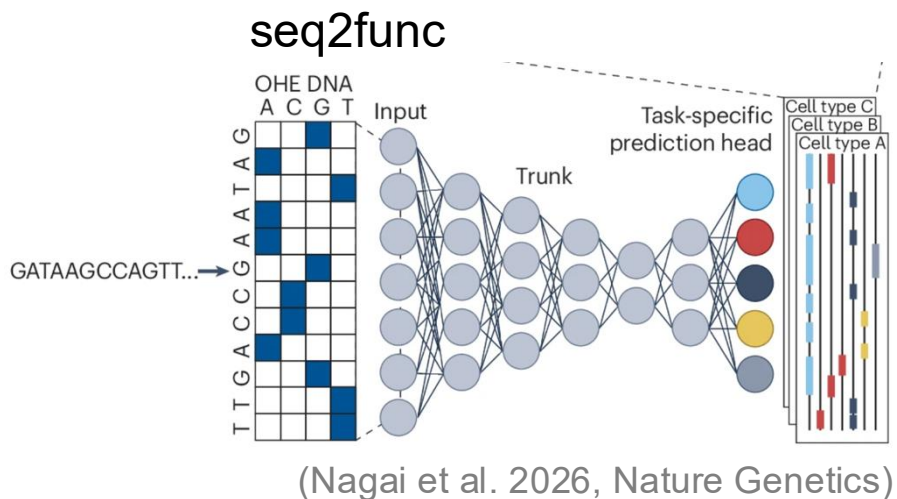


GWAS

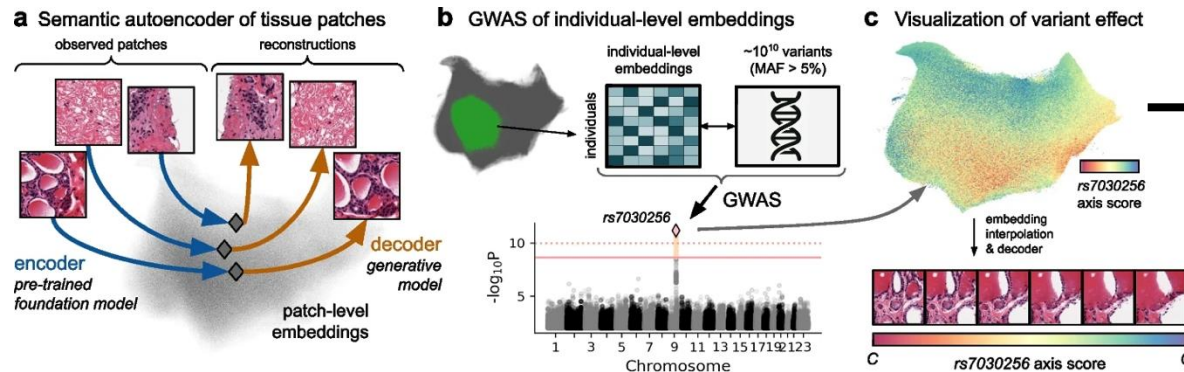


deep learning: model architectures

DNN architectures are made of simple, reoccurring building blocks



GWAS

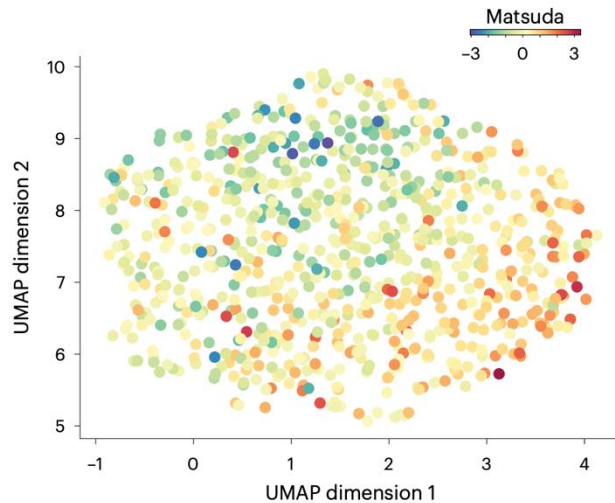
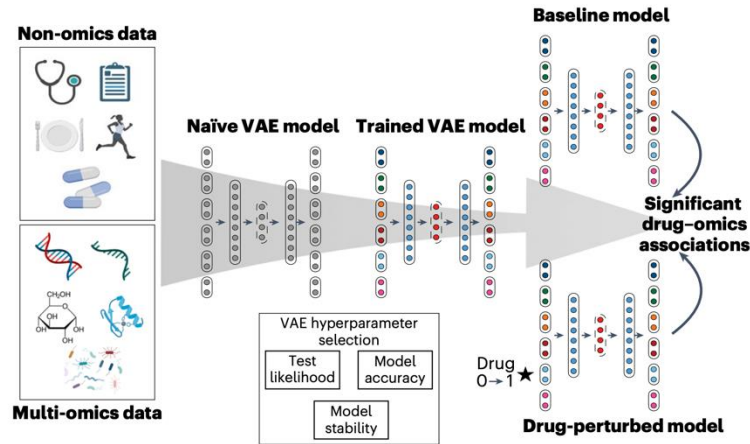


UMAPs

deep learning: model architectures

DNN architectures are made of simple, reoccurring building blocks

population-level multi-omics



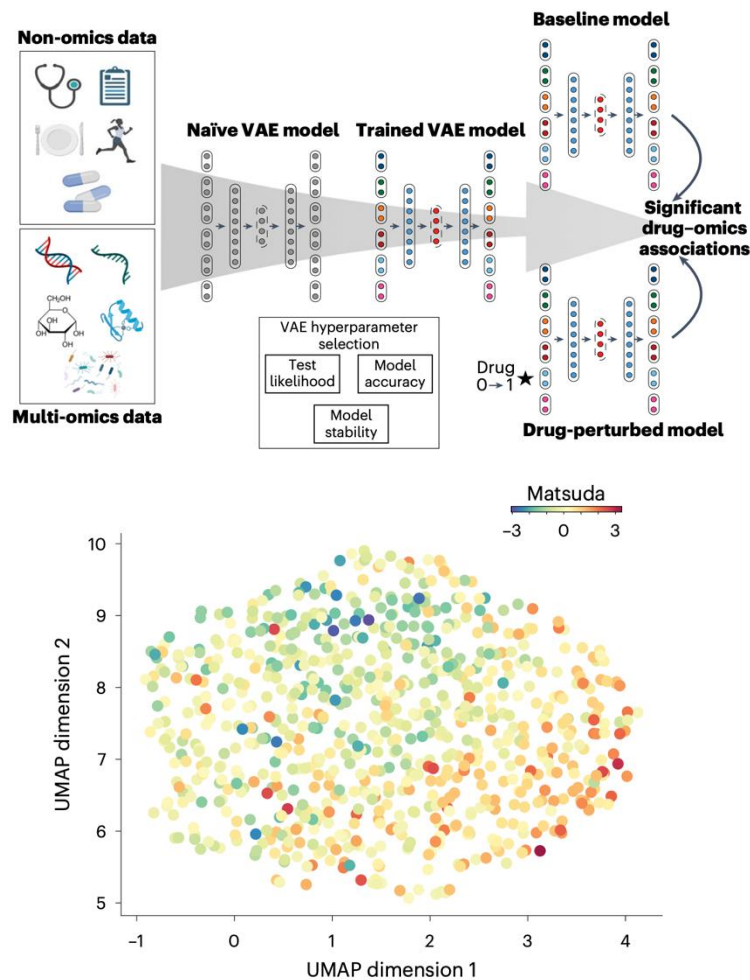
(Allesøe et al. 2023, Nature Biotechnology)

deep learning: model architectures



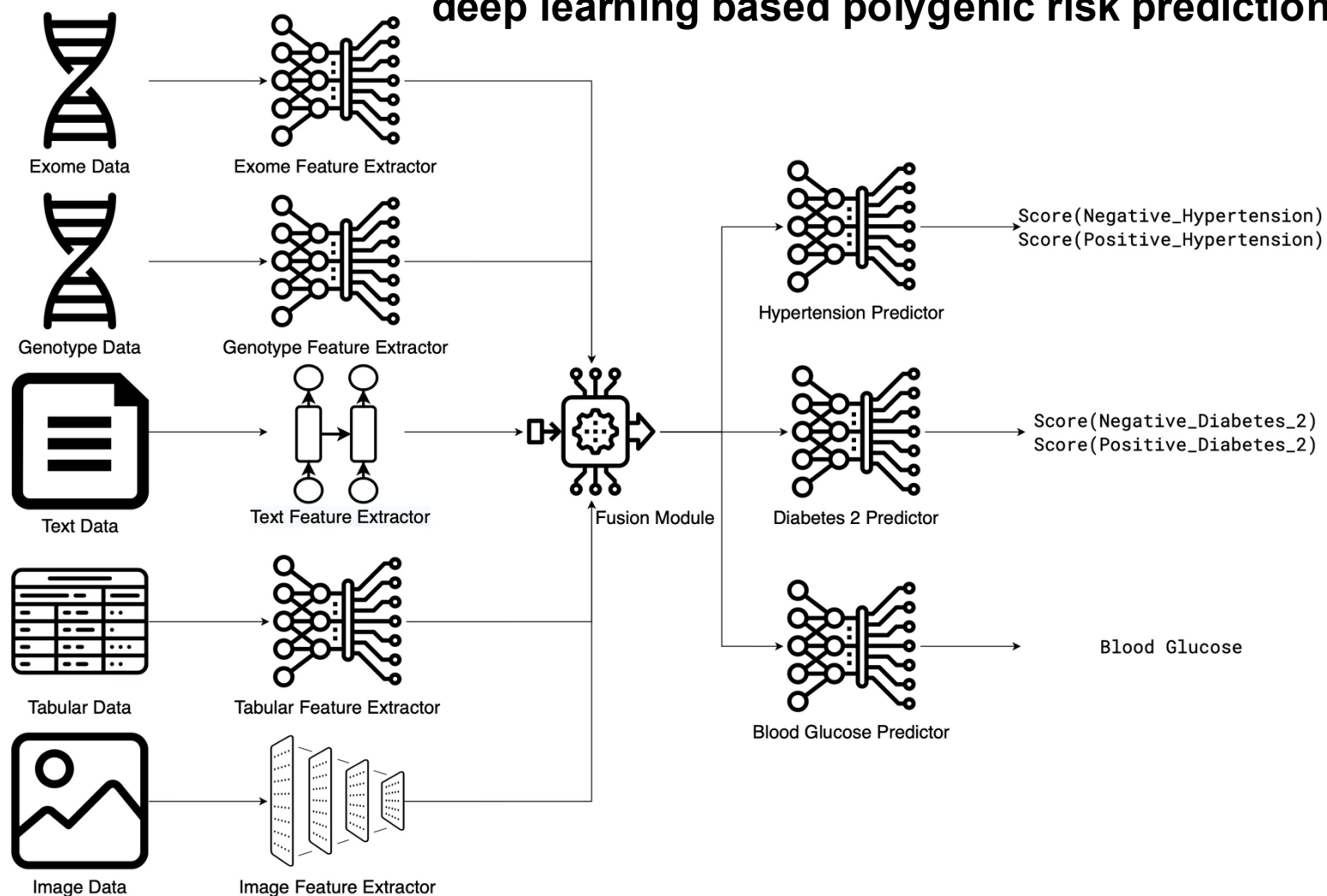
DNN architectures are made of simple, reoccurring building blocks

population-level multi-omics



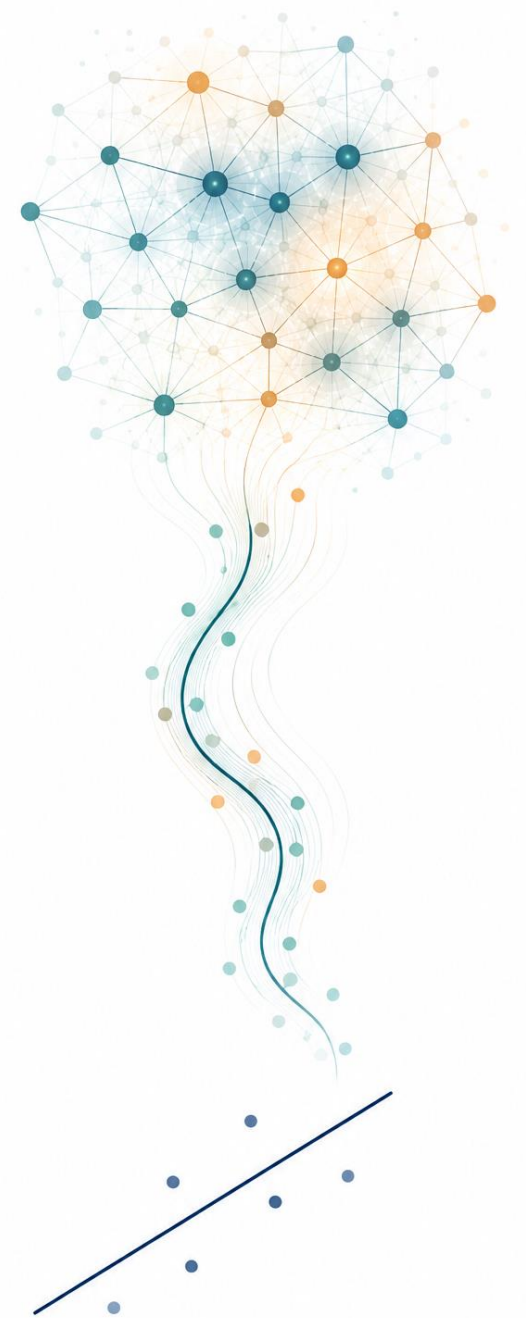
(Allesøe et al. 2023, Nature Biotechnology)

deep learning based polygenic risk prediction



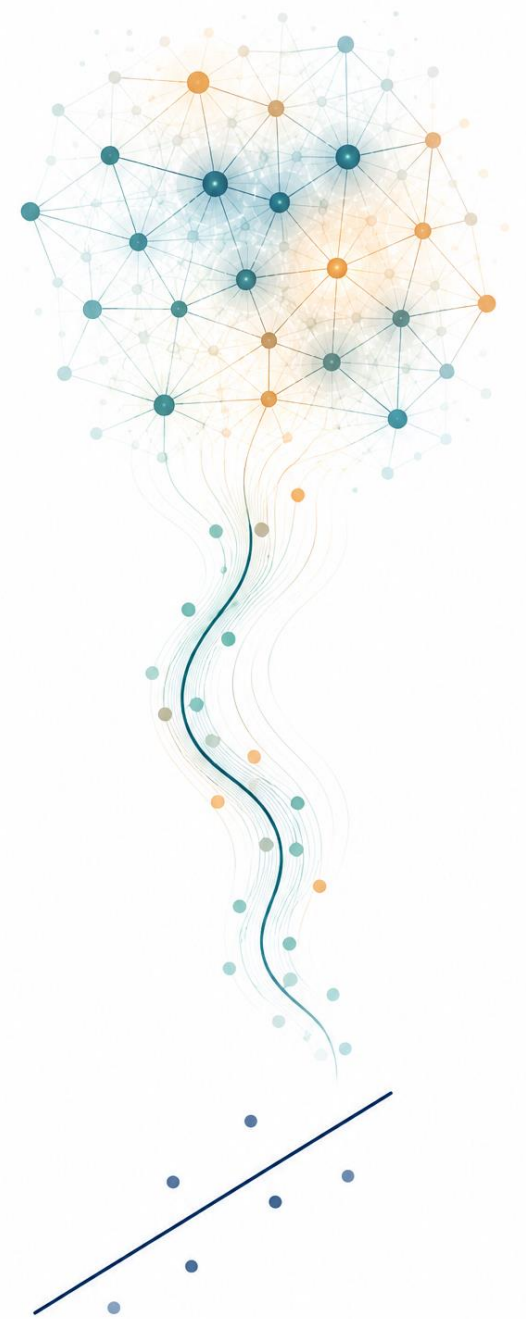
(Sigurðsson et al. 2023, Nucleic Acids Research)

Summary



Summary

- conventional GWAS/PGS methods do not capture the full spectrum of genetic associations.



Summary

- conventional GWAS/PGS methods do not capture the full spectrum of genetic associations.
- statistics & machine learning differ in their philosophies not in their methods.



Summary

- conventional GWAS/PGS methods do not capture the full spectrum of genetic associations.
- statistics & machine learning differ in their philosophies not in their methods.
- we have moved from GWAS to Deep Learning simply by fitting a line.



Summary

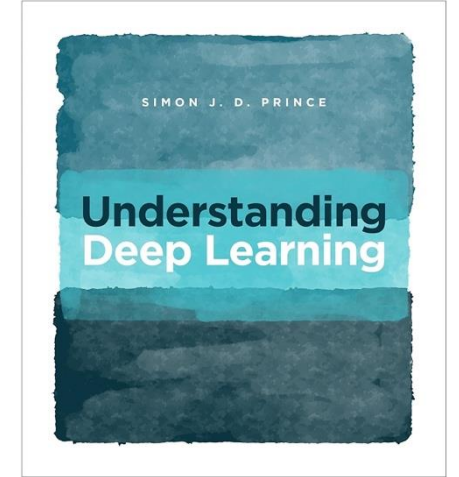
- conventional GWAS/PGS methods do not capture the full spectrum of genetic associations.
- statistics & machine learning differ in their philosophies not in their methods.
- we have moved from GWAS to Deep Learning simply by fitting a line.
- very large DNNs with thousands of parameters can be trained on basically any type of data → including **population-scale multi-omics & PGS**



References & further reading:

resources:

- [*An Owner's Guide to the Human Genome: an introduction to human population genetics, variation and disease](#) by Jonathan Pritchard
- [An Introduction to Statistical Learning](#) by James et al.
- [*Understanding Deep Learning](#) by Simon J.D. Prince
- [*data science courses on kaggle learn](#)
- [EIR-FM repository on github](#)
- Micheal Bronstein's [Biology 2.0 post](#)



*** =: especially recommended**

references:

- Genetic and molecular architecture of complex traits, Lappalainen et al. 2024, Cell
- Clinical use of polygenic risk scores: current status, barriers and future directions, Kullo 2025, Nature Reviews Genetics
- Statistical Modelling: The Two Cultures, Breiman 2001, Statistical Science
- The Neural Network Zoo, Van Veen, asimovinstitute.org
- Toward generalizable and interpretable AI in regulatory genomics, Nagai et al. 2026, Nature Genetics
- HistoGWAS: an AI-enabled framework for automated genetic analysis of tissue phenotypes in histology cohorts, Chaudhary et al. 2026, Genome Biology
- Deep generative modeling for single-cell transcriptomics, Lopez et al. 2018, Nature Methods
- Integrative single-cell analysis, Stuart and Satija 2019, Nature Reviews Genetics
- Discovery of drug–omics associations in type 2 diabetes with generative deep-learning models, Allesøe et al. 2023, Nature Biotechnology
- Deep integrative models for large-scale human genomics, Sigurðsson et al. 2023, Nucleic Acids Research

The Rasmussen lab



Novo Nordisk Foundation
**CENTER FOR
BASIC
METABOLIC
RESEARCH**



UNIVERSITY OF
COPENHAGEN



**NOVO NORDISK
FOUNDATION CENTER**
FOR GENOMIC MECHANISMS OF DISEASE

AT BROAD INSTITUTE

JOACHIM
HERZ
STIFTUNG



questions/contact:

simon.wengert@sund.ku.dk

**And now it's your turn to
implement all of this
manually and from scratch!**

The Rasmussen lab



Novo Nordisk Foundation
**CENTER FOR
BASIC
METABOLIC
RESEARCH**



UNIVERSITY OF
COPENHAGEN



**NOVO NORDISK
FOUNDATION CENTER**
FOR GENOMIC MECHANISMS OF DISEASE

AT BROAD INSTITUTE



Arnór I.
Sigurdsson



Jiyeon
Min

Thank you!

appendix

learning from data: the statistics way

Fitting a line to data

response variable

explanatory variable

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x + \varepsilon$$

first parameter (intercept) second parameter (slope) error term

In statistical inference we optimise the model parameters on the full sample such that they describe the data best.

Parameter optimization with OLS:

$$\varepsilon = y - \hat{y}$$

$$RSS = \varepsilon^2$$

$$RSS = (y - \hat{\beta}_0 - \hat{\beta}_1 x)^2$$

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

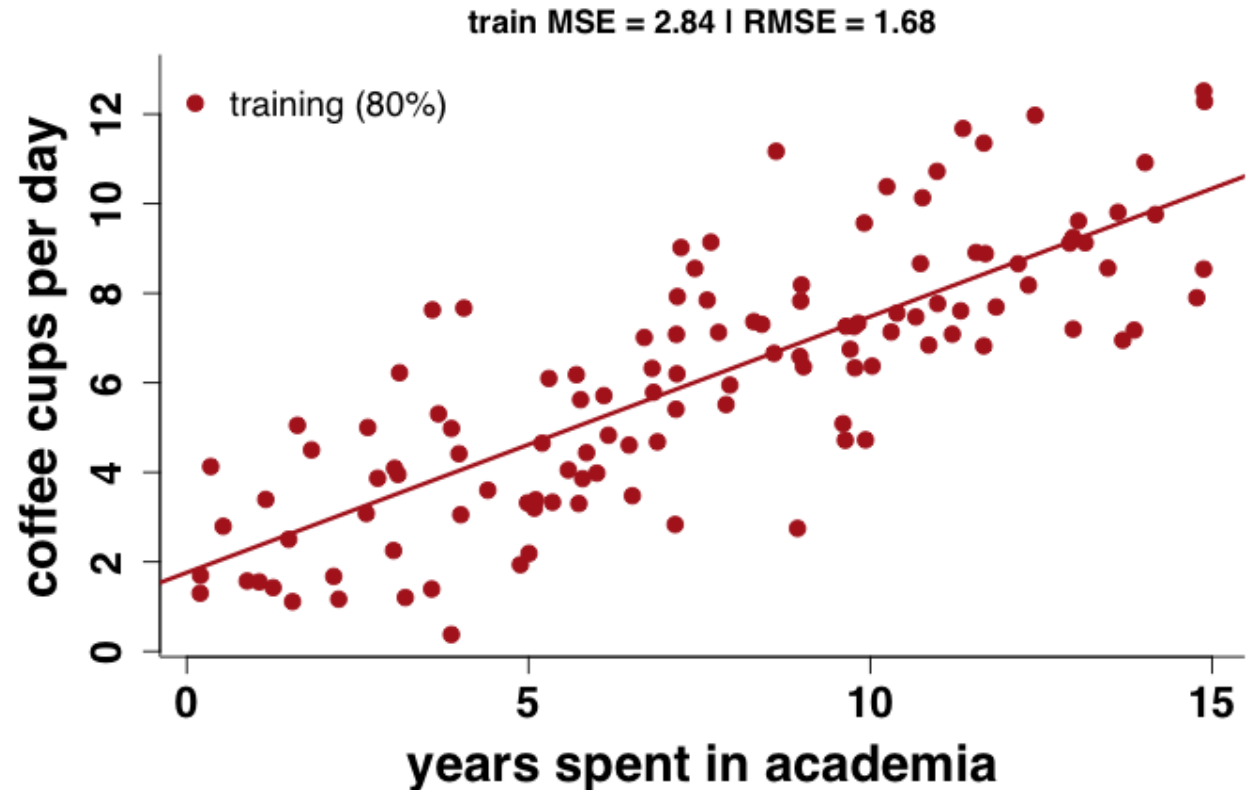
$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$$

learning from data: the machine learning way

Can we learn from the data more autonomously and predict unseen measurements?

- 1) split the data: training & test set
- 2) fit machine learning model to the **training set**
- 3) measure **prediction accuracy**

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$



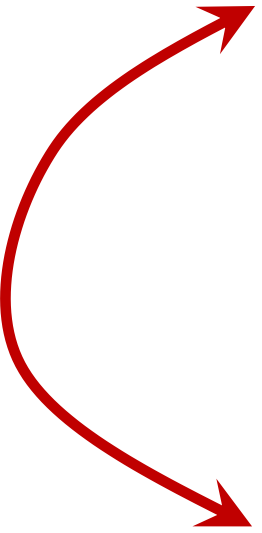
More general & a slight change in notation

Bridging the gap: statistics & machine learning evolved different notations for the same things...

the model

$$y = \phi_0 + \phi_1 x$$

$$\phi = \begin{bmatrix} \phi_0 \\ \phi_1 \end{bmatrix} \begin{array}{l} \text{--- 'intercept'} \\ \text{--- 'slope'} \end{array}$$


$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x + \varepsilon$$

the loss function

$$\begin{aligned} L[\phi] &= \sum_{i=1}^I (f[x_i, \phi] - y_i)^2 \\ &= \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2 \end{aligned}$$

this is the **exact same function** (RSS)
we used to learn $\hat{\beta}_0$ and $\hat{\beta}_1$ in our
previous example

Limitations of a single line

& the case for neural networks and deep learning

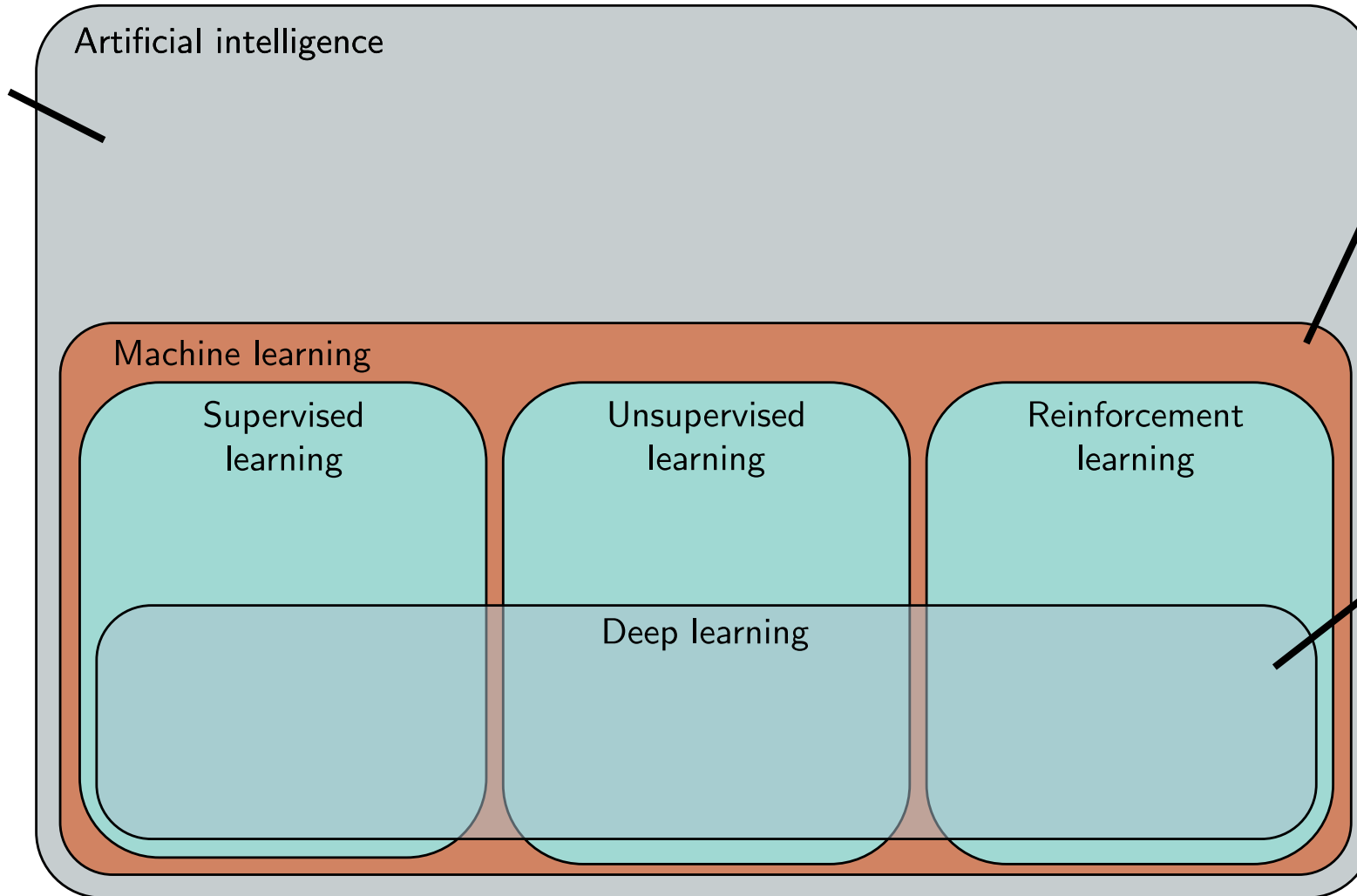
simple models are surprisingly effective (occam's razor/model parsimony).

BUT:

- limited by model assumptions → „everything is a line“
 - more complex, custom stats-models require a lot of expertise (both technical & subject matter)
 - non-linear relationships possible, but explicitly specified
 - limited through human assumptions (inductive bias)
 - often only 1:1 I/O relationships & uni-modal data
- difficult to find unexpected/unspecified patterns in a purely data-driven way

What is machine learning?

artificial intelligence:
building systems that are able to simulate intelligent behavior (very broad term)

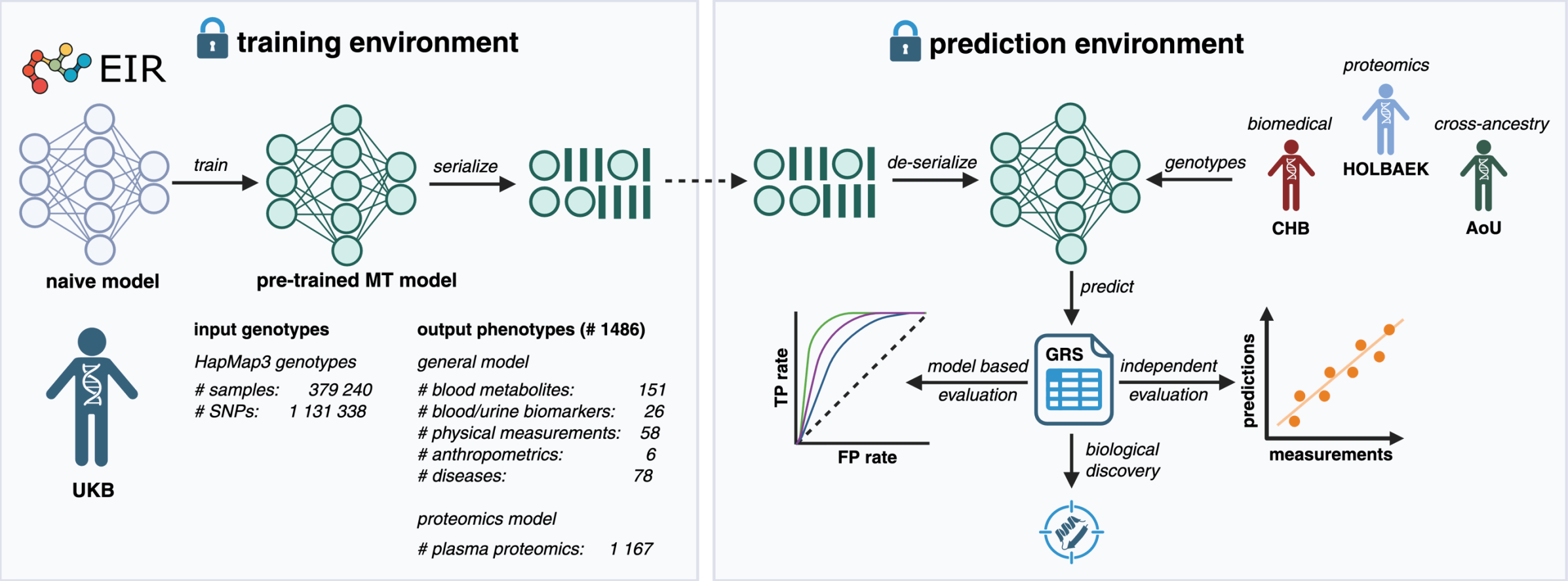


machine learning:
learning to make decisions by fitting models to data

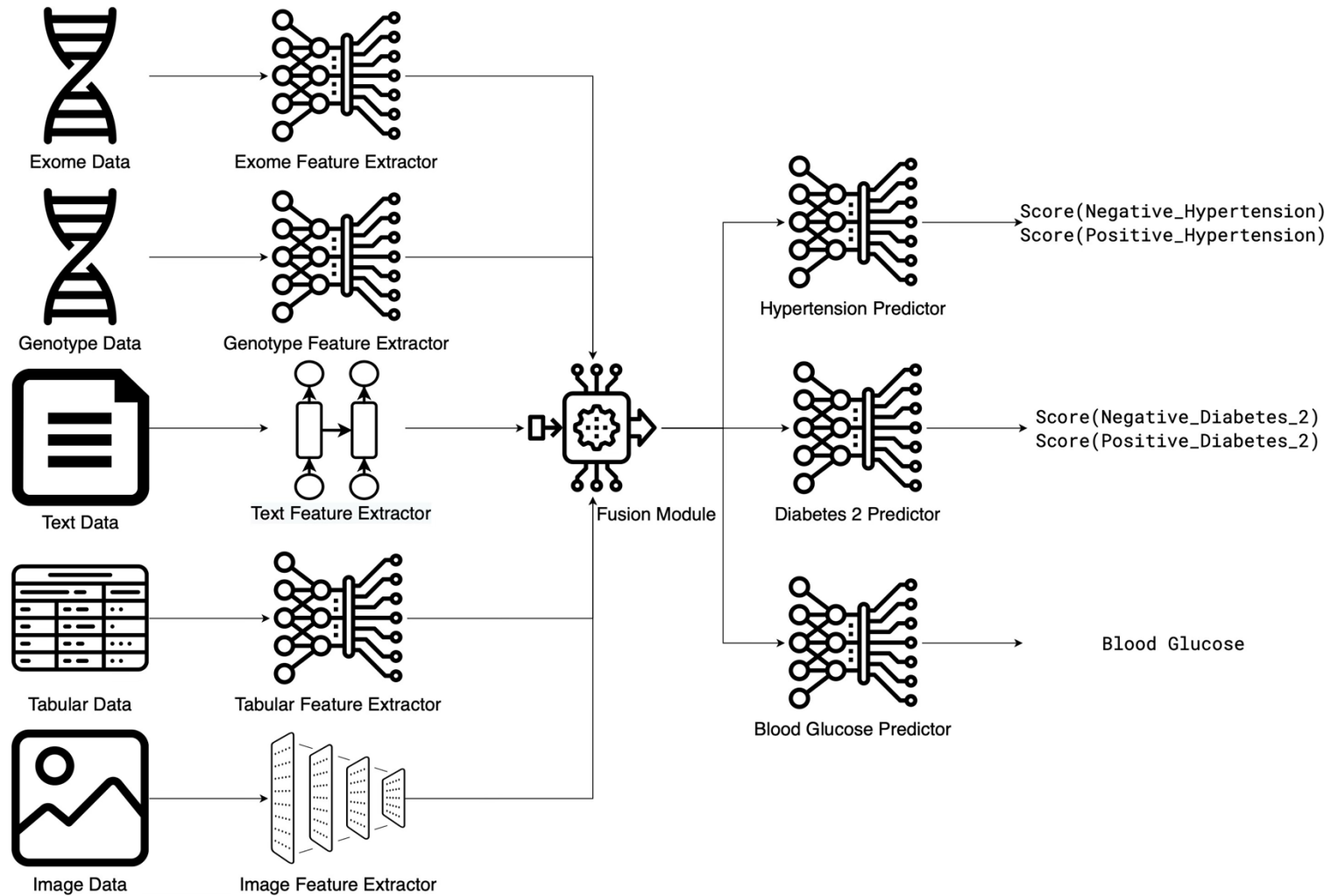
ML $\neq$ AI
→ a subset

deep learning:
a specific way of doing machine learning using a class of models called deep neural networks

EIR-FM: general polygenic representations using DL



EIR-FM model architecture – conceptual



EIR-FM model architecture – building blocks

